

Formal Language and Automata Theory

This book is a part of the course by **Jaipur National University**, Jaipur.
This book contains the course content for Formal Language and Automata Theory.

JNU, Jaipur
First Edition 2013

The content in the book is copyright of **JNU**. All rights reserved.

No part of the content may in any form or by any electronic, mechanical, photocopying, recording, or any other means be reproduced, stored in a retrieval system or be broadcast or transmitted without the prior permission of the publisher.

JNU makes reasonable endeavours to ensure content is current and accurate. **JNU** reserves the right to alter the content whenever the need arises, and to vary it at any time without prior notice.

Index

I. Content.....	II
II. List of Figures	V
III. List of Tables.....	VI
IV. Abbreviations	VII
V. Application.....	117
VI. Bibliography	122
VII. Self Assessment Answers.....	125

Book at a Glance

Contents

Chapter I.....	1
Fundamentals	1
Aim	1
Objectives	1
Learning outcome	1
1.1 Introduction.....	2
1.2 Introduction to Defining Languages	2
1.3 Finite State Machine	3
1.4 Finite Automata Model	3
1.5 Acceptance of Strings and Languages	4
1.6 Deterministic Finite Automata (DFA).....	6
1.7 Non-Deterministic Finite Automata (NFA)	7
1.8 Difference between NFA and DFA	8
Summary.....	10
References	10
Recommended Reading.....	10
Self Assessment.....	11
 Chapter II	 13
Finite Automata.....	13
Aim	13
Objectives	13
Learning outcome	13
2.1 Introduction.....	14
2.2 Finite Automata.....	14
2.3 Significance of Non-Deterministic Finite Automata	14
2.4 NFA with ϵ -Transitions	15
2.4.1 Significance of NFA with ϵ	15
2.4.2 Acceptance of Language	15
2.5 Conversions and Equivalence	16
2.5.1 Conversion from NFA with ϵ to NFA without ϵ	16
2.6 Equivalence of NFAs and DFAs	17
2.6.1 Proof Strategy	17
2.6.2 Subset Construction Algorithm.....	17
2.7 Minimisation of FSM.....	19
2.8 Equivalence between Two FSM's.....	19
2.9 Finite Automata with Output	20
Summary	22
References	22
Recommended Reading.....	22
Self Assessment.....	23
 Chapter III.....	 25
Regular Languages	25
Aim	25
Objectives	25
Learning outcome	25
3.1 Introduction.....	26
3.2 Regular Sets	26
3.3 Regular Languages.....	26
3.4 Regular Expressions.....	26
3.5 Finite Automata and Regular Expressions.....	27
3.6 Conversion of RE to FA.....	27

3.7 Pumping Lemma for Regular Sets.....	29
3.8 Closure Properties of Regular Sets	30
Summary.....	32
References	32
Recommended Reading.....	32
Self Assessment	33
 Chapter IV.....	 35
Context Free Grammars	35
Aim	35
Objectives	35
Learning outcome	35
4.1 Introduction.....	36
4.2 Context-free Languages	36
4.3 Context-free Grammars	36
4.4 Ambiguity in CFG and its Removal.....	40
4.5 Chomsky Normal Form and Greiback Normal Form	44
4.6 Pumping Lemma for Context Free Languages	45
4.7 Enumeration of Properties of CFL.....	46
Summary.....	49
References	49
Recommended Reading.....	49
Self Assessment	50
 Chapter V.....	 52
Push Down Automata	52
Aim	52
Objectives	52
Learning outcome	52
5.1 Introduction.....	53
5.2 Pushdown Automata	53
5.2.1 Formal Definition of a PDA.....	53
5.2.2 PDA Transition Function	54
5.2.3 Graphical Format of PDA.....	55
5.2.4 Moves of a PDA.....	56
5.3 Language of PDA.....	56
5.3.1 Acceptance by Final State.....	56
5.3.2 Acceptance by Empty Stack	56
5.4 Equivalence of CGF and PDA.....	57
5.5 Equivalence of CFL and PDA	58
5.6 Introduction to DCFL and DPDA.....	60
Summary.....	61
References	61
Recommended Reading.....	61
Self Assessment.....	62
 Chapter VI.....	 64
Turing Machines	64
Aim	64
Objectives	64
Learning outcome	64
6.1 Introduction.....	65
6.2 Formal Definition of Turing Machine.....	66
6.2.1 Deterministic Turing Machines Definition	66
6.2.2 Non- Deterministic Turing Machines Definition.....	66

6.3 Some Configuration on Turing Machine (TM).....	74
6.4 Variants of Turing Machines.....	74
6.5 Equivalence with Other Models.....	75
6.6 The Church-Turing Thesis.....	75
6.7 Universal Turing Machine.....	75
Summary.....	77
References.....	77
Recommended Reading.....	77
Self Assessment.....	78
 Chapter VII.....	 80
Finite State Machines.....	80
Aim.....	80
Objectives.....	80
Learning outcome.....	80
7.1 Introduction.....	81
7.2 Informal Description.....	81
7.3 Turing Machines.....	82
7.4 Communicating Finite State Machines.....	83
7.5 Asynchronous Coupling.....	84
7.6 Synchronous Coupling.....	86
7.7 Formal Description.....	86
7.8 Execution of Machines.....	87
7.9 Minimisation of Machines.....	88
7.10 The Conformance Testing Problem.....	91
7.11 Combining Machines.....	91
7.12 Extended Finite State Machines.....	92
7.13 Generalisation of Machines.....	94
7.14 Restricted Models.....	96
7.14.1 Petri Net.....	96
7.14.2 FIFO Nets.....	98
Summary.....	99
References.....	99
Recommended Reading.....	99
Self Assessment.....	100
 Chapter VIII.....	 102
Overview of Proposition and Predicate Logic.....	102
Aim.....	102
Objectives.....	102
Learning outcome.....	102
8.1 Introduction.....	103
8.2 Proposition Logic.....	103
8.2.1 Formal Language Definition.....	103
8.2.2 Truth Tables.....	104
8.2.3 Natural deduction for proposition logic.....	104
8.2.4 Semantic tableaux.....	106
8.3 Predicate Logic.....	108
8.3.1 Formal Language Definition.....	108
8.4 Substitution.....	109
8.5 Semantics.....	110
8.6 Semantic tableaux.....	112
Summary.....	114
References.....	114
Recommended Reading.....	114
Self Assessment.....	115

List of Figures

Fig. 1.1 Model for finite automata	3
Fig. 1.2 Notations used in transition diagram	4
Fig. 1.3 Transition diagram	5
Fig. 1.4 Deterministic finite automata	6
Fig. 1.5 Non deterministic finite automata	7
Fig. 2.1 NFA with ϵ -transitions	15
Fig. 2.2 Moves of DFA	16
Fig. 3.1 Regular set	26
Fig. 3.2 Relationship between FA and regular expression	27
Fig. 3.3 The string w accepted by M	29
Fig. 6.1 Turing machine	65
Fig. 7.1 State transition diagram	82
Fig. 7.2 State transition diagrams, Tables 7.4 and 7.5	85
Fig. 7.3 State transition diagram for Table 7.8	88
Fig. 7.4 Petri Net with hang state	97

List of Tables

Table 1.1 Difference between NFA and DFA.....	8
Table 7.1 Mealy	81
Table 7.2 Non-Determinism.....	82
Table 7.3 Busy Beaver	83
Table 7.4 Sender.....	84
Table 7.5 Receiver	85
Table 7.6 User	86
Table 7.7 Server	86
Table 7.8 Receiver-II.....	88
Table 7.9 Equivalence	90
Table 7.10 Finite state variable	92
Table 7.11 Extended finite state machine	94

Abbreviations

BN	-	Backus-Naur
CFG	-	Context-free Grammars
CFL	-	Context-free Languages
CNF	-	Chomsky Normal Form
DCFL	-	Deterministic Context-free Languages
DFA	-	Deterministic Finite Automaton
DPDA	-	Deterministic Pushdown Automata
FA	-	Finite Automata
FAS	-	Finite Automated System
FSM	-	Finite State Machine
GNF	-	Greibach Normal Form
LIFO	-	Last-in-First-Out
NFA	-	Non-Deterministic Finite Automata
PDA	-	Pushdown Automaton
RE	-	Regular Expression
TM	-	Turing Machine
VLSI	-	Very Large Scale Integration

Chapter I

Fundamentals

Aim

The aim of this chapter is to:

- explore the theory of automata
- explicate finite state machine
- enlist the types of automata

Objectives

The objectives of this chapter are to:

- examine defining languages
- introduce deterministic finite automata (DFA)
- elucidate non-deterministic finite automata (NFA)

Learning outcome

At the end of this chapter, you will be able to:

- recognise the finite automata model
- identify the types of automata
- compare the differences between DFA and NFA

1.1 Introduction

Formal languages and automata theory is based on mathematical computations. These computations are used to represent various mathematical models. Automata theory is a theory of models. Working of every process can be represented by means of models. The model can be theoretical or mathematical model. The model helps in representing the concept of every activity. In this chapter, we will discuss all the fundamentals of automata theory and those are strings, languages, operations on the languages.

1.2 Introduction to Defining Languages

- Alphabet: An alphabet is a finite, non- empty set of symbols. Σ (sigma) is used to denote alphabet. The example of alphabet is,

$$\Sigma = \{a, b, c, \dots, z\}$$

The elements a, b, ..., z are alphabets

$$\Sigma = \{0, 1\}$$

Here, 0 and 1 are alphabets.

- String: String is a finite collection of symbols from alphabet Σ
For example, if $\Sigma = \{a, b\}$ then various strings that can be formed from Σ are $\{ab, aa, aaa, bb, bbb, ba, aba, \dots\}$. An infinite number of strings can be generated from this set.
 - The empty string can be denoted by ϵ (epsilon)
 - Length of a string w is $|w|$
 - The prefix of any string is any number of leading symbols of that string and suffix is any number of trailing.

Operations on string

Various operations that can be carried out on strings are:

- Concatenation: In this operation, two strings are combined together to form a single string.
For example, $x = \{\text{white}\}$ $y = \{\text{house}\}$
Then, the concatenated string will be $xy = \{\text{white house}\}$
- Transpose: The transpose of operation is also called reverse operation.
For example, $(xa)^T = a(x)^T$.
If $(ababbb)^T = (bbbaba)$.
- Palindrome: Palindrome of string is a property of a string in which string can be read same from left to right as well as from right to left. For example, the string 'Madam' is a palindrome because it is same, if we read it from left to right or from right to left.
- Language: The language is a collection of appropriate strings. The language is defined using an input set. L is said to be language over alphabet Σ , if L is a subset of strings in Σ^*

For example

- The language of all strings consisting of n 0's followed by n 1's: $\{\epsilon, 01, 0011, 000111, \dots\}$
- The set of strings of with equal number of 0's and 1's: $\{\epsilon, 01, 10, 0011, 1100, 0101, 1001, \dots\}$.
- $\emptyset$ = Empty language
- Let $L = \{\epsilon\}$

Operations on language

Language is a collection of strings. Hence, operations that can be carried out on strings are the operations that can be carried out on language.

If L_1 and L_2 are two languages then,

- Union of two languages is denoted as $L_1 \cup L_2$
- Concatenation of two languages is denoted by $L_1 L_2$
- Intersection of two languages is denoted by $L_1 \cap L_2$
- Difference of two languages is denoted by $L_1 - L_2$

1.3 Finite State Machine

The finite state system represents a mathematical model of a system with certain input. The model finally gives a certain output. The input, when is given to the machine, is processed by various states, these states are called as intermediate states.

The very good example of finite state system is control mechanism of elevator. The mechanism only remembers the current floor number pressed. It does not remember all the previously pressed numbers.

The finite state system is a very good design tool for the programs such as text editors and lexical analysers. The lexical analyser is a program which scans your program character by character and recognises those words as tokens.

Definition

A finite automata is a collection of 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where,

- Q is a finite set of states, which is non empty
- Σ is input alphabet, indicates input set
- q_0 is a initial state and q_0 is in Q i.e. $q_0 \in Q$
- F is a set of final states
- δ is a transition function or a mapping function. Using this function, the next state can be determined.

1.4 Finite Automata Model

The finite automata can be represented as:

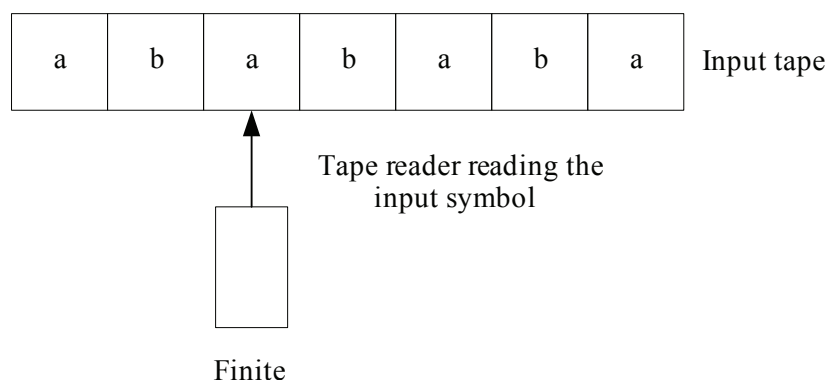


Fig. 1.1 Model for finite automata

The finite automata can be represented using:

- Input tape: It is a linear tape having some number of cells. Each input symbol is placed in each cell.
- Finite control: The finite control decides the next state on receiving particular input from input tape. The tape reader reads the cells one by one from left to right and at a time, only one input symbol is read.

For example, suppose current state is q , and reader is reading the symbol 'a', then it is finite control which decides what will be the next state at input 'a'. The transition from current state ' q ' with input 'w' to next state ' q ' 'producing w' will be represented as,

$$(q, w) \vdash (q', w')$$

If 'w' is a string and 'M' is a finite automata, then 'w' is accepted by the FA if;

$$(w, s) \vdash^* (q, \epsilon)$$

with 'q' as final state.

The set of strings accepted by a FA given by M then M is accepted by language L. The acceptance of M by some language L is denoted by $L(M)$.

A machine M accepts a language L if,

$$L = L(M)$$

1.5 Acceptance of Strings and Languages

The strings and languages can be accepted by finite automata, when it reaches to the final state. There are two preferred notations for describing automata:

Transition diagram

A transition diagram or transition graph can be defined as a collection of:

- Finite set of states K
- Finite set of symbols Σ
- A non-empty set S of K . It is called start state
- A set of $F \subseteq K$ of final state
- A transition function $K \times A \rightarrow K$ as state and A as input from Σ^*

The notations used in transition diagram are:

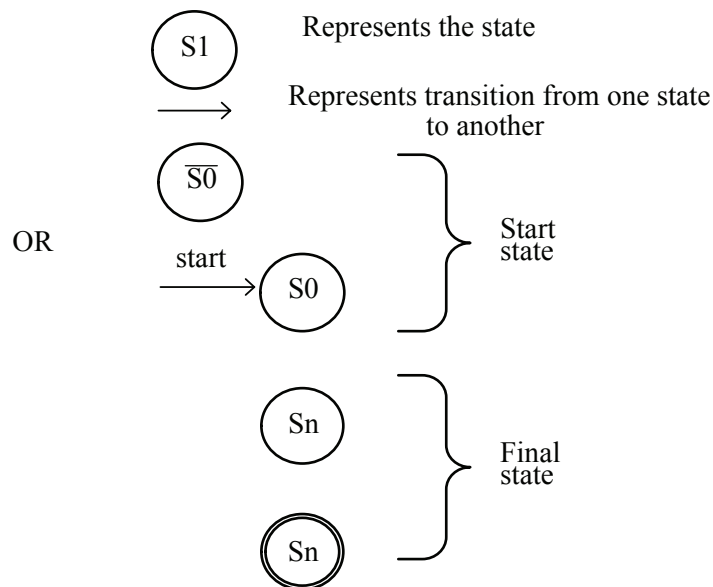


Fig. 1.2 Notations used in transition diagram

Example

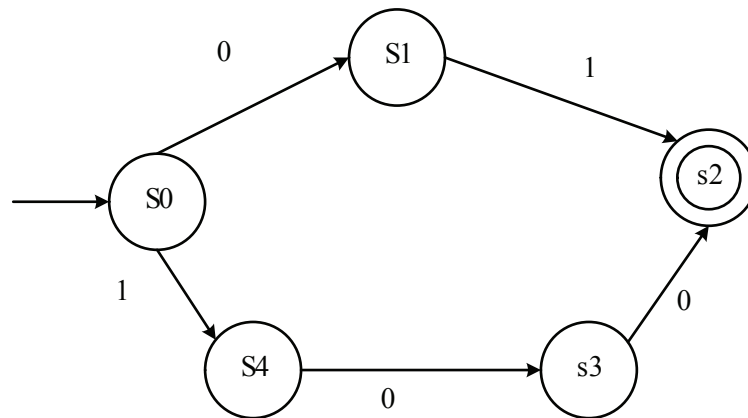
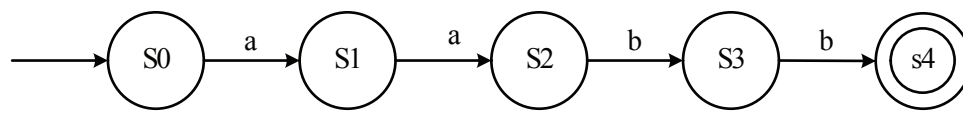


Fig. 1.3 Transition diagram

The FA can be represented using transition graph. The machine initially is in the start state ' S_0 '. Then on receiving input '0', it changes to the state ' S_1 ' from ' S_0 '. On receiving input '1', the machine changes its state to ' S_4 '. The state ' S_2 ' is a final state or an accept state. When we trace the input for transition diagram and reach to the final state at the end of input string, then it is said that the given input is accepted by transition diagram.

Another example



We have drawn a transition diagram for the input aabb.

Note, that the start state is S_0 and final state is S_4 . The input set is $\Sigma = \{a, b\}$. The states S_1, S_2, S_3 are all intermediate states.

Transition table

This is a tabular representation of finite automata. For transition table, the transition function is used.

For example

Input States	a	b
q_0	q_1	-
q_1	-	q_2
q_2	q_2	-

The rows of the table correspond to states and columns of the table correspond to inputs.

1.6 Deterministic Finite Automata (DFA)

The finite automata are called deterministic finite automata (DFA), if there is only one path for a specific input from current state to next state. For example, the DFA can be shown as:

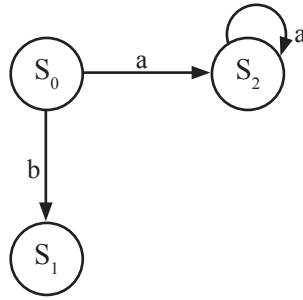


Fig. 1.4 Deterministic finite automata

From state S_0 for input 'a', there is only one path, going to S_2 . Similarly, from S_0 there is only one path for input b going to S_1 .

The DFA can be represented by the same 5-tuple described in the definition of FSM.

Definition of DFA

A deterministic finite automaton A is a five tuple notation.

$$A = (Q, \Sigma, \delta, q_0, F)$$

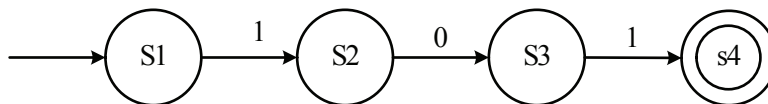
Where,

- Q = finite set of states,
- Σ = finite set of input symbols
- q_0 is the start state such that $q_0 \in Q$.
- F is a set of final states such that $F \in Q$ and
- The transition function denoted by δ . Two parameters are passed to this transition function: one is current state and other is input symbol. The transition function returns a state which can be called as next state.

For example, $q_1 = \delta(q_0, a)$ means from current state q_0 , with input a the next state transition is q_1 .

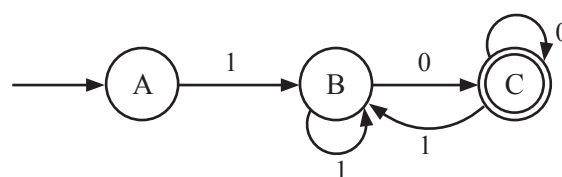
Example 1.1 Design a FA which accepts the only input 101 over the input set $Z = \{0, 1\}$

Solution:



Example 1.2: Design FA which accepts only those strings which starts with 1 and ends with 0.

Solution: The FA will have a start state A from which only the edge with input 1 will go to next state.



In state B, if we read 1, we will be in B state, but if we read 0 at state B, we will reach to state C which is a final state. In state C, if we read either 0 or 1, we will go to state C or B respectively. Note, that the special care is taken for 0, if the input ends with 0 it will be in final state.

1.7 Non-Deterministic Finite Automata (NFA)

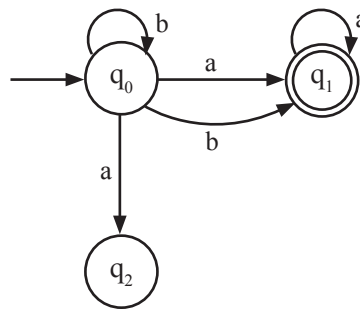


Fig. 1.5 Non deterministic finite automata

The concept of non deterministic finite automata (NDFA) is exactly reverse of deterministic finite automata (DFA). The Finite Automata is called NFA, when there exists many paths for a specific input from current state to next state. The NFA shows from 'q₀' for input 'a', there are two next states 'q₁' and 'q₂'. Similarly, from 'q₀' for input 'b', the next states are 'q₀' and 'q₁'. Thus, it is not fixed or determined with a particular input where to go next. Hence, this FA is called non deterministic finite automata.

Consider the input string bba. This string can be derived as:

Input	b	a	a
Path	q ₀	q ₀	q ₁

Or

Input	b	a	a
Path	q ₀	q ₀	q ₂

Or

Input	b	a	a
Path	q ₀	q ₁	q ₁

Thus, you cannot take decision of which path has to be followed for deriving the given string.

Definition of NFA

The NFA can be formally defined as a collection of 5-tuple:

Q is a finite set of states

Σ is a finite set of inputs

δ is called next state or transition function

q₀ is initial state.

F is final state where $F \subseteq Q$

There can be multiple final states. Thus, the next question might be what is the use of NFA's. The NFA are basically used in the theory of computation because they are more flexible and easier to use than the DFA's.

1.8 Difference between NFA and DFA

DFA	NFA
All transitions are deterministic. Each transition leads to exactly one state.	All transitions are non-deterministic. Each transition lead to a subset of states.
For each state, transition on all possible symbols (alphabet) should be defined.	For each state, not all symbols necessarily have to be defined in the transition function.
Accepts input if the last state is in.	Accepts input if one of the last states is in F.
Sometimes harder to construct because of the number of states.	Generally easier than a DFA to construct.
Practical implementation is feasible.	Practical implementation has to be deterministic (so needs conversion to DFA).

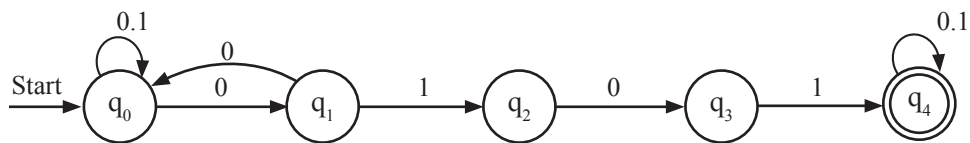
Table 1.1 Difference between NFA and DFA

Example 1.3: Construct a NFA for the language

$L_1 = \{\text{consisting a substring } 0101\}$

$L_2 = \{a^n \cup b^n\}$

Solution: We will consider ' L_1 ' first to design the NFA. There can be any combination of 0 and 1 in the language but the substring 0101 must be present. We will get such a substring then, it leads to a final state or accept state.

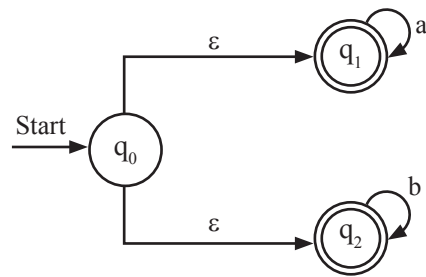


This is a NFA as for '0' input, we have two different paths, one going to ' q_0 ' and the other is going to ' q_1 '. The string 00010101 is acceptable by above given NFA and it is as shown below.

00010101 $0q_0 \vdash 010101$
 $00q_0 \vdash 010101$
 $000q_1 \vdash 10101$
 $0001q_2 \vdash 0101$
 $0001q_3 \vdash 101$
 $000101q_4 \vdash 01$
 $0001010q_4 \vdash 1$
 $00010101q_4 \vdash$
 00010101 is accepted as q_4 is final state.

Now we will build an NFA for ' L_2 '. The language ' L_2 ' is a language in which there be any number of a's or any number of b's. It accepts $\{a, b, aa, bb, aaa, bbb, \dots\}$.

Hence, the NFA will be:



The NFA shows two different states ' q_1 ' and ' q_2 ' for the input ϵ from q_0 state.

Here, ϵ is basically a null move, i.e., a move carrying no symbol from input set Σ . But a state change occurs from one state to other.

Summary

- Formal languages and automata theory is based on mathematical computations.
- An alphabet is a finite, non-empty set of symbols.
- String is a finite collection of symbols from alphabet Σ .
- The language is a collection of appropriate strings.
- The finite state system represents a mathematical model of a system with certain input.
- A very good example of finite state system is control mechanism of elevator.
- The finite state system is a very good design tool for the programs such as text editors and lexical analysers.
- The FA can be represented using transition graph.
- The finite automata are called deterministic finite automata (DFA), if there is only one path for a specific input from current state to next state.
- The finite automata is called NFA, when there exists many paths for a specific input from the current state to the next state.

References

- Puntambekar, A.A., 2009. *Formal Languages and Automata Theory*. Technical publications.
- Sharma, A., 2006. *Theory of Automata and Formal Languages*. Firewall Media.
- *Finite Automata* [Pdf] Available at: <<http://www.eecs.wsu.edu/~ananth/CptS317/Lectures/FiniteAutomata.pdf>> [Accessed 7 June 2013].
- Lawson, M. V., *Finite automata* [Pdf] Available at: <<http://www.ma.hw.ac.uk/~markl/preprints/Lawson.pdf>> [Accessed 7 June 2013].
- *Lecture 38-Finite State Automaton* [Video online] Available at: <<https://www.youtube.com/watch?v=-K0cRsZSnLc>> [Accessed 7 June 2013].
- *Finite Automata* [Video online] Available at: <<https://www.youtube.com/watch?v=-xkcs-t99Do>> [Accessed 7 June 2013].

Recommended Reading

- Linz, P., 2006. *An Introduction to Formal Language and Automata*, 4th ed., Jones & Bartlett Pub
- Hopcroft, J. E., Motwani, R. and Jeffrey D. U., 2006. *Introduction to Automata Theory, Languages and Computation*. 3rd ed.
- Addison Wesley, Simon, M., 1999. *Automata Theory*, 1st ed., World Scientific Publishing Company.

Self Assessment

1. Formal languages and automata theory is based on _____ computations.
 - a. mathematical
 - b. logical
 - c. physical
 - d. theoretical
2. The transpose of operation is also called _____ operation.
 - a. transverse
 - b. complementary
 - c. reverse
 - d. paired
3. The language is a collection of appropriate _____.
 - a. strings
 - b. sets
 - c. indices
 - d. matrix
4. The finite automata is called _____ when there exists many paths for a specific input from current state to next state.
 - a. DFA
 - b. PCP
 - c. TM
 - d. NFA
5. The _____ state system represents a mathematical model of a system with certain input.
 - a. infinite
 - b. finite
 - c. logical
 - d. deterministic
6. Which of the following statement is true?
 - a. An alphabet is a finite, empty set of symbols.
 - b. An alphabet is a finite, non- empty set of symbols.
 - c. An alphabet is a infinite, non- empty set of symbols.
 - d. An alphabet is a infinite, non- empty set of strings.

7. Which of the following statement is true?
 - a. The infinite control decides the next state on receiving particular input from input tape.
 - b. The finite control decides the previous state on receiving particular input from input tape.
 - c. The finite control decides the next state on receiving particular input from input tape.
 - d. The finite control decides the next state on receiving particular input from output tape.
8. Which of the following statement is false?
 - a. String is a finite collection of symbols from alphabet Σ .
 - b. A deterministic finite automaton A is a five tuple notation.
 - c. A deterministic finite automaton A is a six tuple notation.
 - d. The FA can be represented using transition graph.
9. Which of the following is a tabular representation of finite automata?
 - a. Transaction table
 - b. Transition table
 - c. Post correspondence table
 - d. Turing table
10. Two strings are combined together to form a single string in _____ operation.
 - a. Concatenation
 - b. Palindrome
 - c. Transpose
 - d. Combination.

Chapter II

Finite Automata

Aim

The aim of this chapter is to:

- introduce the concept of finite automata
- explain non finite automata (NFA) with ϵ -transitions
- elucidate the conversion of a NFA with ' ϵ ' to a NFA without ' ϵ '

Objectives

The objectives of this chapter are to:

- explicate the significance of non deterministic finite automata
- explain the significance of a NFA with ' ϵ '
- elucidate the equivalence between two finite state machines (FSM's)

Learning outcome

At the end of this chapter, you will be able to:

- determine finite automata
- understand what is finite automata with output
- recognise the equivalence of DFA's and NFA's

2.1 Introduction

Finite automata are important in science, mathematics, and engineering. Engineers like them because they are superb models for circuits (and, since the advent of VLSI systems sometimes finite automata represent circuits). Computer scientists adore them because they adapt very likely to algorithm design. For example, the lexical analysis portion of compiling and translation. They are useful for mathematicians too, due to the fact that there are several nifty mathematical characterisations of the sets they accept.

Can a machine recognise a language? The answer is 'yes' for some machines and an elementary class of machines is called finite automata. Regular languages can be represented by certain kinds of algebraic expressions by Finite automaton and by certain grammars. For example, suppose we need to compute with numbers that are represented in scientific notation.

Can we write an algorithm to recognise strings of symbols represented in this way? To do this, we need to discuss some basic computing machines called finite automata.

Automata will be finite automata if it accepts all the words of any regular language where language means a set of strings. In other words, the class of regular language is exactly the same as the class of languages accepted by FA's., a deterministic finite automata.

2.2 Finite Automata

A system where energy and information are transformed and used for performing some functions without direct involvement of man is called automaton. Examples are= automatic machine tools, automatic photo printing tools, etc.

A finite automata is similar to a finite state machine. A finite automata consists of five parts:

- a finite set of states
- a finite set of alphabets
- an initial state
- a subset of set of states (whose elements are called 'yes' state or; accepting state) and
- a next-state function or a transition state function

Finite automata over a finite alphabet 'A' can be thought of as a finite directed graph with the property that each node omits one labeled edge for each distinct element of A. The nodes are called states. There is one special state called the start (or initial) state, and there is a possible empty set of states called final states.

2.3 Significance of Non-Deterministic Finite Automata

Computers are basically deterministic machines. That means on giving certain input, we get certain output either desirable or undesirable. In similar manner, deterministic finite automata are the machines, in which, we get some predictable state on certain input. But constructing such a deterministic machine is very difficult. Hence, there should be some way by which some easy to construct machines can be buildt. In such a case, we construct a nondeterministic machine which is called NFA or NDFA. This NFA then can be easily converted to DFA. This efficient mechanism is majorly used in compilers. Thus, NFA serves as a bridge between input given and DFA constructed.

2.4 NFA with ϵ -Transitions

The ϵ -transitions in NFA are given in order to move from one state to another without having any symbol from input set Σ .

Consider the NFA with ϵ as:

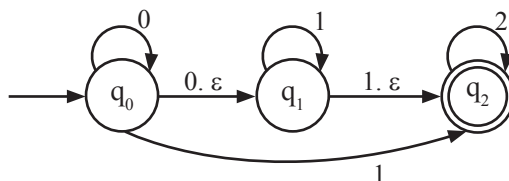


Fig. 2.1 NFA with ϵ -transitions

In this NFA with ϵ , q_0 is a start state. With input 0, one can either be in state q_0 or in state q_1 . If we get at the start a symbol 1 with ϵ -move, we can change state from q_0 to q_1 and then with input we can be in state q_1 . On the other hand, from start state q_0 , with input 1, we can reach to state q_2 . Thus, it is not definite that on input 1 whether we will be in state q_1 or q_2 . Hence, it is called nondeterministic finite automata (NFA) and since there are some ϵ moves by which we can simply change the states from one state to other. Hence, it is called NFA with ϵ .

2.4.1 Significance of NFA with ϵ

As construction of DFA is very difficult for certain input set, we construct NFA. This NFA then can be constructed to DFA. ϵ is an empty string. The ϵ -transitions are used simply to change one state to other. Sometimes, to reach to final state, we do not get proper state from start state. In such cases, we simply want to reach to certain state which leads to final state. Such a transition to that specific state should be without any input symbol. Hence, we require some ϵ -moves by which a proper state can be obtained for reaching to final state. Thus ϵ -moves play an important role in NFA.

2.4.2 Acceptance of Language

The language L accepted by NFA with ϵ , denoted by $M = (Q, \Sigma, \delta, q_0, F)$ can be defined as:

Let, $M = (Q, \Sigma, \delta, q_0, F)$ be a NFA with ϵ

Where,

Q is a finite set of states.

Σ is input set

δ is a transition or mapping function for transitions from $Q \times \{\Sigma \cup \epsilon\}$ to 2^Q

q_0 is a start state

F is a set of final states such that $F \subseteq Q$

The string w in L accepted by NFA can be represented as:

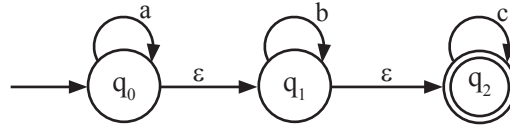
$L(M) = \{w \mid w \in \Sigma^* \text{ and } \delta \text{ transition for } w \text{ from } q_0 \text{ reaches to } F\}.$

Definition of ϵ -closure

The ϵ -closure (p) is a set of all states which are reachable from state p on ϵ -transitions such that:

1. ϵ -closure (p) = p where $p \in Q$
2. If there exists ϵ -closure (p) = $\{q\}$ and $\delta(q, \epsilon) = r$ then
 ϵ -closure (p) = $\{q, r\}.$

Example 2.1: Find ϵ -closure for the following NFA with ϵ



Solution:

ϵ -closure (q_0) = $\{q_0, q_1, q_2\}$ means self state + ϵ -reachable states

ϵ -closure (q_1) = $\{q_1, q_2\}$ means q_1 is a self state and q_2 is a state obtained from q_1 with ϵ input

ϵ -closure (q_2) = $\{q_2\}$

2.5 Conversions and Equivalence

The NFA with ϵ can be converted to NFA without ϵ . And this NFA without ϵ can be converted to DFA. In this section, we will discuss these conversions and will find them equivalent to each other.

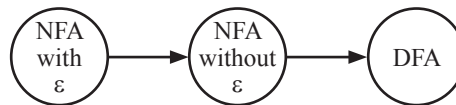


Fig. 2.2 Moves of DFA

2.5.1 Conversion from NFA with ϵ to NFA without ϵ

In this method, we try to remove all the ϵ -transitions from given NFA. The method will be:

1. Find out all the ϵ -transitions from each state from Q . That will be called as ϵ -closure $\{q_i\}$ where $q_i \in Q$
2. Then, δ' transitions can be obtained. The δ' transitions means an ϵ -closure on δ moves.
3. Step-2 is repeated for each input symbol and for each state of given NFA.
4. Using the resultant states, the transition table for equivalent NFA without ϵ can be built.

Theorem: If L is accepted by NFA with ϵ -transitions, then there exists L which is accepted by NFA with ϵ -transitions.

Proof:

Let $M(Q, \Sigma, \delta, q_0, F)$ be an NFA with ϵ -transitions.

Construct $M' = (Q, \Sigma, \delta', q_0, F)$ where

$F' = F \cup \{q_0\}$ if ϵ -closure contains state off

F otherwise

M' is a NFA without ϵ moves. The δ' function can be denoted by δ'' with some input. For example, $\delta'(q, a) = \delta''(q, a)$ for some q in Q and a from Σ . We will apply the method of induction with input x . The x will not be ϵ because

$$\delta'(q_0, \epsilon) = \{q_0\}$$

$\delta''(q_0, \epsilon) = \epsilon$ -closure (q_0). Therefore, we will assume length of string to be 1.

Basis: $|x| = 1$. Then x is a symbol a

$$\delta'(q_0, \epsilon) = \delta''(q, a)$$

Induction: $|x| > 1$ Let $x = wa$

$$\delta'(q_0, w, a) = \delta'(\delta'(q_0, w), a)$$

By inductive hypothesis,

$$\delta'(q_0, w) = \delta''(q_0, w) = p$$

Now we will show that $\delta'(p, a) = \delta(q_0, w, a)$

$$\text{But } \delta'(p, a) = \cup_{q \text{ in } p} \delta'(q, a) = \cup_{q \text{ in } p} \delta''(q, a)$$

as $p = \delta''(q_0, w)$ we have,

$$\cup \delta''(q, a) = \delta''(q_0, wa) \\ q \text{ in } p$$

Thus, by definition δ''
 $\delta'(q_0, wa) = \delta''(q_0, wa)$

Rule for conversion

$\delta'(q, a) = \varepsilon\text{-closure}(\delta(\hat{\delta}(q, \varepsilon), a))$
 Where $\hat{\delta}(q, \varepsilon) = \varepsilon\text{-closure}(q)$

2.6 Equivalence of NFAs and DFAs

We will now show that DFAs and NFAs accept exactly the same set of languages. That is, non-determinism does not make a finite automaton any more powerful. To show that NFAs and DFAs accept the same class of languages, we show two things:

- Any language accepted by a DFA can also be accepted by some NFA.
- Any language accepted by a NFA can also be accepted by some DFA.

2.6.1 Proof Strategy

- To show that any language accepted by NFA, is also accepted by some DFA.
- We describe an algorithm that takes any NFA and converts it into a DFA that accepts the same language.
- The algorithm is called the 'Subset Construction Algorithm'.
- We can use mathematical induction (on the length of a string accepted by the automaton) to prove that the DFA that is constructed accepts the same language as the NFA.

2.6.2 Subset Construction Algorithm

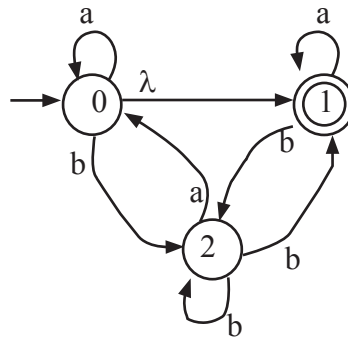
- Given an NFA, it constructs a DFA that accepts the same language.
- The equivalent DFA simulates the NFA by keeping track of the possible states it could be in. Each state of the DFA corresponds to a subset of the set of states of the NFA - hence, the name of the algorithm.
- If the NFA has 'n' states, the DFA can have as many as '2n' states (why?), although it usually has many less.

Steps of Subset Construction Algorithm

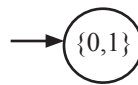
- The initial state of the DFA is the set of all states of NFA that can be taken without reading any input.
- For any state $\{q_i, q_j, \dots, q_k\}$ of the DFA and any input a , the next state of the DFA is the set of all states of the NFA that can result as next states if the NFA is in any of the states $q_i, q_j, \dots, q_k$ when it reads a . This includes states that can be reached by reading a followed by any number of λ -transitions. Use this rule to keep adding new states and transitions until it is no longer possible to do so.
- The accepting states of the DFA are those states that contain an accepting state of the NFA.

Example:

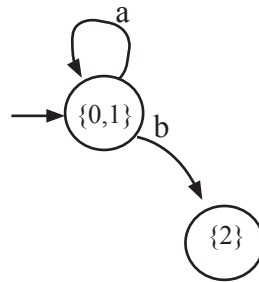
Here is an NFA that we want to convert to an equivalent DFA.

**Step 1**

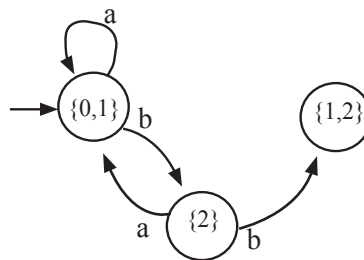
The start state of the DFA is the set of states. The NFA can be in, before reading any input. This includes the start state of the NFA and any states that can be reached by an ϵ -transition.

**Step 2**

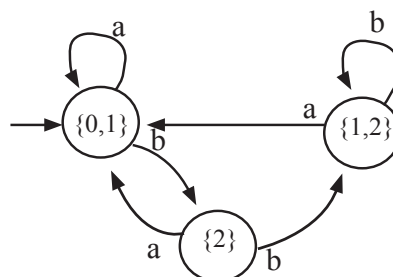
For state $\{0, 1\}$, we create a transition for each possible input, 'a' and 'b'. In the process, we create state $\{2\}$.

**Step 3**

For state $\{2\}$, we create a transition for each possible input, 'a' and 'b'. In the process, we create another state, $\{1, 2\}$.

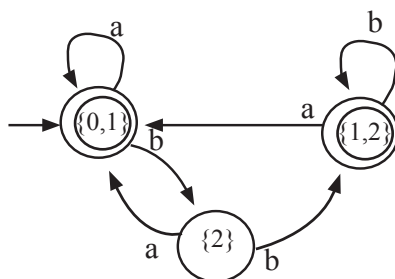
**Step 4**

For state $\{1, 2\}$, we create a transition for each possible input, 'a' and 'b'. At this point, a transition is defined for every state-input pair.



Step 5

The last step is to mark the final states of the DFA.



2.7 Minimisation of FSM

The minimisation of FSM means reducing the number of states from the given FA. Thus, we get the FSM with redundant states after minimising the FSM. While minimising FSM, first find out which two states are equivalent we can represent those two states by one representative state.

The two states q_1 and q_2 are equivalent if both, $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both of them are non final states, for all $x \in \Sigma^*$ (Σ^* indicate any string of any length). We can minimize the given FSM by finding equivalent states.

Method for the construction of minimum state automata:

Step 1: We will create a set Π_0 as $\Pi_0 = \{Q_1^0, Q_2^0\}$ where Q_1^0 is set of all final states and $Q_2^0 = Q - Q_1^0$, where Q is a set of all the states in DFA.

Step 2: Now we will construct Π_{k+1} from Π_k . Let Q_i^k be any subset in Π_k . if q_1 and q_2 are in Q_i^k they are $(K+1)$ equivalent provided $\delta(q_1, a)$ and $\delta(q_2, a)$ are K equivalent. Find out whether $\delta(q_1, a)$ and $\delta(q_2, a)$ are residing in the same equivalent class Π_k , then it is said that q_1 and q_2 are $(K+1)$ equivalent. Thus Q_i^k are further divided into $(K+1)$ equivalence classes. Repeat step 2 for every Q_i^k in Π_k and obtain all the elements of Π_{k+1} .

Step 3: Construct Π_n for $n=1, 2, \dots$ until $\Pi_n = \Pi_{n+1}$

Step 4: then replace all the equivalent states in one equivalent class by representative state. This helps in minimizing the given DFA.

2.8 Equivalence between Two FSM's

The two finite automata are said to be equivalent if both the automata accept the same set of strings, over an input set Σ . When two FAS are equivalent then there is some strings x over Σ , on acceptance of that string if one FA reaches to final state other FA also reaches to final state. We can compare whether two FAS are equivalent or not using following method.

Method for comparing two FAS

Let M and M' be two FAS and Σ is a set of input strings

- We will construct a transition table have pair wise entries (q, q') where $q \in M$ and $q' \in M'$ for each input symbol.
- If we get in a pair as one final state and other non final state then we terminate construction of transition table declaring that two FAs are not equivalent.
- The construction of transition table gets terminated when there is no new pair appearing in the transition table.

2.9 Finite Automata with Output

The finite automata is a collection of $(Q, \Sigma, \delta, q_0, F)$ where Q is a set of states including q_0 as a start state. In FA after reading the input string if we get final state then the string is said to be 'acceptable'. If we do not get final state then it is said that the string is 'rejected'. That means there is no need of output for the finite automata. The 'accept' or 'reject' acts like 'yes' or 'no' output for machine. But if there is a need for specifying the output other than yes or no, then in such a case we require finite automata along with output. There are two types of FA with output and those are:

- Moore machine
- Mealy machine

Moore machine

Moore machine is a finite state machine in which the next state is decided by current state and current input symbol. The output symbol at a given time depends only on the present state of machine. The formal definition of Moore machine is,

'Moore machine is a six tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$

Q is a finite set of states

Σ is input set of input symbols

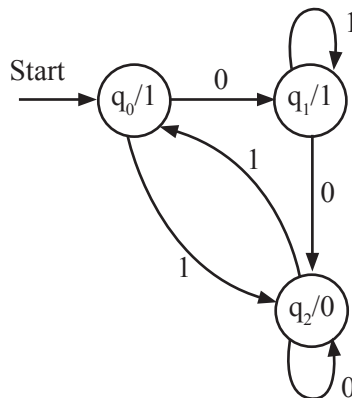
Δ is an output alphabet

δ is a transition function such that $Q \times \Sigma \rightarrow Q$. This is also known as state function

λ is an output function $Q \rightarrow \Delta$. This is also known as machine function

q_0 is the initial state of machine

For example



Consider the Moore machine given below:

The transition table will be

Current state	Next state (δ)		Output (λ)
	0	1	
q_0	q_1	q_2	1
q_1	q_2	q_1	1
q_2	q_2	q_0	0

In Moore machine output is associated with every state. In the above given Moore machines when machine is in q_0 state the output will be 1. For the Moore machine if the length of input string is n then output string has length $n+1$.

For the string 0110 then the output will be 11110.

Mealy machine

Mealy machine is a machine in which output symbol depends upon the present input symbol and present state of the machine. The Mealy machine can be defined as:

Mealy machine is a six tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$ where

Q is a finite set of states

Σ is input set of input symbols

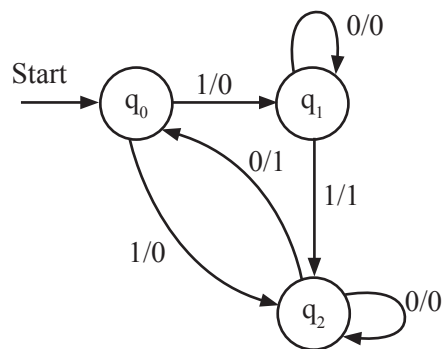
Δ is an output alphabet

δ is state transition function such that $Q \times \Sigma \rightarrow Q$.

λ is machine function such that $Q \times \Sigma \rightarrow \Delta$.

q_0 is the initial state of machine

For example



For the input string 1001 the output will be 0001. In mealy machine the length of input string is equal to length of output string.

Summary

- Finite automata are important in science, mathematics, and engineering.
- Regular languages can be represented by certain kinds of algebraic expressions by Finite automaton and by certain grammars.
- Automata will be finite automata if it accepts all the words of any regular language where language means a set of strings.
- A finite automata is similar to a finite state machine.
- The minimization of FSM means reducing the number of states from the given FA.
- The two finite automata are said to be equivalent if both the automata accept the same set of strings, overran input set Σ .
- The finite automata is a collection of $(Q, \Sigma, \delta, q_0, F)$ where Q is a set of states including q_0 as a start state.
- Moore machine is a finite state machine in which the next state is decided by current state and current input symbol.
- Mealy machine is a machine in which output symbol depends upon the present input symbol and present state of the machine.

References

- Puntambekar, A. A., 2007. *Theory of Automata and Formal Languages*, Technical Publications.
- Krithivasan, K., 2009. *Introduction to Formal Languages, Automata Theory and Computation*, Dorling Kindersley(India) Pvt. Ltd.
- *Finite-State Machines* [Pdf] Available at: <<http://www.cse.chalmers.se/~coquand/AUTOMATA/book.pdf>> [Accessed 7 June 2013].
- *Introduction-Finite Automata* [Pdf] Available at: <<http://users.ece.utexas.edu/~adnan/verif-04/fsa.pdf>> [Accessed 7 June 2013].
- *Deterministic Finite Automata* [Video online] Available at: <https://www.youtube.com/watch?v=3GxTDsQFY_w> [Accessed 7 June 2013].
- *DFA Deterministic finite state automaton* [Video online] Available at: <<https://www.youtube.com/watch?v=FNyRkNzOFXs>> [Accessed 7 June 2013].

Recommended Reading

- Lawson. M. V., 2003. *Finite Automata*. 1st ed., Chapman and Hall/CRC.
- Kohavi, Z. and Jha, N., 2009. *Switching and Finite Automata Theory*, 3rd ed., Cambridge University Press; Dexter C. Kozen. D. C., 1997 *Automata and Computability*. Springer.

Self Assessment

1. _____ languages can be represented by certain kinds of algebraic expressions by finite automaton and by certain grammars
 - a. Regular
 - b. Irregular
 - c. Logical
 - d. Data
2. Automata will be _____ automata if it accepts all the words of any regular language where language means a set of strings.
 - a. infinite
 - b. finite
 - c. regular
 - d. irregular
3. A system where energy and information are transformed and used for performing some functions without direct involvement of man is called _____.
 - a. mechanization
 - b. computation
 - c. automation
 - d. estimation
4. A finite automata is similar to a _____ state machine.
 - a. finite
 - b. infinite
 - c. logical
 - d. physical
5. The _____ are used simply to change one state to other.
 - a. p-transitions
 - b. ϵ -transitions
 - c. q-transitions
 - d. x-transitions.
6. Which of the following statement is true?
 - a. The minimization of FSM means reducing the number of states from the given NFA.
 - b. The minimization of FSM means reducing the number of states from the given FA.
 - c. The minimization of FSM means reducing the number of states from the given DFA.
 - d. The maximization of FSM means reducing the number of states from the given FA.

7. Which of the following statement is true?
 - a. Moore machine is a machine in which output symbol depends upon the present input symbol and present state of the machine.
 - b. Mealy machine is a machine in which input symbol depends upon the present input symbol and present state of the machine.
 - c. Mealy machine is a machine in which output symbol depends upon the present input symbol and present state of the machine.
 - d. Mealy machine is a machine in which output symbol depends upon the previous input symbol and next state of the machine.
8. Which of the following statement is false?
 - a. In Mealy machine output is associated with every state.
 - b. In Moore machine input is associated with every state.
 - c. In Moore machine output is associated with only one input state.
 - d. In Moore machine output is associated with every state.
9. Which of the following machine is a machine in which output symbol depends upon the present input symbol and present state of the machine?
 - a. Moore
 - b. Turing
 - c. Finite
 - d. Mealy
10. Which of the following automata is a collection of $(Q, \Sigma, \delta, q_0, F)$ where Q is a set of states including q_0 as a start state?
 - a. Finite
 - b. Infinite
 - c. Deterministic
 - d. Non deterministic

Chapter III

Regular Languages

Aim

The aim of the chapter is to:

- explore the concept of regular languages
- elucidate regular expressions
- explain properties of regular sets

Objectives

The objectives of this chapter are to:

- explicate finite automata and regular expressions
- explain pumping lemma of regular sets
- enlist closure properties of regular sets

Learning outcome

At the end of this chapter, you will be able to:

- understand regular sets and regular expressions
- determine conversion of regular expressions to finite automata
- identify various closure properties of regular sets

3.1 Introduction

The regular languages are the languages which are accepted by finite automata. These languages are typically represented by regular expressions. The most important theorem called pumping lemma which is used in proving whether the language is regular or not, will be discussed in the chapter. Various closure properties of are also discussed in the chapter regular languages.

3.2 Regular Sets

Regular sets are the sets which are accepted by finite automata.

For example: $L = (\epsilon, 00, 0000, 000000, \dots)$

i.e., the set of even number of zeros

We can represent this set by finite automata as shown in figure

$M = (\{q_0, q_1\}, \{0\}, \delta, q_0, q_0)$

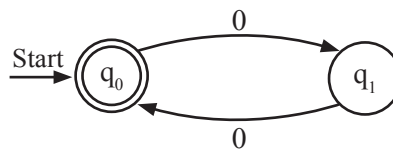


Fig. 3.1 Regular set

The given set L is said to be a regular set because it is represented by finite automata.

3.3 Regular Languages

A language is said to be regular, if it can be marked by a finite-state automation the class of regular languages is denoted by R . The class R is very important since it delimits the language that possesses automation representations that require finite memory when stored in a computer. In the other words, automata are a practical means of manipulating regular languages in analysis or controller synthesis problems. On the other hand, automata are not a practical means for representing non-regular languages, since they would require infinite memory.

3.4 Regular Expressions

Regular expressions are notations for regular languages. Regular languages are set of words, or strings, over an alphabet of characters valid for the language.

- λ is the null string (there are no symbols in this string)
- a^+ is the set of all strings of length greater than or equal to 1

Example

$A = \{a, b\}$ // the alphabet is composed of a and b

$a^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}$

The symbol $*$ is called the Kleene star.

$\emptyset$ (empty set)

λ (empty string)

$()$ delimiter

$\cup$ + union (selection)

Concatenation

The order to describe languages over an alphabet A we introduces a new alphabet A , called a meta-alphabet, that has symbol that correspond to each symbol of A as well as other symbols used to form regular expressions. Namely, if $A = \{a_0, \dots, a_{n-1}\}$, then

$$A = \{a_0, \dots, a_{n-1}, (), \cup, \emptyset, \lambda, *\}$$

Definition: The set of regular expressions over an alphabet A is the set of all words over the meta-alphabet. A defined inductively as follows:

- $a_0, \dots, a_{n-1}$ are regular expressions
- A is a regular expression
- $\emptyset$ is a regular expression

Inductive steps

- If E_0, E_1 are regular expressions, then $(E_0 \cup E_1)$ is a regular expression.
- If E_0, E_1 are regular expressions, then $(E_0 \cdot E_1)$ is a regular expression.
- If e is a regular expression, then (E^*) is a regular expression.

Regular expressions have an important role in computer science applications. In applications involving text, user may want to search for strings that satisfy certain patterns. Regular expressions provide a powerful method for describing such patterns.

3.5 Finite Automata and Regular Expressions

There is a close relationship between a finite automata and regular expression, the relation is as shown in the figure below.

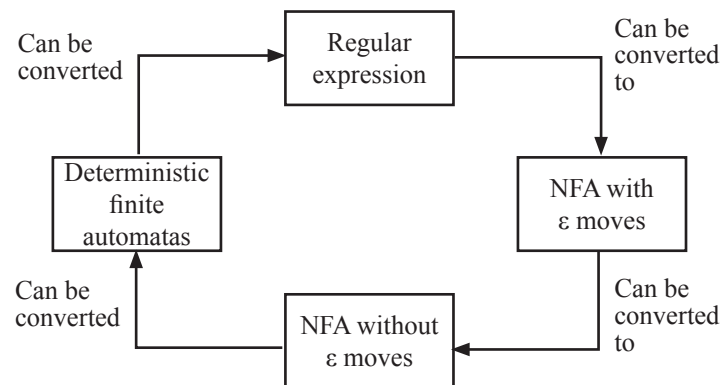


Fig. 3.2 Relationship between FA and regular expression

Fig. 3.2 shows that it is convenient to convert the regular expression to NFA with ϵ -moves.

3.6 Conversion of RE to FA

Here, we see a method of converting regular expressions r into automata recognising $L(r)$. Here, is the main result of this section:

Theorem: Every regular language is FA recognisable.

Let r be a regular expression. We want to prove that $L(r)$ is FA recognisable. In our proof we just follow the inductive definition given for the language $L(r)$ from the previous section and prove that $L(r)$ is FA recognisable. Our proof uses NFA with silent moves.

Base case

Assume that r is defined in the base case. Then, $L(r)$ is either $\{a\sigma\}$ for some $\sigma \in \Sigma$ or $\emptyset$ or $\{\lambda\}$. Clearly in each case $L(r)$ is FA recognisable.

Inductive step

In this step, there are three cases to consider:

Case 1: Assume that r is of the form $(r_1 + r_2)$. We now use the inductive hypothesis applied to r_1 and r_2 . By the hypothesis, the languages $L(r_1)$ and $L(r_2)$ are FA recognisable (and hence DFA recognisable). The language $L(r_1 + r_2)$ is by definition $L(r_1) \cup L(r_2)$. The union of FA recognisable languages is again FA recognisable. Hence, $L(r)$ is FA recognisable.

Case 2: Assume that r is of the form $(r_1 \cdot r_2)$. By the inductive hypothesis, the languages $L(r_1)$ and $L(r_2)$ are FA recognisable. The language $L(r_1 \cdot r_2)$ is by definition $L(r_1) \cdot L(r_2)$. We want to show that $L(r_1) \cdot L(r_2)$ is FA recognisable.

Let $A_1 = (S_1, I_1, T_1, F_1)$ and $A_2 = (S_2, I_2, T_2, F_2)$ be finite automata recognising $L(r_1)$ and $L(r_2)$, respectively. Both automata are NFA that may have silent moves. We may assume that the state sets S_1 and S_2 have no states in common. Using these two automata we want to build a finite automaton recognising $L(r_1) \cdot L(r_2)$.

We now use our method of pretending to be a machine to accept strings in $L(r_1) \cdot L(r_2)$.

Suppose w is an input string. We first pretend that we are the machine A_1 and run A_1 on w . If we never reach a final state of A_1 then we reject w . Assume, that we have reached a final state after processing some prefix of w . In this case we need to non-deterministically choose one of the following actions:

- Continue on running A_1 on w .
- Make a silent transition to the initial state of A_2 and start simulating A_2

This observation suggests that we can put the automata A_1 and A_2 together as follows. We keep the states and transition tables of both machines. We declare the initial states of A_1 to be the initial states, and the final states of A_2 to be the final states of our machine. Finally, we add the λ -transitions from the final states of A_1 to the initial states of A_2 . Based on this description, we formally define the automaton $A = (S, I, T, F)$ recognising $L(r_1) \cdot L(r_2)$ as follows:

- $S = S_1 \cup S_2$
- $I = I_1$

On the input (s, σ) , where $\sigma \in \Sigma \cup \{\lambda\}$, the transition function T is defined according to the following rules.

- If $s \in S_1$ and s is not a final state then $T(s, \sigma) = T_1(s, \sigma)$.
- If $s \in S_1$, s is a final state, and $\sigma \neq \lambda$ then $T(s, \sigma) = T_1(s, \sigma)$.
- If $s \in S_1$, s is a final state, and $\sigma = \lambda$ then $T(s, \sigma) = T_1(s, \sigma) \cup I_2$.
- If $s \in S_2$ then $T(s, \sigma) = T_2(s, \sigma)$.
- $F = F_2$

It is not hard to check that A recognises $L(r_1) \cdot L(r_2)$

Case 3: Assume that r is of the form $(r_1)^*$. By the inductive hypothesis, the language $L(r_1)$ is FA recognisable. The language $L(r_1^*)$ is by definition $L(r_1)^*$. We want to show that $L(r_1)^*$ is FA recognisable.

Let $A_1 = (S_1, q_1, T_1, F_1)$ be a finite automaton recognising $L(r_1)$. We want to construct a finite automaton that recognises the language $L(r_1)^*$. As above, we use our method of pretending to be a machine that accepts strings in $L(r_1)^*$. For an input string w , if it is the empty string, then we accept it because the star of every language contains λ . Otherwise we simulate A_1 and read w .

Every time we reach a final state of A_1 , we have to make a nondeterministic choice. We either continue on running A_1 or make a silent move to one of the initial states of A_1 . Thus, we construct our automaton A recognising $L(r_1)^*$ as follows. We keep all the states of A_1 ; we keep all the transitions of A_1 ; we add λ -transitions from each final state

of A_1 to every initial state of A_1 ; we add one new initial state q_{new} with no outgoing transitions and declare it to be a final state (this is needed to accept the empty string). Here, it is a more formal definition of the automaton $A = (S, I, T, F)$ recognising $L(r_1)^*$:

- $S = S_1 \cup \{q_{\text{new}}\}$, where q_{new} is a state $L(r_1)^*$:
- $I = I_1 \cup \{q_{\text{new}}\}$
- On input (s, σ) , where $\sigma \in \Sigma \cup \{\lambda\}$, the transition function T is defined according to the following rules.
 - If $s \in S_1$ and s is not a final state then $T(s, \sigma) = T_1(s, \sigma)$.
 - If $s \in S_1$, s is a final state, and $\sigma \neq \lambda$ then $T(s, \sigma) = T_1(s, \sigma)$.
 - If $s \in S_1$, s is a final state, and $\sigma = \lambda$ then $T(s, \sigma) = T_1(s, \sigma) \cup I_1$.
 - If $s = q_{\text{new}}$ then $T(s, \sigma) = \emptyset$.
- $F = F_1 \cup \{q_{\text{new}}\}$.

Now, it is not difficult to check that A recognises $L(r_1)^*$. We have proved the theorem.

3.7 Pumping Lemma for Regular Sets

In this section, we discuss an important and basic result, called pumping lemma for regular sets. This lemma gives a necessary condition for an input string to belong to a regular set. It also gives a method for generating (pumping) many input strings from a given string. As the lemma gives a necessary condition, it can be used, to show that certain sets are not regular. It is also useful in the development of algorithms to answer certain questions concerning finite automata such as whether the language accepted by the given FA is finite or infinite.

Theorem:

Pumping lemma

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton with n states. Let L be the regular set accepted by M . Let $w \in L$ and $|w| \geq m$. If $m \geq n$, then there exists x, y, z such that $w = xyz$, $y \neq \epsilon$ and $xy^i z \in L$ for each $i \geq 0$.

Proof: let $w = a_1 a_2 \dots a_m$ $m \geq n$

$\delta(q_0, a_1 a_2 \dots a_i) = q_i$ for $i = 1, 2, \dots, m$;

$Q_1 = \{q_0, q_1, \dots, q_m\}$

This means Q_1 is the sequence of states in the path $w = a_1 a_2 \dots a_m$. As there are only n distinct states, at least two states in Q_1 must coincide. Among various pairs of repeated states, consider the first pair. Assume such a pair as q_j and q_k ($q_j = q_k$).

Then j and k will satisfy the following condition

$$0 \leq j \leq k \leq m$$

The string w now can be decomposed into three substrings as $a_1 a_2 \dots a_j, a_{j+1} \dots a_k, a_{k+1}, \dots, a_m$.

Assume that x, y and z denote these substrings $a_1 a_2 \dots a_j, a_{j+1} \dots a_k, a_{k+1}, \dots, a_m$ respectively. Now since $k \leq m$, $|x, y| \leq n$ and $w = xyz$, therefore, the path w in the transition diagram is shown in the figure given below.

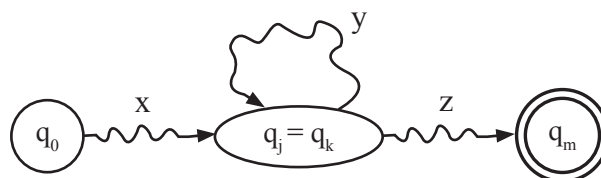


Fig. 3.3 The string w accepted by M

From the figure, following observations are made:

- The automaton ' M ' starts from initial state ' q_0 ' on reading the string ' x ', it reaches state q_j (or q_k as $q_j = q_k$)

- On applying the string 'y', it comes back to q_j . That is after the application of 'y' for each ' $i \geq 0$ ', the automation will be in state q_j .
- On applying 'z', then reaches the final state q_m .
- So, we can conclude ' $xy^i z \in L$ '.

It should be understood that the decomposition of the string 'w' shown above is only valid for the strings of length, greater than or equal to the number of states. For such a string, ' $w = xyz$ ', we can iterate the substring 'y' in xyz as many times as we like and get strings of form ' $xy^i z$ ' which are longer than xyz and are in 'L'.

By considering the path from q_0 and q_k and then the path from q_k to q_m (i.e. without going through the loop), we get a path ending in a final state with the path value 'xz'. This corresponds to the case when ' $i=0$ '.

3.8 Closure Properties of Regular Sets

Closure properties are the theorems, which show that the class of regular language is closed under the operation mentioned. The theorems are of the form, 'if certain languages are regular, and a language L is formed from them by certain operation such as union, intersection etc then L is also regular'. In general, closure properties convey the fact that when languages are regular, then certain related languages are also regular.

The principal closure properties of regular languages are:

The union of two regular languages is regular

If L and M are regular languages, then $L \cup M$

The intersection of two regular languages is regular

If L and M are regular languages, then $L \cap M$

The compliment of two regular languages is regular

If L is a regular language over alphabet Σ , then $\Sigma^* - L$ is also regular language.

The difference of two regular languages is regular

If L and M are regular languages, then $L - M$.

The reversal of a regular language is regular

The reversal of a string means that the string is written backward, i.e., reversal of abcde is edcba.

The reversal of a language is the language consisting of reversal of all its strings, i.e., if

$$L = \{001, 110\} \text{ then}$$

$$L(R) = \{100, 011\}$$

The closure of a regular language is regular

If L is a regular language, then so is L^* .

The concatenation of regular languages is regular

If L and M are regular languages, then LM

The homomorphism of regular language is regular

A homomorphism is a substitution of strings for symbol. Let the function h be defined by $h(0)=a$ and $h(1)=b$ then h applied to 0011 is simply aabb.

If h is a homomorphism on alphabet Σ and string of symbols $w = abcd \dots z$ then

$$h(w) = h(a) h(b) h(c) h(d) \dots h(z)$$

The mathematical definition for homomorphism is:

$$h: \Sigma^* \rightarrow \Gamma^* \text{ such that } \forall x, y \in \Sigma^*$$

A homomorphism can be also be applied to a language by applying it to each of the strings in the language. Let L be a language over alphabet Σ , then,

$$h(L) = \{h(w) \mid w \text{ is in } L\}$$

The theorem can be stated as, 'If L is regular language over language over alphabet Σ , and h is a homomorphism on Σ , then $h(L)$ is also regular'.

The inverse homomorphism of two regular language is regular

Suppose h be a homomorphism from some alphabet Σ to strings in another alphabet T and L be a language over T then $h^{-1}(L)$ is set of strings w in Σ^* such that $h(w)$ is in L .

The theorem states that 'if h is a homomorphism from alphabet Σ to alphabet T , and L is regular language on T , then $h^{-1}(L)$ is also a regular language.'

Summary

- The regular languages are the languages which are accepted by finite automata.
- Regular sets are the sets which are accepted by finite automata.
- A language is said to be regular if it can be marked by a finite-state automation the class of regular languages is denoted by R.
- Regular expressions are notations for regular languages.
- Regular languages are set of words, or strings, over an alphabet of characters valid for the language.
- There is a close relationship between a finite automata and regular expression.
- Every regular language is FA recognisable.
- Pumping lemma gives a necessary condition for an input string to belong to a regular set.
- Closure properties are the theorems, which show that the class of regular language is closed under the operation mentioned

References

- Sunitha, K. V. N. and Kalyani, N., 2010. *Formal Languages and Automated Theory*, Tata McGraw-Hill Education.
- Hopcroft, J. E., 2008. *Introduction to Automated Theory, Languages and Computation*, Pearson Education India.
- *Regular Languages* [Pdf] Available at: <<http://www.cs.uky.edu/~lewis/texts/theory/languages/reg-lang.pdf>> [Accessed 7 June 2013].
- *Regular Languages and Finite Automata* [Pdf] Available at: <<http://www.cl.cam.ac.uk/teaching/0910/RLFA/reglfa.pdf>> [Accessed 7 June 2013].
- Mukherjee, D., 2013. *Automata: Introduction to Grammar* [Video online] Available at: <<https://www.youtube.com/watch?v=sBmg9Nf06tg>> [Accessed 7 June 2013].
- *2010. Context-Free Grammars* [Video online] Available at: <<https://www.youtube.com/watch?v=s0jD8bfvHLg>> [Accessed 7 June 2013].

Recommended Reading

- Webber, A., 2008. *Formal Language: A Practical Introduction*. Franklin, Beedle & Associates, Inc.
- Martin, J., 2002. *Introduction to Languages and the Theory of Computation*, 3rd ed., McGraw-Hill Science/Engineering/Math.
- Rich, E. A., 2007. *Automata, Computability and Complexity: Theory and Applications*, 1st ed., Prentice Hall.

Self Assessment

1. Regular sets are the sets which are accepted by _____ automata.
 - a. infinite
 - b. logical
 - c. finite
 - d. pushdown
2. There is a close relationship between a finite automata and _____.
 - a. regular expression
 - b. regular languages
 - c. context-free grammars
 - d. pushdown automata
3. _____ properties are the theorems, which show that the class of regular language is closed under the operation mentioned.
 - a. Finite
 - b. Regular
 - c. Computation
 - d. Closure
4. Every regular language is _____ recognisable.
 - a. PDA
 - b. FA
 - c. CFG
 - d. PCP
5. Pumping lemma gives a necessary condition for an input string to belong to a regular _____.
 - a. index
 - b. language
 - c. set
 - d. grammar
6. Which of the following statement is true?.
 - a. Regular expressions are notations for regular languages.
 - b. Regular expressions are notations for grammars.
 - c. Regular expressions are notations for context free language.
 - d. Regular expressions are notations for irregular languages.

7. Which of the following statement is true?
 - a. The regular languages are the languages which are accepted by infinite automata.
 - b. The regular languages are the languages which are accepted by pushdown automata.
 - c. The regular languages are the languages which are not accepted by finite automata.
 - d. The regular languages are the languages which are accepted by finite automata.

8. Which of the following statement is false?
 - a. The regular languages are the languages which are accepted by finite automata.
 - b. Irregular languages are set of words, or strings, over an alphabet of characters valid for the language.
 - c. Regular languages are set of words, or strings, over an alphabet of characters valid for the language.
 - d. There is a close relationship between a finite automata and regular expression.

9. Which of the following statement is true?
 - a. The reversal of a regular language is irregular.
 - b. The reversal of a regular language is regular.
 - c. The reversal of a regular language is reversible.
 - d. The reversal of a regular language is finite.

10. Which of the following have an important role in computer science applications?
 - a. Regular expressions
 - b. Regular languages
 - c. Context-free grammars
 - d. Push down automata

Chapter IV

Context Free Grammars

Aim

The aim of the chapter is to:

- explain the concept context free grammars
- explicate Chomsky normal form
- enlist the properties of context free language

Objectives

The objectives of this chapter are to:

- elucidate ambiguity in CFG and its removal
- explain Greiback normal form
- determine pumping lemma for context free languages

Learning outcome

At the end of this chapter, you will be able to:

- understand the concept context free grammars
- recognise Chomsky normal form and Greiback normal form
- identify the properties of CFL

4.1 Introduction

In this chapter, we will introduce context – free grammars, a more powerful method, than finite automata and regular expressions, of describing languages. Such grammars can describe certain features that have a recursive structure which makes them useful in a variety of applications (study of human languages, compilation of programming languages).

4.2 Context-free Languages

Superficially thinking, one might view modern computers as deterministic finite automata. Computers have finite memory, so there are only finitely many states the computer can be at, and the previous state of the computer determines the next state, so the machine has deterministic state transitions. This approach is, however, not reasonable. First of all, the number of states is indeed finite but that finite number is astronomical. And even worse: the memory of the computer is extendible, so the approach is not even theoretically correct. A more accurate mathematical model is needed.

Our next step into the direction of ‘correcting’, the notion of computation is to overcome the limitations of finite memory by introducing new models that allow arbitrarily large memory. The memory can be organised and accessed in different ways. Our next model, context-free languages, assumes an infinite memory that is organised as a stack (=last-in-first-out memory).

As we are going to see, this does not yet capture the full power of computation, but it provides a natural generalisation of regular languages. Every regular language is a context-free language, but also some non-regular languages are included. For example $\{a^n b^n \mid n \geq 0\}$ is a context-free language. On the other hand, there are many simple languages which are not context-free, for example $\{a^n b^n c^n \mid n \geq 0\}$.

Context-free languages have applications in compiler design (parsers). The syntax of programming languages is often given in the form of context-free grammar, or equivalent Backus-Naur form (BN-form).

A grammar consists of a collection of substitution rules, also called productions. Each rule appears as a line in the grammar and comprises a symbol and a string, separated by an arrow. The symbol is called a variable. The string consists of variables and other symbols called terminals.

4.3 Context-free Grammars

A grammar consists of a collection of substitution rules, also called productions. Each rule appears as a line in the grammar and comprises a symbol and a string, separated by an arrow. The symbol is called a variable. The string consists of variables and other symbols called terminals.

Similar to regular languages, we have different ways of representing context-free languages. Let us first consider generative devices, so-called context-free grammars.

Let us begin with an example of a context-free grammar. It uses variables ‘A’ and ‘B’, and terminal symbols ‘a’ and ‘b’. We have the following productions:

$A \rightarrow AbA$
 $A \rightarrow B$
 $B \rightarrow aBa$
 $B \rightarrow b$

In a word containing variables, and possibly terminals, one can replace any variable with the word on the right hand side of a production for that variable. For example, in the word,

aBabAbA

One can, for example, replace the first occurrence of variable A by AbA, obtaining the word,

$$aBabAbAbA$$

Then one can decide, for example, to replace the variable B by b, which gives the word,

$$ababAbAbA$$

Rewriting one variable is called a derivation step and we denote,

$$aBabAbA \Rightarrow aBabAbAbA \Rightarrow ababAbAbA$$

Derivation is non-deterministic: Usually one has many choices of how to proceed. We can continue the derivation as long as there exists a variables in the word. Once the word contains only terminal symbols, the derivation terminates. One variable is called the start symbol. In the case of this example, let 'A' be the start symbol.

All derivations start with the start symbol 'A', i.e., initially the word is 'A'. The language defined by the grammar contains all strings of terminal symbols that can be obtained from the start symbol by applying the productions on the variables. For example, word 'aabaababa' is in the language defined by our grammar since it is derived by;

$$A \Rightarrow AbA \Rightarrow BbA \Rightarrow aBabA \Rightarrow aaBaabA \Rightarrow aabaabA \Rightarrow aabaabB \Rightarrow aabaabaBa \Rightarrow aabaababa.$$

On the other hand, word 'aaa' is not in the language because the only way to produce letter 'a' is to use the production $B \rightarrow aBa$, and it creates two a's, so the number of a's has to be even.

Here is a precise definition of context-free grammars. A context-free grammar $G = (V; T; P; S)$ consists of: two disjoint finite alphabets, V and T, containing variables (=nonterminals) and terminals, respectively,

- a finite set P of productions of the form,

$$A \rightarrow \alpha$$

Where $A \in V$ is a variable, and $\alpha \in (V \cup T)^*$ is a word of terminals and variables.

- A start symbol $S \in V$.

For any $\alpha, \beta \in (T \cup V)^*$, we denote,

$$\alpha \Rightarrow \beta$$

If β is obtained from α by rewriting one variable in α using some production from P, that is, if $\alpha = uAv$ and $\beta = uwv$ for some $u, v, w \in (V \cup T)^*$ and $A \in V$, and $A \rightarrow w$ is in P. This is called a derivation step. The reflexive, transitive closure of $\Rightarrow$ is denoted by $\Rightarrow^*$, $\alpha \Rightarrow^* \beta$ if there is a sequence of derivation steps,

$$\alpha = \alpha_0 \Rightarrow \alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n = \beta$$

That starts with α and leads to β . The number n of derivation steps can be zero, so $\alpha \Rightarrow^* \alpha$ always. We say that α derives β . If we want to emphasise that a derivation takes n steps, we may use the notation $\Rightarrow^n$,

A word $\alpha \in (V \cup T)^*$ is called a sentential form if it can be derived from the start symbol S, i.e., if,

$$S \Rightarrow^* \alpha$$

The language L (G) generated by grammar G consists of all sentential forms that contain only terminals. In other words,

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}$$

A language is called a context-free language if it is $L(G)$ for some context-free grammar G .

Example: Consider the grammar $G = (V; T; P; S)$ where $V = \{S\}$, $T = \{a; b\}$, and P contains the productions

$$\begin{aligned} S &\rightarrow aSb \\ S &\rightarrow \varepsilon \end{aligned}$$

To make notations shorter, we may use the following convention: productions of the same variable may be combined on one line, separated by symbol $|$. So we may write,

$$S \rightarrow aSb | \varepsilon$$

$$\text{We easily see that } L(G) = \{a^n b^n \mid n \geq 0\}.$$

Let us determine $L(G)$. Clearly every word that belongs to $L(G)$ must contain equally many a 's and b 's. (Every production adds the same number of a 's and b 's to the sentential form.)

Let us now prove that $L(G)$ contains all the words $w \in \{a, b\}^*$ with equally many a 's and b 's.

Let,

$$L = \{w \in \{a, b\}^* \mid w \text{ has equally many } a\text{'s and } b\text{'s}\}$$

We observe first that any non-empty word $w \in L$ can be written as,

$$w = a u b v \text{ or } w = b u a v$$

for some $u, v \in L$. Let us prove this fact. Assume ' $w \in L$ ' and ' w ' starts with letter ' a ' (if it starts with letter ' b ', the proof is symmetric). Let ' au ' be the longest prefix of ' w ' with more a 's than b 's. The next letter coming after ' au ' has to be ' b ' because ' aua ' has more a 's than b 's and is longer than ' au '. So, w starts ' $a u b$ '.... In the word ' au ', the number of a 's is exactly one greater than the number of b 's:

If the difference would be more than one, then also prefix $a u b$ would contain more a 's than b 's, and it is longer than ' au '. So ' u ' has equally many a 's and b 's. The suffix ' v ' that follows ' $a u b$ ' also has equally many a 's and b 's. So, ' $w = a u b v$ ' and ' $u, v \in L$ '.

Now, we are ready to prove that every word ' $w \in L$ ' is in ' $L(G)$ '. We use induction on the length of ' w '.

- If $|w| = 0$ then $w = \varepsilon \in L(G)$ because $S \rightarrow \varepsilon$ is in P .
- Assume $|w| > 0$, and we know that every short word with equal number of a 's and b 's can be derived from S . We demonstrated above that,

$$w = a u b v \text{ or } w = b u a v$$

Where ' u ' and ' v ' are words that contain equally many a 's and b 's. According to the inductive hypothesis,

$$S \Rightarrow^* u \text{ and } S \Rightarrow^* v$$

If ' $w = a u b v$ ' then, we can derive w as follows:

$$S \Rightarrow a S b S \Rightarrow^* a u b v.$$

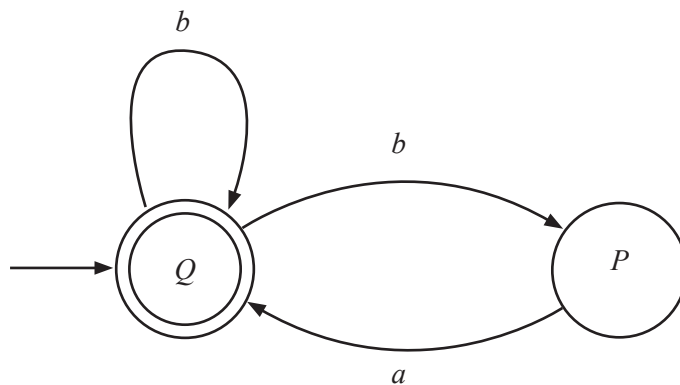
The case 'w = b u a v' is similar. In either case, we have a derivation for w so $w \in L(G)$.

For example, ababba has the following derivation:

$$S \Rightarrow aSbS \Rightarrow abSaSbS \Rightarrow abaSbS \Rightarrow ababS \Rightarrow ababbSaS \Rightarrow ababbaS \Rightarrow ababba.$$

Let us prove next that every regular language can be generated by a context-free grammar.

Example: Let L be the regular language recognised by the NFA A



Let us construct a grammar G that generates the same language L. The variables of G are the states {Q; P} and the terminals are {a; b}. The start symbol is the initial state 'Q'. Every transition of A is simulated by a production:

$$\begin{aligned} Q &\rightarrow bQ \\ Q &\rightarrow bP \\ P &\rightarrow aQ \end{aligned}$$

Whenever the automaton reads a letter and goes to state $q = Q$ or P , the grammar produces the same letter and changes the variable to indicate the current state 'q'.

Each sentential form consists of a terminal word w followed by one variable. Clearly, word 'wq' is a sentential form, if and only if the automaton can go to state 'q' reading input 'w'. For example, 'bbaQ' is a sentential form because the machine can read bba ending up in state Q:

$$Q \Rightarrow bQ \Rightarrow bbP \Rightarrow bbaQ.$$

To terminate the simulation, we add the transition:

$$Q \rightarrow \epsilon.$$

This is correct because 'Q' is a final state: the generation is terminated if the corresponding computation by 'A' is accepting. For example, when accepting the word 'bab', the automaton is in states Q, P, Q, Q, in this order. The corresponding derivation by the grammar is

$$Q \Rightarrow bP \Rightarrow baQ \Rightarrow babQ \Rightarrow bab$$

It is easy to prove that $L(G) = L$.

The similar construction can be done on any NFA, proving the following theorem.

Theorem: Every regular language is context-free.

Proof:

Let L be a regular language and let $A = (Q, \Sigma, \delta, q_0, F)$ be an NFA such that $L = L(A)$. An equivalent grammar is $G = (V; T; P; S)$ where

$$\begin{aligned} V &= Q \\ T &= \Sigma \\ S &= q_0 \end{aligned}$$

and P contains a production $q \rightarrow ap$ for all $q, p \in Q$ and $a \in \Sigma$ such that $p \in \delta(q; a)$, and a production $q \rightarrow \epsilon$ for every $q \in F$. This grammar only derives sentential forms that contain at most one variable, and the variable must appear as the last symbol of the form. Moreover, using a straightforward induction on the length of w one sees that $S \Rightarrow^* wq$ for $w \in \Sigma^*$ if and only if $q \in \delta(q_0, w)$. Hence, $S \Rightarrow^* w$ for $w \in \Sigma^*$ if and only if $w \in L(A)$.

A grammar is called right linear, if all productions are of the forms;

$A \rightarrow wB$, and

$A \rightarrow w$

For strings of terminals ' $w \in T^*$ ' and variables $A, B \in V$. In other words, the right hand side of a production may contain only one variable, and it has to be the last symbol.

The construction in the proof of above theorem always produces a right linear grammar, so every regular language is generated by a right linear grammar. The converse is also easily seen true: a language generated by a right linear grammar is always regular. This observation provides yet another equivalent way to define regular languages, in terms of right-linear grammars.

4.4 Ambiguity in CFG and its Removal

Grammars ' G ' and ' G' ' are called equivalent if ' $L(G) = L(G')$ '. Certain types of productions are undesirable in a grammar, and we would like to find an equivalent grammar that does not contain such productions. In the following, we want to remove productions of the forms,

$A \rightarrow \epsilon$ ("ε-production"), and

$A \rightarrow B$ ("unit production")

Where ' A ' and ' B ' are variables. We also want to remove symbols that are unnecessary in the sense in that they are not used in any terminating derivations. The simplification is done in the following order:

- Remove ε-productions
- Remove unit productions
- Remove variables that do not derive any terminal strings
- Remove symbols that cannot be reached from the start symbol

All four steps are effective, i.e., doable by an algorithm. We use the following grammar as a running example:

$S \rightarrow AC \mid aB \mid AD$

$A \rightarrow \epsilon \mid ab \mid S$

$B \rightarrow Aa \mid AB$

$C \rightarrow Aaa \mid \epsilon$

$D \rightarrow Ebd$

$E \rightarrow bb$

Step 1: Remove ε-productions

- If $\epsilon \notin L(G)$ we can construct an equivalent grammar G' that does not have any ε-productions.
- If $\epsilon \in L(G)$ we can construct a grammar G' that does not have any ε-productions, and $L(G') = L(G) \setminus \{\epsilon\}$.

Let us call a variable X nullable if ' $X \Rightarrow^* \epsilon$ ':

- Find all nullable variables using a marking procedure: First, mark nullable all variables X that have a production $X \rightarrow \epsilon$. Then mark variables Y that have productions $Y \rightarrow \alpha$ where α is a string of variables that have all been already marked nullable. Repeat this until no new variables can be marked. It is clear that all nullable variables are found. Note that $\epsilon \in L(G)$ if and only if the start symbol S is nullable. In the sample grammar above, variables A , C and S are nullable.
- Construct a new set P' of productions as follows: For each original production,

$$A \rightarrow X_1 X_2 \dots X_n$$

In P , where all $X_i \in V \cup T$, we include in P' all productions,

$$A \rightarrow \alpha_1 \alpha_2 \dots \alpha_n$$

Where,

$$\begin{aligned} \alpha_i &= X_i \text{ if } X_i \text{ is not a nullable variable,} \\ \alpha_i &= \epsilon \text{ or } \alpha_i = X_i \text{ if } X_i \text{ is a nullable variable,} \\ \alpha_1 \alpha_2 \dots \alpha_n &\neq \epsilon. \end{aligned}$$

If several X_i are nullable, all combinations of $\alpha_i = \epsilon$ and X_i are used. One original production can result in up to 2^k new productions if the right-hand side contains k nullable variables. In our example, the new productions will be;

$$\begin{aligned} S &\rightarrow AC \mid C \mid A \mid aB \mid AD \mid D \\ A &\rightarrow ab \mid S \\ B &\rightarrow Aa \mid a \mid AB \mid B \\ C &\rightarrow AAa \mid Aa \mid a \\ D &\rightarrow EbD \\ E &\rightarrow bb \end{aligned}$$

There are no ϵ -productions. The language generated by the new grammar is $L(G) \setminus \{\epsilon\}$.

The construction works because the empty string ϵ is directly "plugged in" the right hand side of the production, whenever there is a variable that can produce ". When we use that production during a derivation, we decide at that point whether the nullable variable on the right hand side will be used to produce an empty string, or a non-empty string. For example, an original derivation;

$$\begin{aligned} S &\Rightarrow AC \Rightarrow abC \Rightarrow ab \\ &\text{is replaced by the more direct derivation} \\ S &\Rightarrow A \Rightarrow ab: \end{aligned}$$

Since C derived ϵ in the old derivation, we used $S \rightarrow A$ instead of $S \rightarrow AC$.

Step 2: Remove unit productions. Assume we have removed all ϵ -productions from P . The right-hand side of every production in P is some non-empty word. Let us find an equivalent grammar that does not contain ϵ -productions or unit productions. The idea is similar to Step 1: We anticipate all possible unit derivations, and plug them in directly into the productions.

- For every variable A , find all variables B , such that

$$A \Rightarrow^* B:$$

All such variable-to-variable derivations $A \Rightarrow^* B$ can be found using a reachability algorithm in the directed graph whose vertices are the variables and there is an edge from A to B if

$A \rightarrow B$ is in P . Clearly, $A \Rightarrow^* B$ if there is a path in the graph from A to B . In our example, after the simplification step 1, the unit productions are,

$S \rightarrow C \mid A \mid D$
 $A \rightarrow S$
 $B \rightarrow B$

So, we have the following letter-to-letter derivations:

$S \Rightarrow^* S \mid C \mid A \mid D$
 $A \Rightarrow^* A \mid S \mid C \mid D$
 $B \Rightarrow^* B$
 $C \Rightarrow^* C$
 $D \Rightarrow^* D$
 $E \Rightarrow^* E$

- For every pair of variables X and Y , $X \neq Y$, such that $X \Rightarrow^* Y$ and $Y \Rightarrow^* X$, we can remove one of the variables: They both derive exactly the same words. So we trim P by replacing Y by X everywhere, both left- and right-hand sides of all productions.

In our example we can remove A since it derives same words as S . This leaves us with a smaller number of variables:

$S \rightarrow \mid SC \mid C \mid S \mid aB \mid SD \mid D \mid ab$
 $B \rightarrow Sa \mid a \mid SB \mid B$
 $C \rightarrow SSa \mid Sa \mid a$
 $D \rightarrow EbD$
 $E \rightarrow bb$

- Now we construct P' from the trimmed set P . For every non-unit production

$$Y \rightarrow \alpha$$

in P and every variable X such that $X \Rightarrow^* Y$, we include in P' the non-unit production

$$X \rightarrow \alpha$$

We obtain an equivalent grammar whose derivations may be shorter: An original derivation

$$\beta X_1 \Rightarrow^* \beta Y_1 \Rightarrow \beta \alpha_1$$

that used unit productions $X \Rightarrow^* Y$ is replaced by a single derivation step

$$\beta X_1 \Rightarrow \beta \alpha_1$$

In our example, we end up with the following productions,

$S \rightarrow \mid SC \mid aB \mid SD \mid ab \mid SSa \mid Sa \mid a \mid EbD$
 $B \rightarrow Sa \mid a \mid SB$
 $C \rightarrow SSa \mid Sa \mid a$
 $D \rightarrow EbD$
 $E \rightarrow bb$

Step 3: Remove variables that do not derive any terminal strings.

Such variables can be simply removed together with all productions containing the variable on either side of the production. This does not affect the generated language since the variable is not used in any terminating derivation.

To find variables that do generate some terminal string, we apply similar marking procedure as with ϵ -productions: First we mark all variables X such that there is a production,

$$X \rightarrow w$$

in P , where $w \in T^*$ contains only terminals. Then we mark variables Y such that there is a production,

$$Y \rightarrow \alpha$$

in P where α contains only terminals, and variables that have already been marked. This is repeated until no more variables can be marked. The variables that have not been marked at the end can be removed.

In our sample grammar, variable D does not derive any terminal string and can be removed.

We obtain the following grammar,

$S \rightarrow SC \mid aB \mid SD \mid ab \mid SSa \mid Sa \mid a$
 $B \rightarrow Sa \mid a \mid SB$
 $C \rightarrow SSa \mid Sa \mid a$
 $E \rightarrow bb$

Step 4: Remove symbols (variables and terminals) that are not reachable from the start symbol S .

We use a similar marking procedure: Initially mark the start symbol S . Then mark all symbols appearing on the right hand side of a production for a variable that has been already marked. Repeat until no new symbols can be marked. All symbols that have not been marked cannot be reached from the initial symbol, and they can be removed. Note that it is important that step 3 is executed before step 4, not the other way around. Step 3 may introduce new unreachable symbols.

In our example, variable E is unreachable and can be removed. (But it became unreachable only after we removed D !) We end up with the grammar,

$S \rightarrow SC \mid aB \mid ab \mid SSa \mid Sa \mid a$
 $B \rightarrow Sa \mid a \mid SB$
 $C \rightarrow SSa \mid Sa \mid a$

After steps 1, 2, 3 and 4 we have found an equivalent (except the possible loss of the empty word ϵ) grammar that does not have ϵ -productions, unit productions, or useless symbols.

If the empty word was in the original language and we want to include it in the new grammar we may introduce a non-recursive start symbol. That is, we add a new variable S' , make it the start symbol, and add the productions

$S' \rightarrow \alpha$; for every production $S \rightarrow \alpha$ from the original start symbol S ,

$S' \rightarrow \epsilon$

Where S is the old start symbol. The new grammar is equivalent to the original one, and it has only one ϵ -production that can be used only once.

4.5 Chomsky Normal Form and Greiback Normal Form

In many proofs, it is useful to be able to assume that the productions in the grammar are of some restricted, simple form. Several such simplified types of grammars are known to be equivalent to general context-free grammars. They are called normal forms of grammars.

A grammar G is said to be in the Chomsky Normal Form (CNF), if all productions are of the forms;

$A \rightarrow BC$; or

$A \rightarrow a$

Where A, B and C are variables, and A is a terminal. Right-hand-sides of productions consist of two variables, or one terminal.

Theorem: Every context-free language L without ϵ is generated by a grammar that is in the Chomsky normal form. An equivalent CNF grammar can be constructed effectively.

Proof:

Let G be a grammar that generates L . First we use the simplification steps 1, 2, 3 and 4 above to find an equivalent grammar G_0 that does not contain ϵ -productions or unit productions. Because there are no unit productions, all productions with one symbol on the right hand side are terminal productions $A \rightarrow a$, i.e., they are already in the Chomsky normal form.

We only need to work on productions;

$$A \rightarrow X_1 X_2 \dots X_n$$

Where $n \geq 2$, and $X_i \in V \cup T$. For every terminal symbol 'a' that appears on the right hand side we introduce a new variable V_a . If ' $X_i = a$ ', this replaced ' X_i ' with ' V_a '. After this is done for all terminals, the right hand side of the production contains only variables. For example, the production,

$$S \rightarrow cAbbS$$

will be replaced by the production,

$$S \rightarrow V_c A V_b V_b S$$

Where V_b and V_c are the new variables,

Then, we add new terminal productions,

$$V_a \rightarrow a$$

for all terminals 'a'. The only word derivable from V_a is a. Note that these productions are in the Chomsky normal form.

Clearly the new grammar is equivalent to the original one: Every application of the original production,

$$S \rightarrow cAbbS$$

is replaced by a sequence of derivation steps that first uses production,

$$S \rightarrow V_c A V_b V_b S$$

and then applies productions,

$$V_c \rightarrow c \text{ and } V_b \rightarrow b$$

to replace all occurrences of variables 'Vc' and 'Vb' by terminals 'c' and 'b'.

So far we have constructed an equivalent grammar whose productions are of the forms,

$$A \rightarrow B_1 B_2 \dots B_n, \text{ and} \\ A \rightarrow a$$

Where $n \geq 2$, and $A, B_1, B_2, \dots, B_n$ are variables, and a denotes a terminal symbol.

The only productions that are not in CNF are,

$$A \rightarrow B_1 B_2 \dots B_n$$

Where $n \geq 3$.

For each such production we introduce $n - 2$ new variables $D_1, D_2, \dots, D_{n-2}$, and replace the production by the productions,

$$\begin{aligned} A &\rightarrow B_1 D_1 \\ D_1 &\rightarrow B_2 D_2 \\ D_2 &\rightarrow B_3 D_3 \\ &\dots \\ D_{n-3} &\rightarrow B_{n-2} D_{n-2} \\ D_{n-2} &\rightarrow B_{n-1} B_n \end{aligned}$$

One application of the original production gives the same result as applying the new productions sequentially, one after the other. The new grammar is clearly equivalent to the original one.

Another normal form is so-called Greibach normal form (or GNF). A grammar is in Greibach normal form if all productions are of the form,

$$A \rightarrow a B_1 B_2 \dots B_n$$

Where $n \geq 0$ and $A, B_1, B_2, \dots, B_n$ are variables, and 'a' is a terminal symbol. In other words, all productions contain exactly one terminal symbol and it is the first symbol on the right hand side of the production.

4.6 Pumping Lemma for Context Free Languages

Theorem: If G is any context-free grammar in Chomsky Normal Form with p live productions and w is any word generated by G with length $> 2^p$, we can subdivide w into five pieces $uvxyz$ such that $x \neq \Lambda$, v and y are not both Λ and $|vxy| \leq 2^p$ and all words of the form uv^nxy^nz for $n \geq 0$ can also be generated by grammar G .

Proof: If the length of w is $> 2^p$, then there are always self-embedded non-terminals in any derivation tree for w . Choose one such self-embedded non-terminal, call it P , and let the first production used for P be $P \rightarrow QR$. Consider the part of the tree generated from the first P . This part of the tree tells us how to subdivide the string into its five parts. The sub-string vxy is made up of all the letters generated from the first occurrence of P . The sub-string x is made up of all the letters generated by the second occurrence of P . The string v contains letters generated from the first P , but to the left of the letters generated by the second P , and y contains letters generated by the first P to the right of those generated by the second P . The string u contains all letters to the left of v and the string z contains all letters to the right of y . By using the production $P \rightarrow QR$ more times, the strings v and y are repeated in place or 'pumped'. If we use the production $P \rightarrow QR$ only once instead of twice, the tree generates the string uxz .

Here is an example of a derivation that produces a self-embedded non-terminal and the resulting division of the string,

$$\begin{aligned} S &\rightarrow PQ \\ Q &\rightarrow QS \\ p &\rightarrow a \end{aligned}$$

If we pump 'v' and 'y' once, we get the string 'a| b| ab| ab = ababab' three times, it produces a| b| ab| ab| ab = abababab, etc.

Using the pumping lemma for CFLs

We use the Pumping Lemma for context-free languages to prove that a language is not context-free. The proofs are always the same:

- Assume that the language in question is context-free and that the Pumping Lemma thus applies.
- Pick the string w , $|w| > 2^p$
- Sub-divide w into $uvxyz$ such that $|vxy| < 2^p$
- Pick i so that uv^ixy^iz is not in the language. As in pumping lemma $uvxyz \in L$, but it is not true. So, our assumption is not correct and the language in the question is not CFL.

4.7 Enumeration of Properties of CFL

Since the set of regular language is closed under all the operations of union, concatenation, Kleen star, intersection and complement. The set of context free languages is closed under union, concatenation, Kleen star only.

Union

Theorem: if L_1 and L_2 are context-free languages, then $L_1 \cup L_2$ is a context-free language.

Proof: If L_1 and L_2 are context-free languages, then each of them has a context-free grammar; call the grammars G_1 and G_2 . Our proof requires that the grammars have no non-terminals in common. So we shall subscript all of G_1 's non-terminals with a 1 and subscript all of G_2 's non-terminals with a 2. Now combine the two grammars into one grammar that will generate the union of the two languages. To do this, we add one new non-terminal, S , and two new productions,

$$S \rightarrow S_1 \\ | S_2$$

S is the starting non-terminal for the new union grammar and can be replaced either by the starting non-terminal for G_1 or for G_2 , thereby generating either a string from L_1 or from L_2 . Since the non-terminals of the two original languages are completely different, and once we begin using one of the original grammars, we must complete the derivation using only the rules from that original grammar. Note that there is no need for the alphabets of the two languages to be the same.

Concatenation

Theorem: If L_1 and L_2 are context-free languages, then $L_1 L_2$ is a context-free language.

Proof: This proof is similar to the last one. First, subscript all of the non-terminals of G_1 with a 1 and all the non-terminals of G_2 with a 2. Then, we add a new non-terminal, S , and one new rule to the combined grammar

$$S \rightarrow S_1 S_2$$

S is the starting non-terminal for the concatenation grammar and is replaced by the concatenation of the two original starting non-terminals.

Kleene Star

Theorem: If L is a context-free language, then L^* is a context-free language.

Proof: Subscript the non-terminals of the grammar for L with a 1. Then add a new starting non-terminal, S , and the rules

$$S \rightarrow S_1 S \\ | \Lambda$$

The rule $S \rightarrow S_1 S$ is used once for each string of L that we want in the string of L^* , then the rule $S \rightarrow \Lambda$ is used to kill off the S .

Intersection

Now, we will show that the set of context-free languages is not closed under intersection. Think about the two languages $L_1 = \{a^n b^n c^m | n, m \geq 0\}$ and $L_2 = \{a^m b^n c^n | n, m \geq 0\}$. These are both context-free languages and we can give a grammar for each one:

G_1 :

$S \rightarrow AB$

$A \rightarrow aAb$

$|\Lambda$

$B \rightarrow cB$

$|\Lambda$

G_2 :

$S \rightarrow AB$

$A \rightarrow aA$

$|\Lambda$

$B \rightarrow bBc$

$|\Lambda$

The strings in L_1 contain the same number of a's as b's, while the strings in L_2 contain the same number of b's as c's. Strings that have to be both in L_1 and in L_2 , i.e., strings in the intersection, must have the same numbers of a's as b's and the same number of b's as c's.

Thus, ' $L_1 \cap L_2 = \{a^n b^n c^n | n \geq 0\}$ '. Using pumping lemma for context-free languages it can be proved easily that ' $\{a^n b^n c^n | n \geq 0\}$ ' is not context-free language. So, the class of context-free languages is not closed under intersection.

Although the set is not closed under intersection, there are cases in which the intersection of two context-free languages is context-free. All regular languages are context-free, and the intersection of two regular languages is regular. We have some other special cases in which an intersection of two context-free languages is context free.

Suppose that L_1 and L_2 are context-free languages and that $L_1 \subseteq L_2$. Then $L_2 \cap L_1 = L_1$ which is a context-free language. An example is $EQUAL \cap \{a^n b^n\}$. Since strings in $\{a^n b^n\}$ always have the same number of a's as b's, the intersection of these two languages is the set $\{a^n b^n\}$, which is context-free.

Another special case is the intersection of a regular language with a non-regular context-free language. In this case, the intersection will always be context-free. An example is the intersection of $L_1 = a^+ b^+ a^+$, which is regular, with $L_2 = PALINDROME$.

$L_1 \cap L_2 = \{a^n b^m a^n | m, n \geq 0\}$. This language is context-free.

Complement

The set of context-free languages is not closed under complement, although there are again cases in which the complement of a context-free language is context-free.

Theorem: The set of context-free languages is not closed under complement.

Proof: Suppose the set is closed under complement. Then, if L_1 and L_2 are context-free, so are L_1' and L_2' . Since the set is closed under union, $L_1' \cup L_2'$ is also context-free, as is $(L_1' \cup L_2')'$. But, this last expression is equivalent to $L_1 \cap L_2$ which is not guaranteed to be context-free. So, our assumption must be incorrect and the set is not closed under complement.

Here is an example of a context-free language whose complement is not context-free. The language $\{a^n b^n c^n | n \geq 1\}$ is not context-free, but the author proves that the complement of this language is the union of seven different context-free languages and is thus, context-free.

Strings that are not in $\{a^n b^n c^n \mid n \geq 1\}$ must be in one of the following languages:

1. $M_{pq} = \{a^p b^q c^r \mid p, q, r \geq 1 \text{ and } p > q\}$ (more a's than b's)
2. $M_{qp} = \{a^p b^q c^r \mid p, q, r \geq 1 \text{ and } q > p\}$ (more b's than a's)
3. $M_{pr} = \{a^p b^q c^r \mid p, q, r \geq 1 \text{ and } s > r\}$ (more a's than c's)
4. $M_{rp} = \{a^p b^q c^r \mid p, q, r \geq 1 \text{ and } r > p\}$ (more c's than a's)
5. $M =$ the complement of $a^+ b^+ c^+$ (letters out of order)

Using Closure Properties

Sometimes, we can use closure properties to prove that a language is not context-free. Consider the language our author calls **DOUBLEWORD** $= \{ww \mid w \in (a+b)^*\}$. Is this language context-free? Assume that it is. Form the intersection of **DOUBLEWORD** with the regular language $a^+ b^+ a^+ b^+$, we know that the intersection of a context-free language and a regular language is always context-free. The intersection of **DOUBLEWORD** and is $\{a^n b^m a^n b^m \mid n, m \geq 1\}$. But, this language is not context-free, so **DOUBLEWORD** cannot be context-free.

Think carefully when doing unions and intersections of languages if one is a superset of the other. The union of **PALINDROME** and $(a+b)^*$ is $(a+b)^*$, which is regular. So, sometimes the union of a context-free language and a regular language is regular. The union of **PALINDROME** and a^* is **PALINDROME**, which is context-free but not regular.

Summary

- Every regular language is a context-free language, but also some non-regular languages are included.
- Context free grammars is a more powerful method, than finite automata and regular expressions for describing languages.
- A grammar consists of a collection of substitution rules, also called productions.
- The string consists of variables and other symbols called terminals.
- Similar to regular languages, we have different ways of representing context-free languages.
- Every regular language is context-free.
- A grammar G is said to be in the Chomsky normal form (or CNF) if all productions are of the forms $A \rightarrow BC$
- A grammar is in Greibach normal form if all productions are of the form $A \rightarrow aB_1B_2 \dots B_n$
- The set of context free languages is closed under union, concatenation, Kleen star only.

References

- Natarajan, A. M., 2005. *Theory of Automata and Formal Languages*, New Age International.
- Krithivasan, K. 2009. *Introduction to Formal Languages, Automata Theory and computation*, Dorling Kindersley(India) Pvt.Ltd.
- *Context-free grammars and languages* [Pdf] Available at: <<http://www.cs.bath.ac.uk/~cf/cm10020/lecture07.pdf>> [Accessed 7 June 2013].
- *Context Free Grammars* [Pdf] Available at: <<http://people.cs.umass.edu/~mccallum/courses/inlp2007/lect5-cfg.pdf>>[Accessed 7 June 2013].
- *Lecture 5-Context Free Languages* [Video online] Available at:<<https://www.youtube.com/watch?v=7axUgepqCFc>>[Accessed 7 June 2013].
- *Automata Theory : Context Free Grammar (CFG) to Chomsky Normal Form (CNF) part 1* [Video online] Available at: <<https://www.youtube.com/watch?v=zCmGPUZM-RE>> [Accessed 7 June 2013].

Recommended Reading

- Laura, K., 2010. *Parsing Beyond Context-Free Grammars*, 1st ed., Springer.
- Higuera, C., 2010. *Grammatical Inference: Learning Automata and Grammars*. Cambridge University Press,
- Shallit, J., 2008. *A Second Course in Formal Languages and Automata Theory*, 1st ed., Cambridge University Press.

Self Assessment

1. A grammar consists of a collection of substitution rules, also called _____.
 - a. productions
 - b. report
 - c. language
 - d. alphabets
2. The string consists of variables and other symbols called _____.
 - a. nodes
 - b. terminals
 - c. integers
 - d. alphabets
3. The set of _____ languages is closed under union, concatenation, Kleen star only.
 - a. context free
 - b. regular
 - c. reversible
 - d. irregular
4. Every regular language is _____.
 - a. context-free
 - b. reversible
 - c. irrational
 - d. rational
5. Which of the following statement is true?
 - a. Not all languages are error-free.
 - b. Not all languages are context-free.
 - c. All languages are context-free.
 - d. Not all languages are non context-free.
6. Which of the following statement is true?
 - a. Context-free languages have applications in medical science.
 - b. Context-free languages do not have applications in compiler design (parsers).
 - c. Context-free languages have applications in compiler design (parsers).
 - d. Non context-free languages have applications in compiler design (parsers).
7. Which of the following statement is true?
 - a. A grammar G is said to be in the Chomsky normal form (or CNF) if all productions are of the forms $A \rightarrow BC$.
 - b. A grammar G is said to be in the Chomsky normal form (or CNF) if all productions are of the forms $A * BC$.
 - c. A grammar G is said to be in the Chomsky normal form (or CNF) if all productions are of the forms $A < BC$.
 - d. A grammar G is said to be in the Chomsky normal form (or CNF) if all productions are of the forms $A > BC$.

8. Which of the following statement is false?
- a. Not every regular language is context-free.
 - b. Every regular language is a context-free language, but also some non-regular languages are included.
 - c. Not all languages are context-free.
 - d. Context-free languages have applications in compiler design (parsers).
9. A grammar is in Greibach normal form if all productions are of which of the following form?
- a. $A \rightarrow aB_1B_2 \dots B_n$
 - b. $A \rightarrow aB_1B_2 \dots B_n$
 - c. $A \rightarrow aB_1B_2 \dots B_n$
 - d. $A \rightarrow aB_1B_2 \dots B_n$.
10. Which of the following properties can be used to prove that a language is not context-free?
- a. Closure properties
 - b. Finite properties
 - c. Intrinsic properties
 - d. Extrinsic properties

Chapter V

Push Down Automata

Aim

The aim of this chapter is to:

- explicate the concept pushdown automata
- explain the equivalence of Context free grammars and Pushdown Automata
- elucidate Deterministic Context Free Language and Deterministic Pushdown Automata

Objectives

The objectives of this chapter are to:

- examine the formal definition of pushdown automata
- elucidate the languages of Pushdown Automata
- identify the equivalence of Context Free Language and Pushdown Automata

Learning outcome

At the end of this chapter, you will be able to:

- define pushdown automata
- determine PDA transition function
- understand graphical format of PDA

5.1 Introduction

We will now consider a new notion of Pushdown Automata (PDA). PDAs are finite automata with a stack, i.e., a data structure which can be used to store an arbitrary number of symbols (hence PDAs have an infinite set of states) but which can be only accessed in a last-in first-out (LIFO) fashion. The languages which can be recognised by PDA are precisely the context free languages.

5.2 Pushdown Automata

A pushdown automaton (PDA) is a non-deterministic finite state automaton that has access to an infinite memory, organised as a stack. We'll see that the family of languages recognised by PDA is exactly the family of context-free languages. A PDA consists of the following:

Stack: The stack is a string of symbols. The PDA has access only to the leftmost symbol of the stack. This is called the top of the stack. During one move of the PDA, the leftmost symbol may be removed ('popped' from the stack) and new symbols may be added (pushed) on top of the stack.

Input tape: Similar to finite automata, the input string is normally scanned one symbol at a time. But also ϵ -moves are possible.

Finite state control unit: The control unit is a non-deterministic finite state machine. Transitions may depend on the next input symbol and the topmost stack symbol.

There are two types of moves: normal moves, and ϵ -moves.

Normal moves: Depending on

- the current state of the control unit
- the next input letter and
- the topmost symbol on the stack

The PDA may

- change the state
- pop the topmost element from the stack
- push new symbols to the stack and
- move to the next input symbol

Spontaneous ϵ -moves don't have (b) and (D), i.e. they are done without using the input tape.

5.2.1 Formal Definition of a PDA

Formally, a pushdown automaton M consists of

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Where,

- Q is a finite state set
- Σ is the finite input alphabet
- Γ is the finite stack alphabet
- $q_0 \in Q$ is the initial state
- $Z_0 \in \Gamma$ is the start symbol of the stack
- $F \subseteq Q$ is the set of final accepting states
- δ is the transition function from.

5.2.2 PDA Transition Function

δ = transition function, which takes the triple: $\delta(q, a, Z)$

Where

- q = state in Q
- a = input symbol in Σ
- Z = stack symbol in Γ

The output of δ is the finite set of pairs (p, γ) where p is a new state and γ is a new string of stack symbols that replaces X at the top of the stack.

- If $\gamma = \epsilon$ then we pop the stack
- If $\gamma = X$ the stack is unchanged
- If $\gamma = YZ$ then X is replaced by Z and Y is pushed on the stack.
- If $X = \epsilon$ then we push on γ .

Example 1

As a first example, let us consider the following PDA that recognises the language.

$L = \{a_n b_n \mid n \geq 1\}$. The PDA has two states, S_a and S_b , and the stack alphabet contains two symbols, A and Z_0 . In the beginning, the machine is in the initial state S_a , and the stack contains only one symbol Z_0 , called the start symbol of the stack. Possible moves are summarised in the following table,

State	Top of stack	InputSymbol		ϵ
		a	b	
S_a	Z_0	Add one A to the stack, stay in state S_a	—	—
S_a	A	Add one A to the stack, stay in state S_a	Remove one A from the stack, go to state S_b	—
S_b	A	—	Remove one A from the stack, stay in state S_b	—
S_b	Z_0	—	—	Remove Z_0 from the stack, stay in state S_b

An input word is accepted, if the PDA can reach the empty stack after reading all input symbols.

Example 2

Here is a formal description of the PDA that recognises L

$$L = \{0^n 1^n \mid n \geq 0\}.$$

$$Q = \{q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$\Gamma = \{0, Z_0\}$$

$$F = \{q_1, q_4\}$$

And δ is described by the table below,

Input	0			1			ϵ		
Top Stack	0	Z_0	ϵ	0	Z_0	ϵ	0	Z_0	ϵ
q_1									q_1, Z_0
q_2			$q_2, 0$	q_3, ϵ					
q_3				q_3, ϵ				q_4, ϵ	
q_4									

An instantaneous description (ID) records the configuration of a PDA at any given time. It is a triplet (q, w, γ) where,

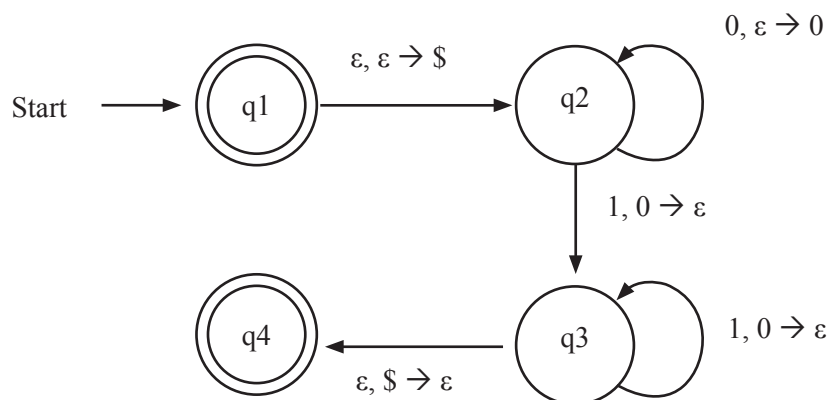
- q is the state of the PDA,
- w is the remaining input, i.e. the suffix of the original input that has not been used yet,
- γ is the content of the stack.

5.2.3 Graphical Format of PDA

Pushdown automata uses the following format:

Input-Symbol, Top-of-Stack $\rightarrow$ String-to-replace-top-of-stack

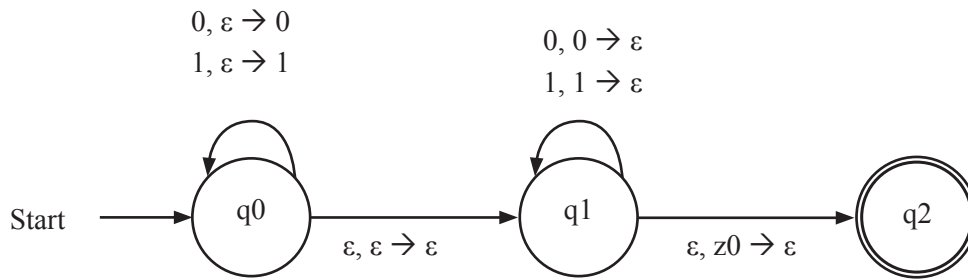
Any of these may be empty.



Example 4

Here is the graphical description of the PDA that accepts the language

- $L = \{ww^R \mid w \text{ is in } (0+1)^*\}$
- Stays in state q_0 when we are reading w and saving the input symbol. Every time we guess that we have reached the end of ' w ' and are beginning ' w^R ' by going to ' q_1 ' on an epsilon-transition.
- In state q_1 , we pop off each 0 or 1 we encounter that matches the input. Any other input will 'die' for this branch of the PDA. If we ever reach the bottom of the stack, we go to an accepting state.



Requires Z_0 initially on the stack.

5.2.4 Moves of a PDA

- To describe the process of taking a transition, we can adopt a notation similar to δ like we used for DFA's. In this case we use the "turnstile" symbol $\vdash$ which is used as:
- $(q, aw, X\beta) \vdash (p, w, \alpha\beta)$
- In other words, we took a transition such that we went from state q to p , we consumed input symbol a , and we replaced the top of the stack X with some new string α
- We can extend the move symbol to taking many moves:
 $\vdash^*$ represents zero or more moves of the PDA

5.3 Language of PDA

There are two ways to define the language of a PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0; F)$ ($L(M) \subseteq \Sigma^*$) because there are two notions of acceptance.

5.3.1 Acceptance by Final State

$$L(M) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \gamma) \wedge q \in F\}$$

That is the PDA accepts the word 'w' if there are any sequence of ID's starting from (q_0, w, Z_0) and leading to (q, ϵ, γ) , where $q \in F$ is one of the final states. Here it doesn't play a role what the contents of the stack are at the end.

In this example, the PDA P_0 would accept 0110 because $(q_0, 0110, \#) \vdash^* P_0 (q_2, \epsilon, \epsilon)$ and $q_2 \in F$. Hence we conclude $0110 \in L(P_0)$.

On the other hand since there is no successful sequence of IDs starting with $(q_0, 0011, \#)$ we know that $0011 \notin L(P_0)$.

5.3.2 Acceptance by Empty Stack

$$L(M) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \epsilon, \epsilon)\}$$

That is the PDA accepts the word w if there is any sequence of IDs starting from (q_0, w, Z_0) and leading to (q, ϵ, ϵ) , in this case, the final state plays no role.

If we specify a PDA for acceptance by empty stack, we will leave out the set of final states F and just use $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0)$.

Our example automaton P_0 also works, if we leave out F and use acceptance by empty stack.

We can always turn a PDA which uses one acceptance method into one which uses the other. Hence, both acceptance criteria specify the same class of languages.

5.4 Equivalence of CGF and PDA

A context-free grammar and pushdown automata are equivalent in power.

For every CFG there is an equivalent PDA,

It has been said many times that PDA's are an abstract machine just powerful enough to recognise the languages generated by CFG's. In order to prove this one must show that for any CFG G there exists a PDA M

$$L(M) = L(G).$$

Furthermore one must also show that for any PDA M there exists a CFG G

$$L(G) = L(M).$$

It is interesting to note that PDA's were formally introduced by Oettinger in 1961 and CFG's by Chomsky in 1956. The equivalence between these two was first perceived by Chomsky in 1962 and Evey in 1963. In both directions you will see a constructive proof.

Theorem:

If $G = (V, \Sigma, S, P)$ is a CFG, there exists a PDA $M = (Q, \Sigma, \Gamma, q_0, Z_0, \delta, A)$ such that $L_f(M) = L(G)$.

Proof:

We define our PDA M as follows:

$Q = \{q_0, q_1, q_2\}$ - It is interesting to note that any CFG can be recognised (via final state) by a PDA with only three states.

$\Gamma = V \cup \Sigma \cup \{Z_0\}$, $Z_0 \notin V \cup \Sigma$ - Our stack symbols are the grammar variables, a stack start symbol and the input alphabet.

$A = \{q_2\}$ - Only need one accepting state for any CFG.

We must now define our PDA's transition function δ

- $\delta(q_0, \epsilon, Z_0) = \{(q_1, SZ_0)\}$
- For $A \in V$, $\delta(q_1, \epsilon, A) = \{(q_1, \alpha) | (A \rightarrow \alpha) \in P\}$
- For each $a \in \Sigma$, $\delta(q_1, a, a) = \{(q_1, \epsilon)\}$
- $\delta(q_1, \epsilon, Z_0) = \{(q_2, Z_0)\}$

The proof can be completed by showing $L(G) \subseteq L_f(M)$ and $L_f(M) \subseteq L(G)$.

The basic idea of the construction is to use the start state and a required Q -transition first move to transit to state q_1 while placing the grammar's start symbol on the stack. Whenever a grammar variable is on the top of the stack we can pop it off replacing it with its right-hand side string.

Whenever the input symbol matches the stack's top (grammar terminals) we write the input and pop the symbol off the stack. Finally when input is exhausted and Z_0 is all that is left on the stack (nothing was ever placed under it), the PDA transits to the accepting state.

Hopefully, one sees that the constructed PDA will correctly guess the steps necessary for a leftmost string derivation for $x \in L(G)$. Simply put if at some point in the derivation the current string looks like $x\alpha$, where $x \in \Sigma^*$ (a string of terminals), then at some point in the PDA's simulation of the leftmost derivation, the input string read so far is x and that stack contains α .

It should be clear that this PDA is not a DPDA. Multiple productions for any given variable ' A ' introduces non-determinism. Any nondeterminism inherent in the grammar is replicated in the PDA.

For every PDA there is an equivalent CFG

A word (or two) is in order before looking at how to construct a CFG out of any PDA M . Assume that the given PDAM accepts language L via empty stack $L = L_e(M)$. (Given a PDA M' that accepts L via final state one can convert M' into a PDA M such, that $L = L_e(M) = L_f(M')$).

We already know what the terminal symbols are, but which are to be used for the grammar's variables? It should be obvious that one cannot use the PDA's states; there are probably not enough of them for our purposes. Furthermore, one cannot use either the set Γ of stack symbols (not enough information encoded) or all the possible stack configurations (infinitely many). Instead our variables will be things of the form $[p, A, q]$ where $p, q \in Q; A \in \Gamma$. This will certainly provide more variables than needed.

Whenever the PDA can move from state p to state q , read $a \in \Sigma \cup \{\epsilon\}$ from the input stream and pop $A, A \in \Gamma$, off the stack (a true pop operation, not a replace) we introduce into the grammar the production $[p, A, q] \rightarrow a$. Hence, our grammar's variables incorporate both, state information and stack top symbol information.

Theorem

If $M = (Q, \Sigma, \Gamma, q_0, Z_0, \delta, A)$ is a PDA there exists a CFG $G = (V, \Sigma, S, P)$ such that $L_e(M) = L(G)$

Proof:

The CFG G is defined as follows:

$V = \{S\} \cup \{[p, A, q] \mid A \in \Gamma; p, q \in Q\}$ – more than is needed but it is harmless to include useless variables.

The productions of the grammar are:

- For every $q \in Q; (S \rightarrow [q_0, Z_0, q]) \in P$
- If $q_1, q_2 \in Q; a \in \Sigma \cup \{\epsilon\}; A \in \Gamma$; and $\delta(q_1, a, A)$ contains (q_2, ϵ) then $([q_1, A, q_2] \rightarrow a) \in P$
- If $m \geq 1; q_0, q_1 \in Q; a \in \Sigma \cup \{\epsilon\}; A \in \Gamma; \delta(q_0, a, A)$ contains $(q_1, B_1 B_2, \dots, B_m)$ for some string $B_1, B_2, \dots, B_m \in \Gamma$ then for every possible choice (i.e. assignment) of $q_a, \dots, q_m \in Q$, the production $[q_0, A, q_m] \rightarrow a [q_1, B_1, q_a][q_a, B_2, q_b] \dots [q_l, B_m, q_m]$

is included in P .

The proof is completed by showing $L(G) \subseteq L_e(M)$ and $L_e(M) \subseteq L(G)$.

5.5 Equivalence of CFL and PDA

Pushdown Automata and CFLs are equivalent

Theorem: Each context-free language is recognised by a PDA

Proof:

- Take any context-free language L .
- Select any CNF (Chomsky Normal Form) grammar $G = (V, \Sigma, R, S)$ for the language.
- Permissible productions are of the form:
- $S \rightarrow \epsilon$
- $A \rightarrow BC$, where B and C are in V .
- $A \rightarrow a$, where $a \in \Sigma$
- Construct a PDA N that checks whether the input is generated by the grammar using leftmost production.
- The input alphabet and the stack alphabet of N are Σ and V , respectively.
- N has, among others, three states q_0, q_1, q_2 , and q_3 , where q_0 is the initial state and q_3 is a unique accept state.

Transitions:

- q_3 has no possible moves.
- If $S \rightarrow \epsilon$ is a permissible rule, then $(q_3, \epsilon) \in \delta(q_0, \epsilon, \epsilon)$.
- $(q_1, \$) \in \delta(q_0, \epsilon, \epsilon)$.
- $(q_2, S) \in \delta(q_1, \epsilon, \epsilon)$.
- $(q_3, \epsilon) \in \delta(q_2, \epsilon, \$)$.

q_0 and q_1 are used to turn the stack contents to $S\$$

- For each rule $r: A \rightarrow BC$, there are transitions:
 $(q_r, C) \in \delta(q_2, \epsilon, A)$ and
 $(q_2, B) \in \delta(q_r, \epsilon, \epsilon)$.

Here, q_r is a special state associated with r .

These transitions have the role of replacing an A at the top of stack to BC .

- For each rule $r: A \rightarrow a$, there is a transition:
 $(q_2, \epsilon) \in \delta(q_r, a, A)$.

This transition matches an input symbol a with a stack symbol A .

Theorem: Every language recognised by PDA is context-free.

Proof:

The proof is constructive. We will develop a CFG from a PDA. We will develop a CFG from a PDA.

Let L be recognised by a PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. We will assume the following:

- (*) M has a unique final state and, when it enters the state, the stack is empty.
- (**) In a single move, M may not both pop and push

Modify M to create an equivalent PDA $N = (Q', \Sigma, \Gamma', \delta', q_0, \{q_f\})$

- $\Gamma' = \Gamma \cup \{\perp\}$.
- Q' is a superset of Q .
- I is a new initial state.
- q_f is a new, unique accept state.
- More additional states are in Q' :
- a clean-up state C and
- a number of new states

N simulates M after adding a new special symbol $\perp$ to the stack.

If M enters an accept state, N may choose to empty the stack until it encounters $\perp$, when N may accept.

The role of q_0 , $\perp$, and C

- There is only one move in state q_0 : $\{(p_0, \perp)\} = \delta'(q_0, \epsilon, \epsilon)$.
The meaning is: place a $\perp$ on stack and then proceed to the initial state of M .
- In each state $p \in F$, $(C, \epsilon) \in \delta'(p, \epsilon, \epsilon)$.
The meaning is: from any accept state of F , you may proceed to C .
- For each $a \in \Gamma$, $\{(C, \epsilon)\} = \delta'(C, \epsilon, a)$, and $\{(q_f, \epsilon)\} = \delta'(C, \epsilon, \perp)$.
The meaning is: in C empty the stack before entering q_f .

Executing M's Moves

- For each $(q, c) \in \delta(p, a, b)$ such that $a \in \Sigma \epsilon$ and $b, c \in \Gamma \epsilon$, add (q, c) to $\delta'(p, a, b)$ if at least one of b and c is ϵ .
- For each $(q, c) \in \delta(p, a, b)$ such that $a \in \Sigma \epsilon$ and $b, c \in \Gamma$, add a new unique state r to Q' , (r, ϵ) to $\delta'(q, a, b)$, and (q, c) to $\delta'(r, \epsilon, \epsilon)$.

Now N satisfies the two conditions and accepts $L(M)$.

5.6 Introduction to DCFL and DPDA

A deterministic PDA (DPDA) is a PDA with only 1 choice in any situation

Definition:

A PDA $M = (Q; \Sigma; \Gamma; \delta; q_0; z_0; F)$ is called deterministic if it is an automaton as defined in, subject to the restrictions

that, $\forall q \in Q; a \in \Sigma \cup \{\lambda\}$, and $b \in \Gamma$,

$\delta(q; a; b)$ has at most 1 element,

if $\delta(q; \lambda; b) \neq \emptyset$ then $\delta(q; c; b) = \emptyset; \forall c \in \Sigma$.

The last restriction insists, for determinism, that, if a λ move is possible for a given stack top, then no input-consuming alternative is possible for that stack top.

Definition: A language L is a deterministic CFL (DCFL) when there is a DPDA M such that $L = L(M)$.

- The language $L_1 = \{a^n b^n : n \geq 0\}$ is a DCFL.
- The language $L_2 = \{a^n b^{2n} : n \geq 0\}$ is a DCFL.
- Is the language $L = L_1 \cup L_2$ a DCFL?

It appears not to be: How would a DPDA know, after reading all the a symbols, whether to pop 1 b symbol or 2 b symbols for each a ?

We develop this into a sketch of a proof that L is not a DCFL.

Claim: If L is a DCFL, then

$\tilde{L} = L \cup \{a^n b^n c^n : n \geq 0\}$

is a CFL.

We can sketch a construction of $\tilde{M}$ accepting $\tilde{L}$ from DPDA M accepting L .

Summary

- PDA's are finite automata with a stack, i.e. a data structure which can be used to store an arbitrary number of symbols.
- A pushdown automaton (PDA) is a non-deterministic finite state automaton that has access to an infinite memory, organised as a stack.
- The PDA has access only to the leftmost symbol of the stack.
- The control unit is a non-deterministic finite state machine.
- A context-free grammar and pushdown automata are equivalent in power.
- Pushdown Automata and CFLs are Equivalent.
- Each context-free language is recognised by a PDA.
- Every language recognised by PDA is context-free.
- A language L is a deterministic CFL (DCFL) when there is a DPDA M such that $L = L(M)$.

References

- Padma Reddy, A.M., 2011. *Finite Automata and Formal Languages: A Simple Approach*, Sanguine Technical Publishers.
- Simon, M., 1999. *Automata Theory*, World Scientific Publishing Co. Pte, Ltd.
- *Pushdown Automata* [Pdf] Available at: <<http://www.eecs.wsu.edu/~ananth/CptS317/Lectures/PDA.pdf>> [Accessed 7 June 2013].
- *Pushdown Automata* [Pdf] Available at: <<http://www.math.uaa.alaska.edu/~afkjm/cs351/handouts/pda.pdf>> [Accessed 7 June 2013].
- 2011. *Lec-20 Pushdown Automata* [Video online] Available at: <<https://www.youtube.com/watch?v=8O6FUFwWt5A>> [Accessed 7 June 2013].
- Kumar, R, 2011. *Lecture on Push Down Automata* [Video online] Available at: <<https://www.youtube.com/watch?v=7GZi7tk-mvA>> [Accessed 7 June 2013].

Recommended Reading

- Meduna, A., 2000. *Automata and Languages: Theory and Applications*, 1st ed., Springer.
- Revesz, G. E., 1991. *Introduction to Formal Languages*, Dover Publications.
- Singh, A., 2009. *Elements of Computation Theory*, 1st ed., Springer.

Self Assessment

1. The control unit is a _____ finite state machine.
 - a. deterministic
 - b. Moore
 - c. non-deterministic
 - d. Mealy

2. _____ may depend on the next input symbol and the topmost stack symbol.
 - a. Conversion
 - b. Alteration
 - c. Variations
 - d. Transitions

3. A context-free grammar and pushdown automata are equivalent in _____.
 - a. power
 - b. size
 - c. length
 - d. dimension

4. Pushdown Automata and CFLs are _____.
 - a. equivalent
 - b. complimentary
 - c. incomparable
 - d. differential

5. Each context-free language is recognised by _____.
 - a. PDA
 - b. PCP
 - c. CFG
 - d. TM

6. Which of the following statement is true?
 - a. Every language recognised by PCP is context-free.
 - b. Every language recognised by PDA is context-free.
 - c. Every language recognised by PDA is non-context-free.
 - d. Not every language recognised by PDA is context-free.

7. Which of the following statement is true?
 - a. A language L is a deterministic CFL (DCFL) when there is a DPDA M such that $L \geq L(M)$.
 - b. A language L is a deterministic CFL (DCFL) when there is a DPDA M such that $L \neq L(M)$.
 - c. A language L is a deterministic CFL (DCFL) when there is a DPDA M such that $L = L(M)$.
 - d. A language L is a deterministic CFL (DCFL) when there is a DPDA M such that $L \cap L(M)$.

8. Which of the following statement is true?
- a. Each non context-free language is recognised by a PDA.
 - b. Each context-free language is recognised by a PDA.
 - c. Each context-free language is recognised by a FA.
 - d. Each context-free language is recognised by a NFA.
9. Which of the following is a non-deterministic finite state automaton that has access to an infinite memory, organised as a stack?
- a. A pushdown automaton
 - b. Finite automaton
 - c. Non deterministic automaton
 - d. Deterministic automaton
10. The PDA has access only to the leftmost symbol of _____.
- a. Counter
 - b. Register
 - c. Queue
 - d. Stack

Chapter VI

Turing Machines

Aim

The aim of this chapter is to:

- explore the concept of Turing machine
- enlist the variants of the Turing machine
- analyse the Church-Turing Thesis.

Objectives

The objectives of this chapter are to:

- examine informal definition of Turing machine
- elucidate non-deterministic Turing machine
- identify the Church-Turing Thesis

Learning outcome

At the end of this chapter, you will be able to:

- understand the universal turing machine
- enlist the variants of Turing machine
- evaluate the universal Turing machine

6.1 Introduction

Turing machine (TM) is the next more powerful model of automata approach which recognises more languages than Pushdown automata models do. Also Phrase- structure model is the corresponding grammatical model that matches Turing machines in computational power.

TM is an abstract entity constituted of mathematical objects like sets and a partial function. However in order to help our understanding of the subject-matter of TMs, we can visualise a TM as a physical computing device that can be represented in the following diagram:

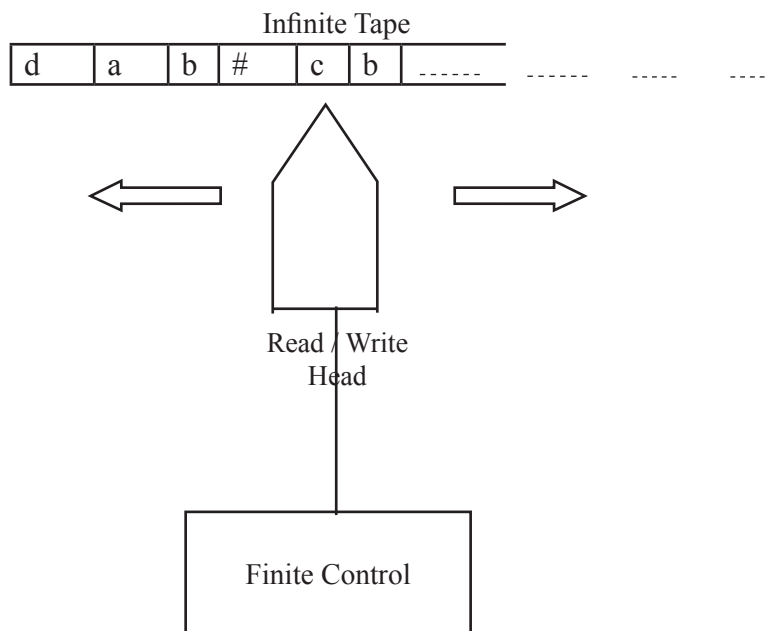


Fig. 6.1 Turing machine

According to the above figure, TM consists of:

- A tape, with an end on the left but infinite on the right side. The tape is divided into squares or cells, with each cell capable of holding one of the tape symbols including the blank symbol #.
- A finite control, which can be in any one of the finite number of states.

The states in TM can be divided in three categories, viz.,

- The Initial state, the state of the control just at the time when TM starts its operations. The initial state of a TM is generally denoted by q_0 or s .
 - The Halt state, in which TM stops all further operations. The halt state is generally denoted by h . The halt state is distinct from the initial state. Thus, a TM has at least two states.
 - Other states
- A tape head (or simply Head) is always stationed at one of the tape cells and provides communication for interaction between the tape and the finite control. The Head can read or scan the symbol in the cell under it. The symbol is communicated to the finite control. The control taking into consideration the symbol and its current state decides for further course of action including
 - The change of the symbol in the cell being scanned and/or change of its state and/or
 - Moving the head to the Left or to the Right. The control may decide not to move the head.

6.2 Formal Definition of Turing Machine

Similar to a finite automaton but with an unlimited and unrestricted memory, a Turing machine is a much more accurate model of a general purpose computer. A Turing machine can do everything that a real computer can do. Nonetheless, even a Turing machine cannot solve certain problems. In a very real sense, these problems are beyond the theoretical limits of computation.

The following list summarises the differences between finite automata and Turing machines:

- A Turing machine can both write on the tape and read from it.
- The read – write head can move both to the left and to the right.
- The tape is infinite.
- The special states for rejecting and accepting take immediate effect.

Definition: Formally a TM $M = (Q, \Sigma, \Gamma, \delta, s)$ where,

- Q = a finite set of states
- Σ = input alphabet not containing the blank symbol B
- Γ = the tape alphabet of M
- s in Q is the start state
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the (partial) transition function.

6.2.1 Deterministic Turing Machines Definition

A Deterministic Turing machine is a 7 – tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

Where Q, Σ, Γ are all finite sets and:

1. Q is the set of states.
2. Σ is the input alphabet not containing the special blank symbol B .
3. Γ is the tape alphabet, where $B \in \Gamma$ and $\Sigma \subseteq \Gamma$
4. $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function.
5. $q_0 \in Q$ is the start state.
6. $q_{\text{accept}} \in Q$ is the accept state.
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.

6.2.2 Non- Deterministic Turing Machines Definition

A Non- Deterministic Turing machine is a 7 – tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$

Where Q, Σ, Γ are all finite sets and:

1. Q is the set of states.
2. Σ is the input alphabet not containing the special blank symbol B .
3. Γ is the tape alphabet, where $B \in \Gamma$ and $\Sigma \subseteq \Gamma$
4. $\delta: Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$ is the transition function.
5. $q_0 \in Q$ is the start state.
6. $q_{\text{accept}} \in Q$ is the accept state.
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.

- For a Turing machine, δ takes the form: $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$. That is, when the machine is in a certain state q_1 and the head is over a tape square containing a symbol a , and if
- $(q_1, a) = (q_2, b, L)$ the machine writes the symbol b replacing a , and goes to state q_2 .
- Initially, M receives its input ' $w = w_1 w_2 \dots w_n \in \Sigma^*$ ' on the leftmost ' n ' squares of the tape, and the rest of the tape is blank.

- As a Turing machine computes, changes occur in the current state, the current tape contents, and the current head location. A setting of these three items is called configuration of the Turing machine.
- A Turing machine 'M' accepts input 'w', if a sequence of configurations $C_1, C_2, \dots, C_k$ exists where.
 - C_1 is the start configuration of M on input w
 - Each C_i yields C_{i+1}
 - C_k is an accepting configuration.
- The collection of strings that M accepts is the language of M, denoted $L(M)$.
- Call a language Turing – recognisable if some Turing machine recognises it.
- When we start a TM on an input, three outcomes are possible. The machine may accept, reject, or loop. By loop, we mean that the machine simply does not halt. It is not necessarily repeating the same steps in the same way forever as the connotation of looping may suggest. Looping may entail any simple or complex behavior that never leads to a halting state.
- We prefer Turing machines that halt on all inputs; such machines never loop. These machines are called deciders because they always make a decision to accept or reject. A decider that recognises some language also is said to decide that language.
- Call a language 'Turing – decidable' or simply 'decidable' if some Turing machine decides it.
- Every decidable language is Turing – recognisable but certain Turing – recognisable languages are not decidable.

Example:

TM for $\{0^n 1^n | n > 0\}$

Description of how the machine works

- Look for 0's.
- If 0 found, change it to x and move right, else, reject.
- Scan (to the right) 0's and y's until you reach 1.
- If 1 found, change it to y and move left, else reject.
- Move left scanning past 0's and y's.
- If x found move right.
- If 0 found, loop back to step 2.
- If 0 not found, scan (to the right) y's and accept.

Head is on the left or start of the string. X and y are just variables to keep track of equality

Head is on the left or start of the string.

State	Symbol	Next state action
q_0	0	(q_1, x, R)
q_0	1	Halt/reject
q_0	X	Halt/reject
q_0	y	(q_3, y, R)

Head is on the left or start of the string.

State	Symbol	Next state action
q_1	0	$(q_1, 0, R)$
q_1	1	(q_2, y, L)
q_1	X	Halt/reject
q_1	y	(q_1, y, R)

Head is on the left or start of the string.

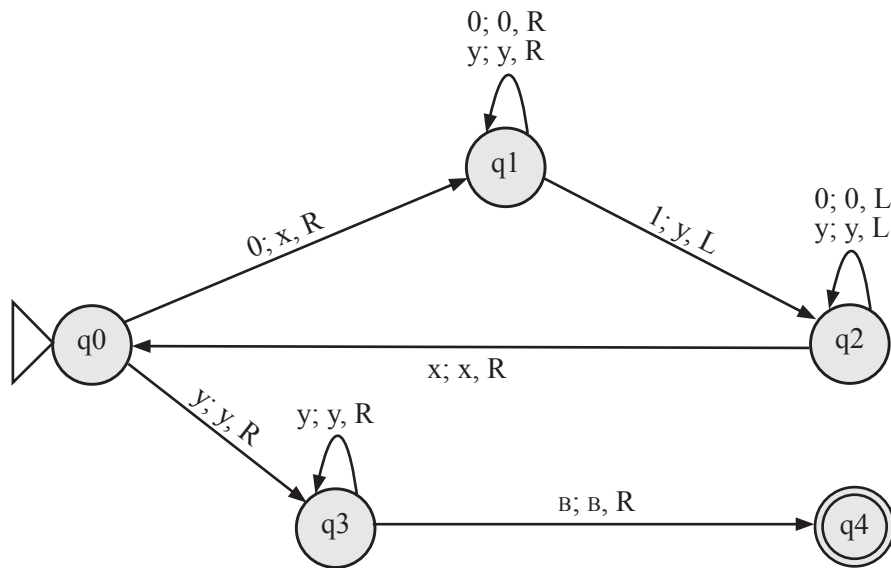
State	Symbol	Next state action
q_2	0	$(q_2, 0, L)$
q_2	1	Halt/reject
q_2	X	(q_0, x, R)
q_2	y	(q_2, y, L)

Head is on the left or start of the string.

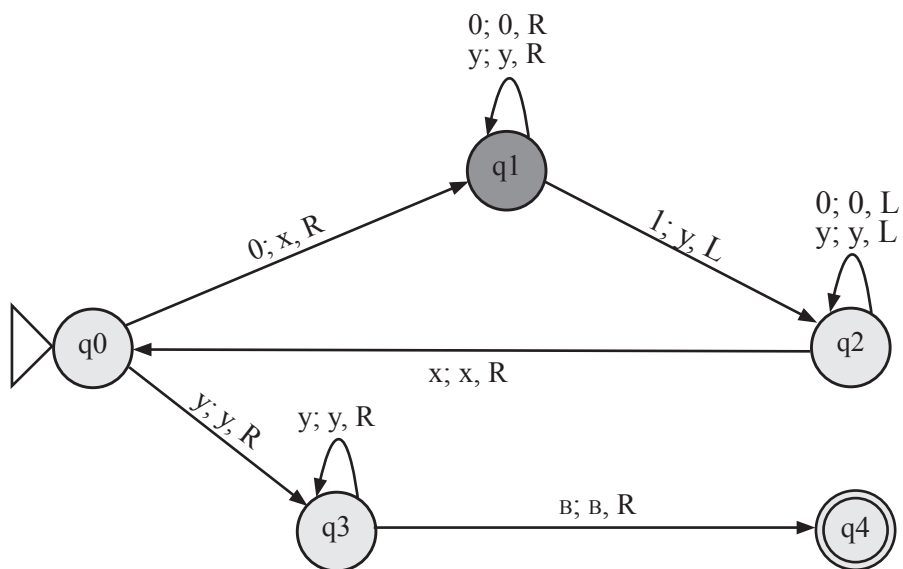
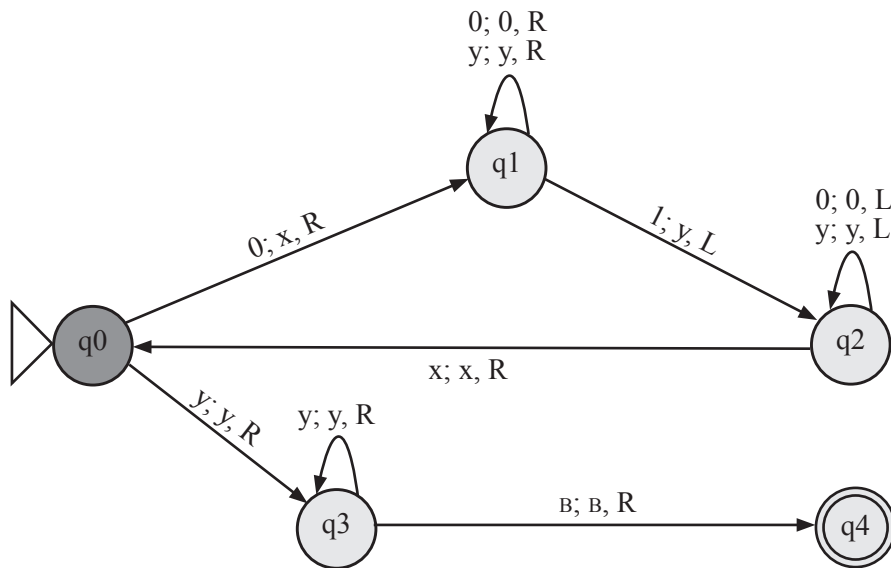
State	Symbol	Next state action
q_3	0	Halt/reject
q_3	1	Halt/reject
q_3	X	Halt/reject
q_3	y	(q_3, y, R)
q_3	B	(q_4, B, R)

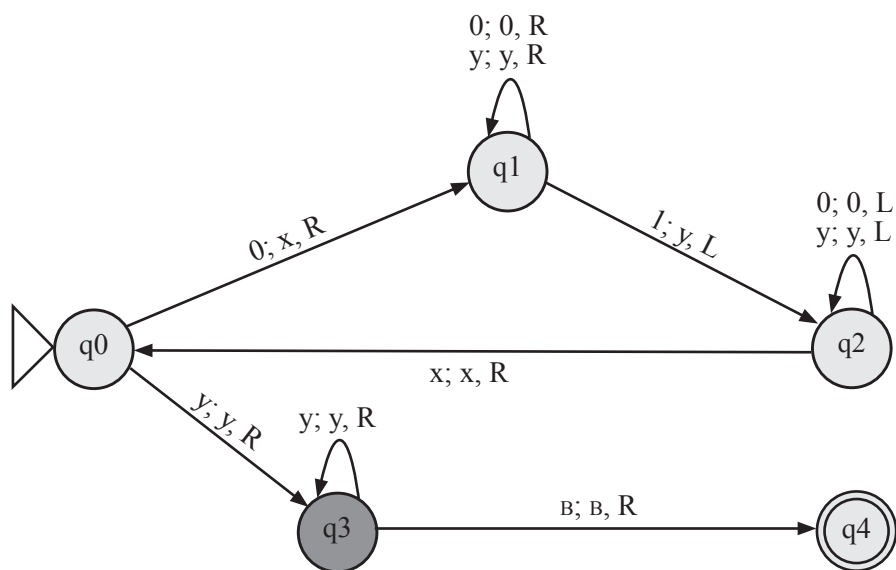
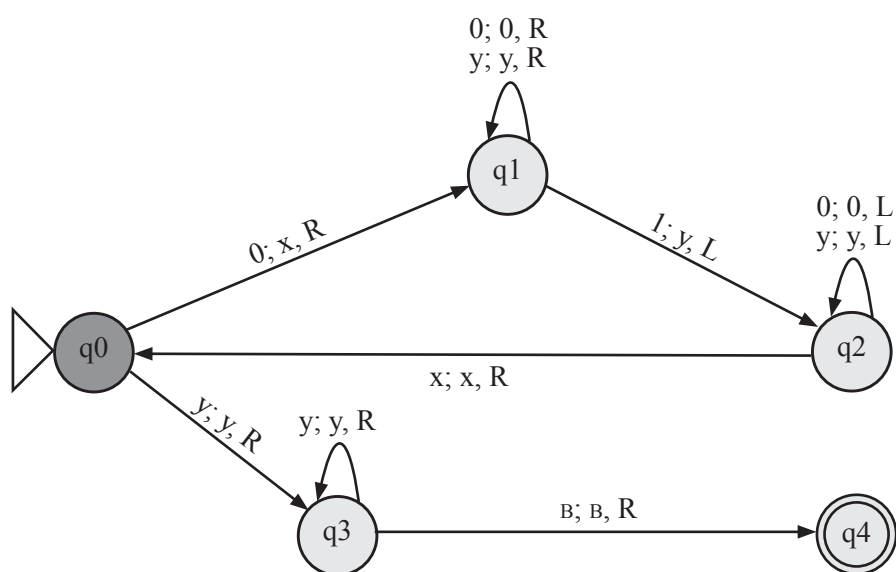
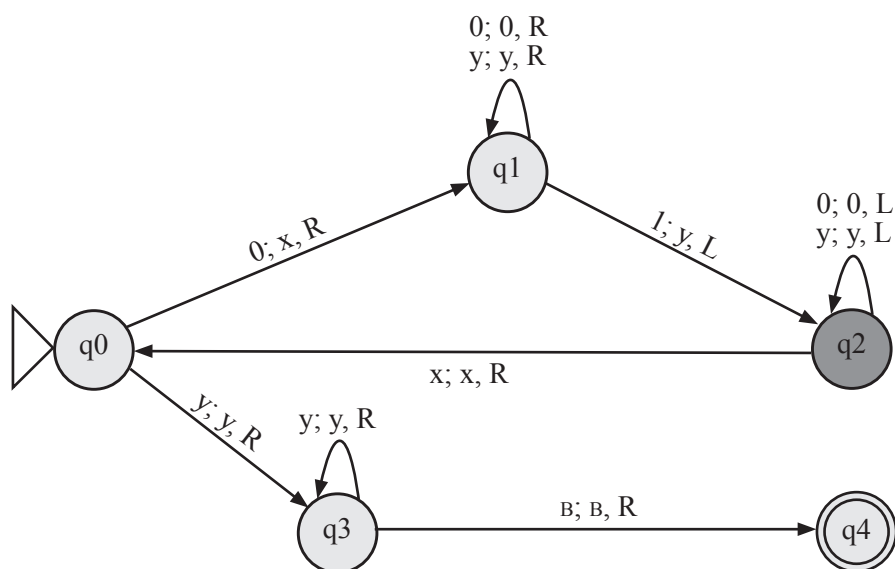
Head is on the left or start of the string.

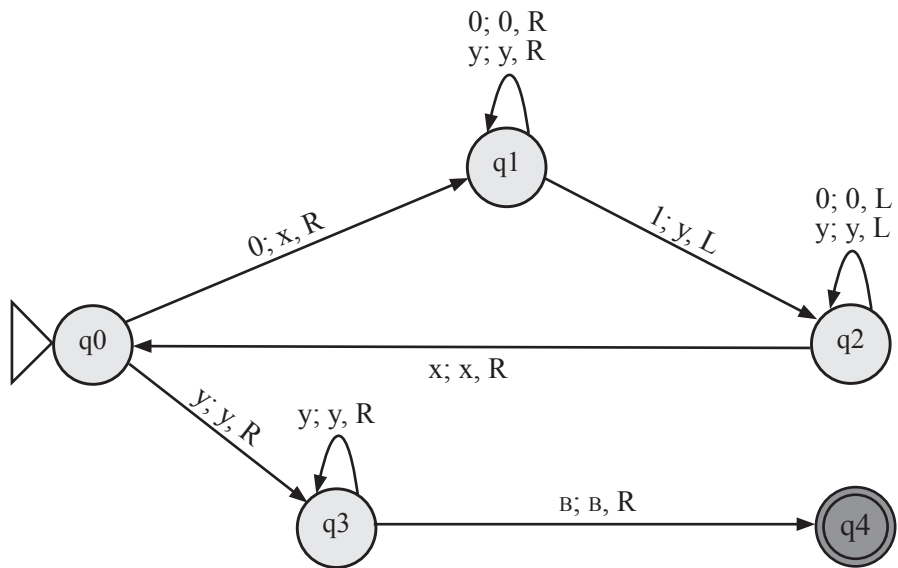
State	Symbol	Next state action
q_4	0	Illegal i/p
q_4	1	Illegal i/p
q_4	X	Illegal i/p
q_4	y	Illegal i/p
q_4	B	Halt/accept



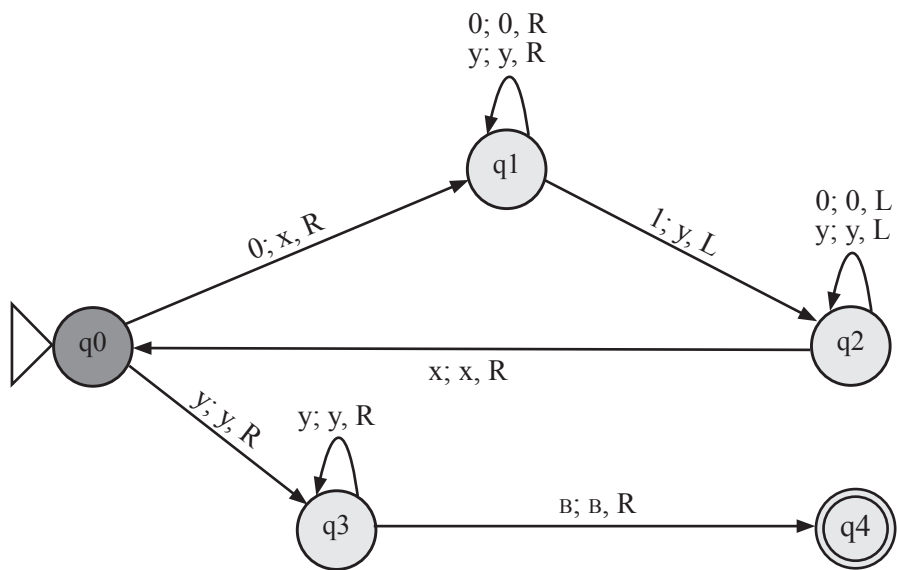
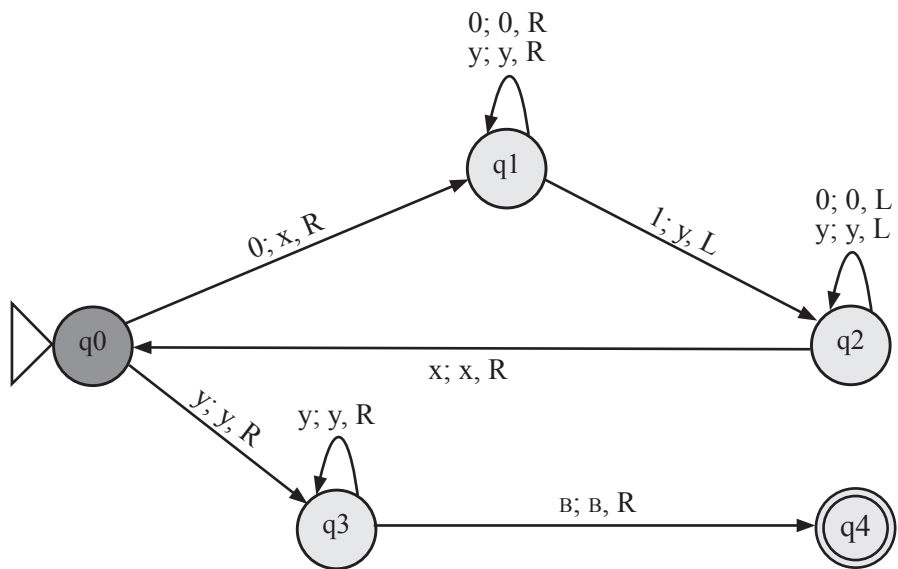
Input 1: 01



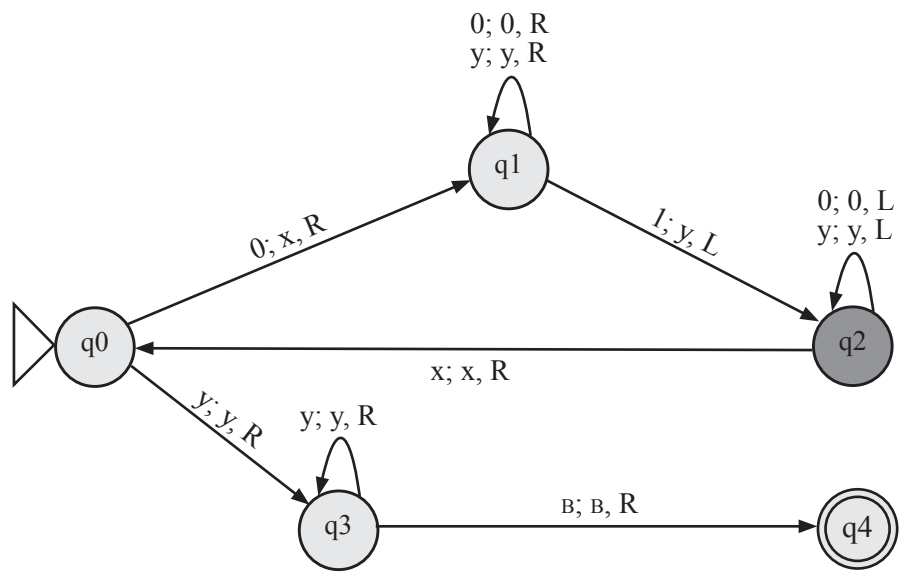
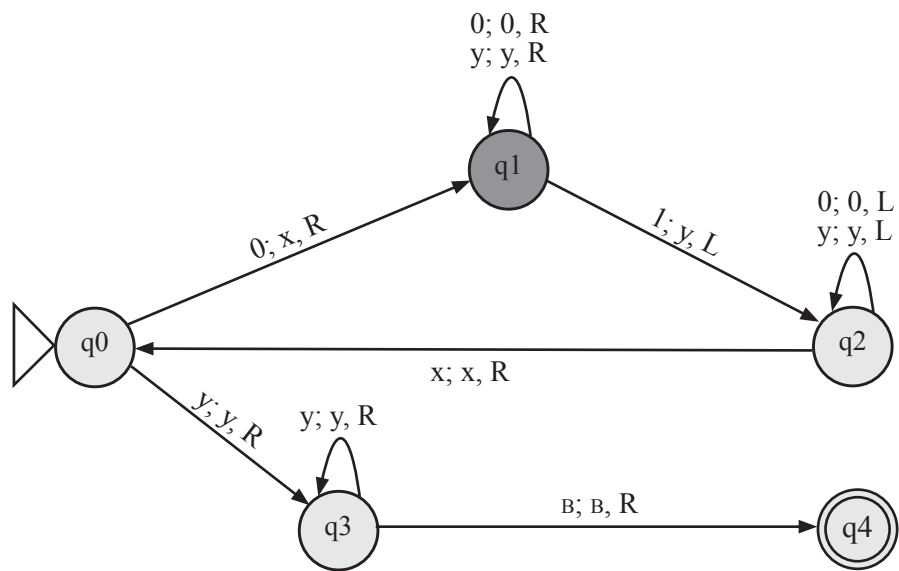
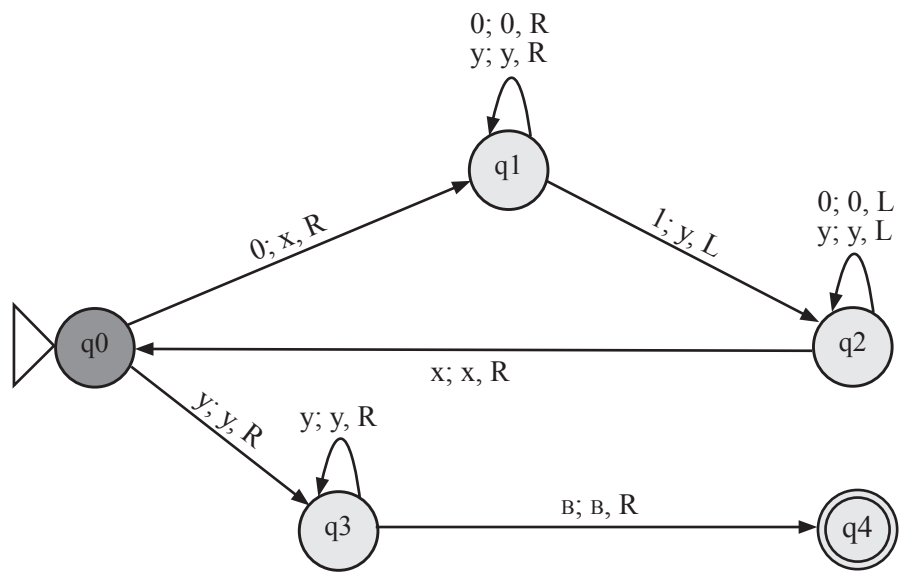


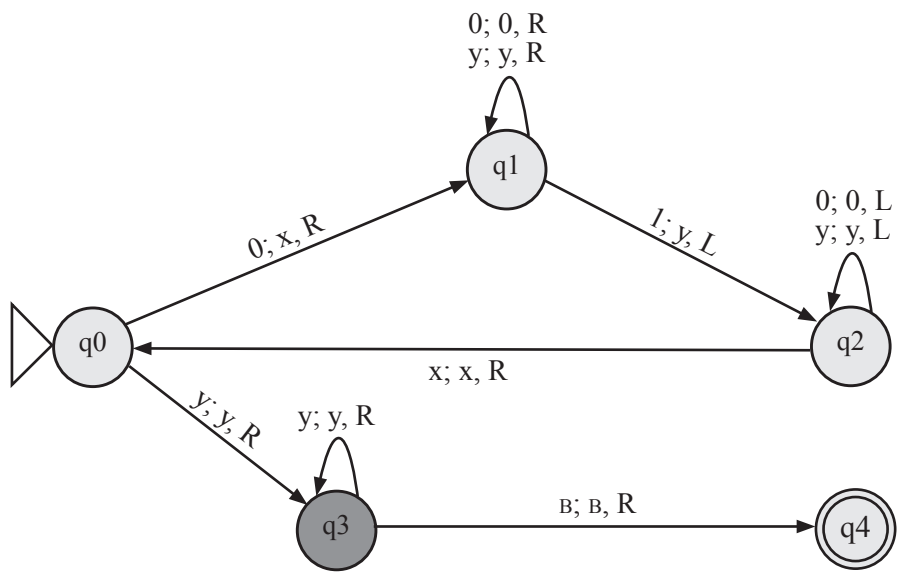
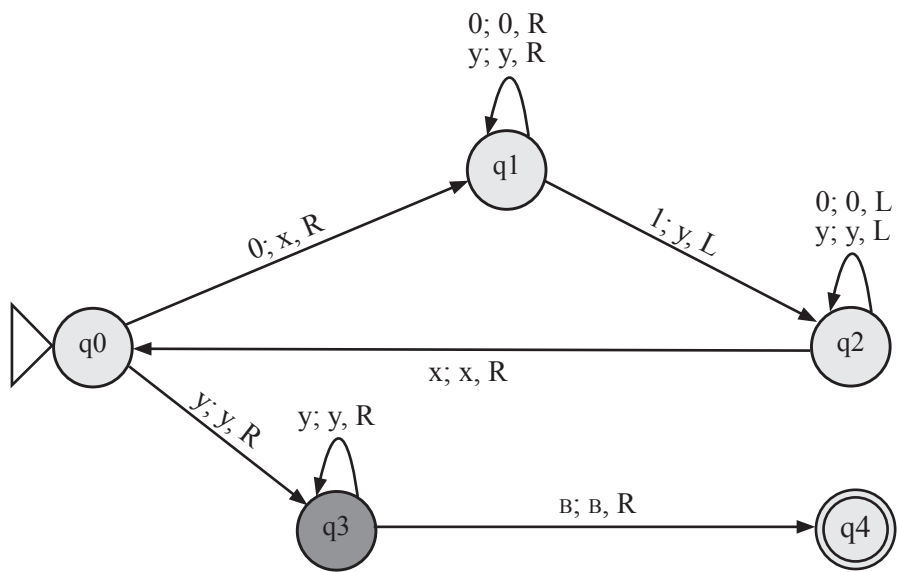
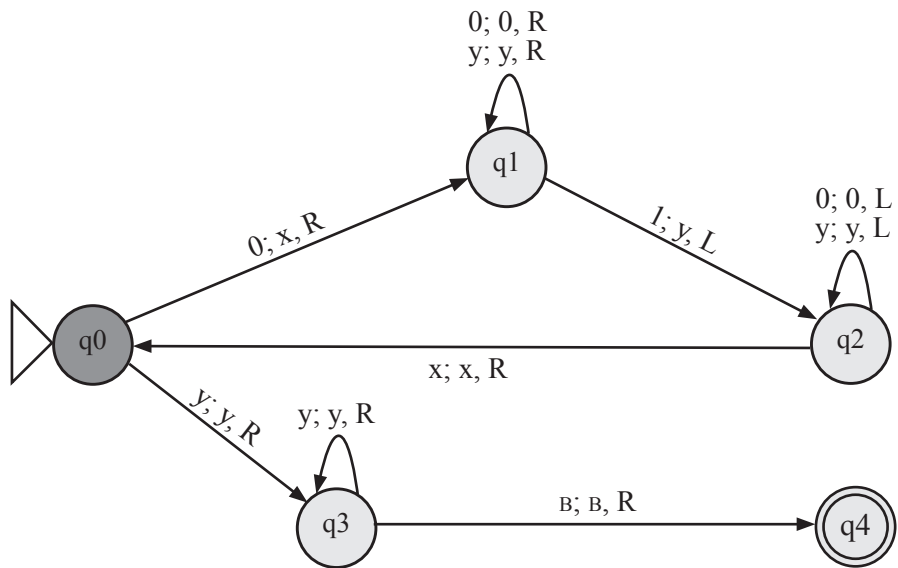


Input 2: 10



Input 3: 011





6.3 Some Configuration on Turing Machine (TM)

- Start configuration q_0w .
- Accepting configuration q_{accept}
- Reject configuration q_{reject}
- Halting configuration: accepting and rejecting configurations.

6.4 Variants of Turing Machines

- The original TM model and its reasonable variants, all have the same power – they recognise the same class of languages.
- To show that two models are equivalent, we simply need to show that we can simulate one by the other.
- Robustness: invariant power to certain changes

Example: $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$

- TM with multi-tapes: A multitape TM is like an ordinary Turing machine with several tapes. Each tape has its own head for reading and writing. Initially, the input appears on tape 1, and the others start out blank. Two machines are equivalent if they recognise the same language. Every multitape Turing machine has an equivalent single tape Turing machine.

Example: 2-tape TM for copying

- Assume there are 2-tapes.
- Move the head on the first tape left until a blank is found.
- Move the head on both tapes to the right, copying each symbol on the first tape onto the second tape, until a blank is found on the first tape. The first square of the second tape should be left blank.
- Move the head on the second tape to the left until a blank is found.
- Again move the heads on both tapes to the right, this time copying symbols from the second tape onto the first tape. Halt when a blank is found on the second tape.
- Nondeterministic TM's: A nondeterministic Turing machine is defined in the expected way. At any point in a computation the machine may proceed according to several possibilities. The transition function for a nondeterministic is

$$\delta: Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$$

The computation of a nondeterministic Turing machine is a tree, whose branches correspond to different possibilities for the machine. (If you want to simulate a nondeterministic TM with a normal TM you have to perform a breadth – first search through the tree, because with depth – first you can lose yourself in an infinite branch of the tree and miss the accept state).

If some branch of the computation leads to the accept state, the machine accepts its input. Every nondeterministic Turing machine has an equivalent deterministic Turing machine. A language is Turing – recognisable if and only if some nondeterministic Turing machine recognises it. We call a nondeterministic Turing machine a decider if all branches halt on all inputs. A language is decidable if and only if some nondeterministic TM decides it.

- Enumerators: Loosely defined, an enumerator is a Turing machine with an attached printer. A language is Turing – recognisable if and only if some enumerator enumerates it.
 - From a Turing machine M , construct an enumerator E .
 - From an enumerator E , construct a Turing machine M .
- TM with multi-strings (and multi-cursors).
- TM with multi-cursors
- TM with multi-tracks
- TM with two-way infinite tape
- TM with multi-dimensional tape

6.5 Equivalence with Other Models

- Turing machine has many variations.
- Many other models of general purpose computation have been proposed.
 - They are the same in power as Turing machine.
- Power of programming languages are the same.
 - Program of any language can be compiled to a program of other language.

Example 1

TM M_2 recognises the language consisting of all strings of 0s whose length is a power of 2. It decides the language $A = \{0^{2^n} \mid n \geq 0\}$.

Solution:

$M_2 =$ 'on input string w '

- Sweep left to right across the tape, crossing off every other 0.
- If in stage 1 the tape contained a single 0, accept.
- If in stage 1 the tape contained more than a single 0 and the number of 0s was odd, reject
- Return the head to the left-hand end of the tape.
- Go to stage 1.

6.6 The Church-Turing Thesis

The thesis states that:

- The Turing Machine is the formal equivalent of the informal concept of algorithm.
- It is not a theorem, since it cannot be proved. (Algorithm is an informal concept).
- Recall, algorithm -- a sequence of precise steps, each of which can be executed in a finite amount of time with finite resources. Precise means no intelligence or judgment required can be executed literally.
- All reasonable formal models so far have turned out to be equivalent to TM.
- Reasonable -- means no infinite resources or power.
- Examples of other formal models:
 - Post Correspondence systems
 - Random Access Machine model
 - Unrestricted grammars
 - Partial Recursive Functions, etc.

6.7 Universal Turing Machine

A Universal Turing Machine is a special TM introduced by Turing, that can simulate any other TM. It served as the inspiration for the notion of general-purpose computing. Turing himself saw a direct parallel between the capability of a computer to accept and execute programs, and his notion of a universal machine. The principle of universality can easily be extended to any other class of algorithmic machines. As long as each machine in that class can be captured by a finite description, prescribing what this machine would do in every possible configuration, a TM can be created to simulate all machines in that class:

Universality Thesis: Any class of effective devices for computing functions can be simulated by a TM.

Analogously to the Turing thesis, the Universality thesis combines with the mathematical worldview to obtain the following corollary:

Universality Corollary: Any computer can be simulated by a TM.

This corollary is the second of the two claims that have been used to corroborate the Turing Thesis myth. Again, the undergraduate textbooks played a key role. In order to present the expressiveness of TMs in a more accessible (and dramatic) fashion, the Universality Corollary was melded with the Turing Thesis Corollary, resulting in the following statement that (incorrectly) summarised the role of TMs:

Summary

- Turing machine (TM) is the next more powerful model of automata approach which recognises more languages than Pushdown automata models do.
- TM is an abstract entity constituted of mathematical objects like sets and a (partial) function.
- A Turing machine can do everything that a real computer can do.
- Even a Turing machine cannot solve certain problems.
- A Turing machine can both write on the tape and read from it.
- The computation of a nondeterministic Turing machine is a tree whose branches correspond to different possibilities for the machine.
- Every nondeterministic Turing machine has an equivalent deterministic Turing machine.
- A language is decidable if and only if some nondeterministic TM decides it.
- Enumerator is a Turing machine with an attached printer.
- A language is Turing – recognisable, if and only if, some enumerator enumerates it.
- Turing machine has many variations.
- A Universal Turing Machine is a special TM introduced by Turing, that can simulate any other TM.

References

- Kumar, R. 2010. *Theory of Automata, Languages and Computation*, Tata Mc Graw Hill Publication Private Limited.
- Meduna, A., 1999. *Automata and Languages: Theory and Applications*, Springer.
- *Finite-State Machines and Pushdown Automata* [Pdf] Available at: <http://cs.brown.edu/~jes/book/pdfs/ModelsOfComputation_Chapter4.pdf> [Accessed 7 June 2013].
- *Turing Machines* [Pdf] Available at: <<http://www.cs.uky.edu/~lewis/texts/theory/computability/tur-mach.pdf>> [Accessed 7 June 2013].
- *Lec-26 Turing Machine* [Video online] Available at: <<https://www.youtube.com/watch?v=IhyEGNn-7Uo>> [Accessed 7 June 2013].
- *Lecture 12-Turing Machines* [Video online] Available at: <<https://www.youtube.com/watch?v=mPec64RUCsk>> [Accessed 7 June 2013].

Recommended Reading

- Herken, R., 1995. *The Universal Turing Machine*, 2nd ed., Springer.
- Carlisle, M. C., 2009. *An Introduction to Languages and Machines*, CreateSpace.
- Agar, J., 1997. *Turing and the Universal Machine: The Making of the Modern Computer* (Revolutions of Science). Totem Books.

Self Assessment

1. A _____ can do everything that a real computer can do.
 - a. Turing machine
 - b. Turning machine
 - c. Trail machine
 - d. Tuner machine
2. TM is an abstract entity constituted of _____ objects like sets and a (partial) function.
 - a. physical
 - b. mathematical
 - c. logical
 - d. geometrical
3. The computation of a _____ Turing machine is a tree whose branches correspond to different possibilities for the machine.
 - a. deterministic
 - b. PCP
 - c. nondeterministic
 - d. finite.
4. Every nondeterministic Turing machine has an equivalent _____ Turing machine.
 - a. finite
 - b. infinite
 - c. church
 - d. deterministic.
5. Enumerator is a Turing machine with an attached _____.
 - a. printer
 - b. scanner
 - c. fax machine
 - d. photo copier
6. Which of the following statement is true?
 - a. The Turing Machine is the informal equivalent of the formal concept of algorithm.
 - b. The Turing Machine is the formal equivalent of the informal concept of algorithm.
 - c. The Turing Machine is the formal equivalent of the informal concept of flowchart.
 - d. The Turing Machine is equivalent to algorithm.
7. Which of the following statement is true?
 - a. A language is Turing – non recognisable if and only if some enumerator enumerates it.
 - b. A language is not Turing – recognisable if some enumerator enumerates it.
 - c. A language is Turing – recognisable if and only if some enumerator enumerates it.
 - d. A language is Turing – recognisable if and only if some printer enumerates it.

8. Which of the following statement is false?
 - a. Turing machine has many variations.
 - b. Even a Turing machine cannot solve certain problems.
 - c. A language is decidable if and only if some nondeterministic TM decides it.
 - d. Turing machine do not have variations.
9. A Universal Turing Machine is a special TM introduced by whom of the following?
 - a. Turing
 - b. Henry
 - c. Church
 - d. Hilbert
10. Which of the following is an abstract entity constituted of mathematical objects like sets and a (partial) function?
 - a. TNM
 - b. TM
 - c. PCP
 - d. TP

Chapter VII

Finite State Machines

Aim

The aim of this chapter is to:

- introduce variations of the basic finite state machine model
- elucidate basic finite state machine model
- explain the transition tables for different machines

Objectives

The objectives of this chapter are to:

- define finite state machine
- describe basic finite state machine model
- explicate extended finite state machine

Learning outcome

At the end of this chapter, you will be able to:

- understand asynchronous coupling
- recognise the conformance testing problem
- distinguish the Petri Net and the FIFO Net

7.1 Introduction

At a low level of abstraction, a protocol is often most easily understood as a state machine. Design criteria can also easily be expressed in terms of desirable or undesirable protocol states and state transitions. In a way, the protocol state symbolises the assumptions that each process in the system makes about the others. It defines what actions a process is allowed to take, which events it expects to happen, and how it will respond to those events.

The formal model of a communicating finite state machine plays an important role in three different areas of protocol design: formal validation, protocol synthesis, and conformance testing. This chapter introduces the main concepts. First the basic finite state machine model is discussed. There are several, equally valid, ways of extending this basic model into a model for communicating finite state machines. We select one of those models and formalise it in a definition of a generalised communicating finite state machine. The model can readily be applied to represent PROMELA specifications and to build an automated validator.

7.2 Informal Description

A finite state machine is usually specified in the form of a transition table, much like the one shown in Table 7.1 below.

Condition		Effect	
Current State	In	Out	Next State
q0	–	1	q2
q1	–	0	q0
q2	0	0	q3
q2	1	0	q1
q3	0	0	q0
q3	1	0	q1

Table 7.1 Mealy

For each control state of the machine the table specifies a set of transition rules. There is one rule per row in the table, and usually more than one rule per state. The example table contains transition rules for control states named q0, q1, q2, and q3. Each transition rule has four parts, each part corresponding to one of the four columns in the table. The first two are conditions that must be satisfied for the transition rule to be executable. They specify:

- The control state in which the machine must be
- A condition on the “environment” of the machine, such as the value of an input signal

The last two columns of the table define the effect of the application of a transition rule. They specify:

- How the “environment” of the machine is changed, e.g., how the value of an output signal changes
- The new state that the machine reaches if the transition rule is applied

In the traditional finite state machine model, the environment of the machine consists of two finite and disjoint sets of signals: input signals and output signals. Each signal has an arbitrary, but finite, range of possible values. The condition that must be satisfied for the transition rule to be executable is then phrased as a condition on the value of each input signal, and the effect of the transition can be a change of the values of the output signals. The machine in Table 7.1 illustrates that model. It has one input signal, named In and one output signal, named Out.

A dash in one of the first two columns is used as a shorthand to indicate a “don’t care” condition (that always evaluates to the Boolean value ‘true’). A transition rule, then, with a dash in the first column applies to all states of the machine, and a transition rule with a dash in the second column applies to all possible values of the input signal. Dashes in the last two columns can be used to indicate that the execution of a transition rule does not change the environment. A dash in the third column means that the output signal does not change, and similarly, a dash in the fourth column means that the control state remains unaffected.

In each particular state of the machine there can be zero or more transition rules that are executable. If no transition rule is executable, the machine is said to be in an end- state. If precisely one transition rule is executable, the machine makes a deterministic move to a new control state. If more than one transition rule is executable a non deterministic choice is made to select a transition rule. A nondeterministic choice in this context means that the selection criterion is undefined. Without further information either option is to be considered equally likely. From here on, we will call machines that can make such choices nondeterministic machines. Table 7.2 illustrates the concept. Two transition rules are defined for control state q1. If the input signal is one, only the first rule is executable. If the input signal is zero, however, both rules will be executable and the machine will move either to state q0 or to state q3.

Current State	In	Out	Next State
q1	-	0	q0
q1	0	0	q3

Table 7.2 Non-Determinism

The behavior of the machine in Table 7.1 is more easily understood when represented graphically in the form of a state transition diagram, as shown in Fig.7.1.

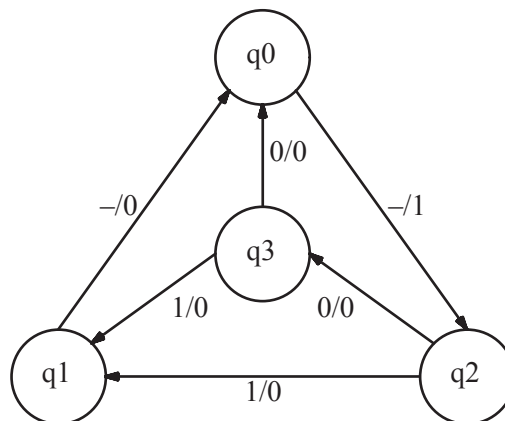


Fig. 7.1 State transition diagram

The control states are represented by circles, and the transition rules are specified as directed edges. The edge labels are of the type c/e , where c specifies the transition condition (e.g., the required set of input values) and e the corresponding effect (e.g., a new assignment to the set of output values).

7.3 Turing Machines

The above definition of a finite state machine is intuitively the simplest. There are many variants of this basic model that differ in the way that the environment of the machines is defined and thus in the definition of the conditions and the effects of the transition rules. For truly finite state systems, of course, the environment must be finite state as well (e.g., it could be defined as another finite state machine). If this requirement is dropped, we obtain the well-known Turing Machine model. It is used extensively in theoretical computer science as the model of choice in, for instance, the study of computational complexity. The Turing machine can be seen as a generalisation of the finite state machine model, although Turing’s work predates that of Mealy and Moore by almost two decades.

The “environment” in the Turing machine model is a tape of infinite length. The tape consists of a sequence of squares, where each square can store one of a finite set of tape symbols. All tape squares are initially blank. The machine can read or write one tape square at a time, and it can move the tape left or right, also by one square at a time. Initially the tape is empty and the machine points to an arbitrary square. The condition of a transition rule now consists of the control state of the finite state machine and the tape symbol that can be read from the square that the machine currently points to. The effect of a transition rule is the potential output of a new tape symbol onto the current square, a possible left or right move, and a jump to a new control state.

The tape is general enough to model a random access memory, be it an inefficient one. Table 7.3 illustrates this type of finite state machine.

Condition		Effect	
Current State	In	Out/Move 1/L	Next State
q0	0	1/L	q1
q0	1	1/R	q2
q1	0	1/R	q0
q1	1	1/L	–
q2	0	1/R	q1
q2	1	1/L	q3
q3	–	–	–

Table 7.3 Busy Beaver

This machine has two output signals: one is used to overwrite the current square on the tape with a new symbol, and other is used to move the tape left or right one square. State q3 is an *end* state.

It is fairly hard to define an extension of this variant of the model with a practical method for modeling the controlled interaction of multiple finite state machines. The obvious choice would be to let one machine read a tape that is written by another, but this is not very realistic. Furthermore, the infinite number of potential states for the environment means that many problems become computationally intractable. For the study of protocol design problems, therefore, we must explore other variants of the finite state machine.

7.4 Communicating Finite State Machines

Consider what happens if we allow overlap of the sets of input and output signals of a finite state machine of the type shown in Table 7.1. In all fairness, we cannot say what will happen without first considering in more detail what a “signal” is.

We assume that signals have a finite range of possible values and can change value only at precisely defined moments. The machine executes a two-step algorithm.

- In the first step, the input signal values are inspected and an arbitrary executable transition rule is selected.
- In the second step, the machine changes its control state in accordance with that rule and update its output signals.

These two steps are repeated forever. If no transition rule is executable, the machine will continue cycling through its two-step algorithm without changing state, until a change in the input signal values, affected by another finite state machine, makes a transition possible. A signal, then, has a state, much like a finite state machine. It can be interpreted as a variable that can only be evaluated or assigned to at precisely defined moments.

The behavior of the machine from Table 7.1 is now fully defined, even if we assume a feedback from the output to the input signal. In this case the machine will loop through the following sequence of three states forever: q_0 , q_2 , q_1 . At each step, the machine inspects the output value that was set in the previous transition. The behavior of the machine is independent of the initial value of the input signal.

We can build elaborate systems of interacting machines in this way, connecting the output signals of one machine to the input signals of another. The machines must share a common “clock” for their two-step algorithm, but they are not otherwise synchronised. If further synchronisation is required, it must be realised with a subtle system of handshaking on the signals connecting the machines. This problem has three noticeable features: it is a hard problem, it has been solved, and, from the protocol designer’s point of view, it is irrelevant. Most systems provide a designer with higher-level synchronisation primitives to build a protocol. An example of such synchronisation primitives are the send and receive operations defined in PROMELA.

7.5 Asynchronous Coupling

In protocol design, finite state machines are most useful if they can directly model the phenomena in a distributed computer system. There are two different and equally valid ways of doing this, based on an asynchronous or a synchronous communication model. With the asynchronous model, the machines are coupled via bounded FIFO (first-in first-out) message queues. The signals of a machine are now abstract objects called messages. The input signals are retrieved from input queues, and the output signals are appended to output queues. All queues, and the sets of signals, are still finite, so we have not given up the finiteness of our model.

Synchronisation is achieved by defining both input and output signals to be conditional on the state of the message queues. If an input queue is empty, no input signal is available from that queue, and the transition rules that require one are unexecutable. If an output queue is full, no output signal can be generated for that queue, and the transition rules that produce one are similarly unexecutable.

From this point on we restrict the models we are considering to those with no more than one synchronising event per transition rule; that is, a single rule can specify an input or an output, but not both. The reason for this restriction is twofold. First, it simplifies the model. We do not have to consider the semantics of complicated composites of synchronising events that may be inconsistent (e.g., two outputs to the same output queue that can accommodate only one of the two). Second, it models the real behavior of a process in a distributed system more closely. Note that the execution of a transition rule is an atomic event of the system. In most distributed systems a single send or receive operation is guaranteed to be an atomic event. It is therefore appropriate not to assume yet another level of interlocking in our basic system model.

State	In	Out	Next state
q_0	-	mesg0	q_1
q_1	ack1	-	q_0
q_1	ack0	-	q_2
q_2	-	mesg1	q_3
q_3	ack0	-	q_2
q_3	ack1	-	q_0

Table 7.4 Sender

As an example of asynchronous coupling of finite state machines, Tables 7.4 and 7.5 give transition table models for a simple version of the alternating bit protocol. The possibility of a retransmission after a timeout is not modeled in Table 7.4. We could do so with spontaneous transitions, by adding two rules:

State	In	Out	Next State
q1	-	mesg0	-
q3	-	mesg1	-

The table can model the possibility of retransmissions in this way, though not their probability. Fortunately, this is exactly the modeling power we need in a system that must analyse protocols independently of any assumptions on the timing or speed of individual processes.

State	In	Out	Next State
q0	mesg1	-	q1
q0	mesg0	-	q2
q1	-	ack1	q3
q2	-	ack0	q0
q3	mesg0	-	q4
q3	mesg0	-	q5
q4	-	ack0	q0
q5	-	ack1	q3

Table 7.5 Receiver

The last received message can be accepted as correct in states q1 and q4. A state transition diagram for Tables 7.4 and 7.5 is given in Fig. 7.2. The timeout option in the sender would produce an extra self-loop on states q1 and q3.

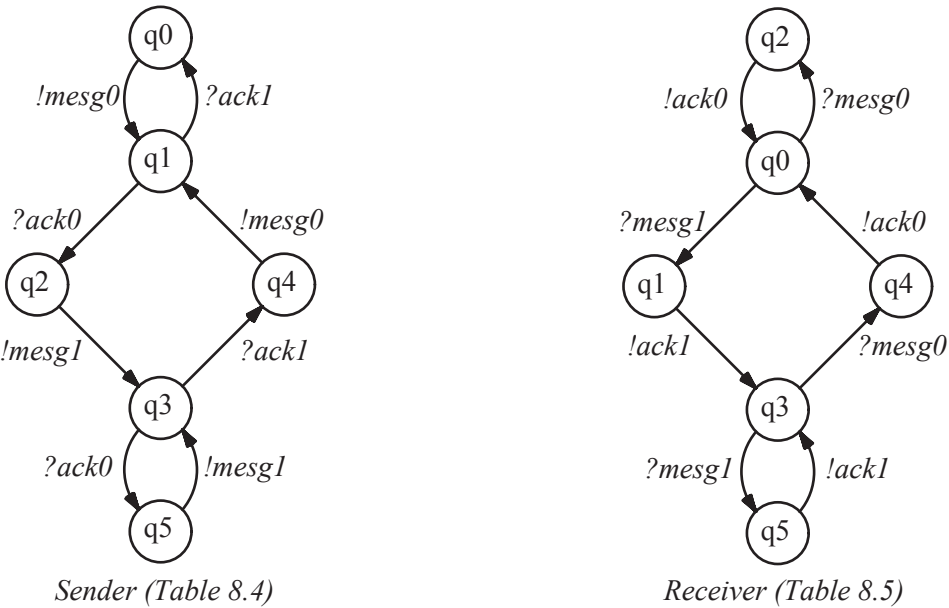


Fig. 7.2 State transition diagrams, Tables 7.4 and 7.5

We do not have parameter values in messages just yet. In the above model the value of the alternating bit is therefore tagged onto the name of each message.

7.6 Synchronous Coupling

The second method for coupling machines is based on a synchronous model of communication. The transition conditions are now the “selections” that the machine can make for communication. Again we allow only one synchronising event per transition rule. The machine can select either an input or an output signal for which a transition rule is specified. To make a move, a signal has to be selected by precisely two machines simultaneously, in one machine as an output and in the other as an input. If such a match on a signal occurs, both machines make the corresponding transition simultaneously and change their selections in accordance with the new states they reach.

Tables 7.6 and 7.7 give an example of synchronously coupled finite state machines. The machine in Table 7.6 can make just one input selection P in state q0 and one output selection V in state q1.

State	In	Out	Next State
q0	P	-	q1
q1	-	V	q0

Table 7.6 User

The second machine is almost the same as the first, but has the inputs and outputs swapped.

State	In	Out	Next State
q0	-	P	q1
q1	V	-	q0

Table 7.7 Server

If we create two machines of type 7.6 and combine them with one machine of type 7.7, we can be certain that for all possible executions the first two machines cannot be in state q1 at the same time. Note that synchronous communication was defined to be binary: exactly two machines must participate, one with a given input selection and the other with the matching output selection. Typically, a parameter value will be passed from sender to receiver in the synchronous handshake. The value transfer, however, is not in the model just yet.

We can again consider the synchronous communication as a special case of asynchronous communication with a queue capacity of zero slots. In the remainder of this chapter we therefore focus on the more general case of a fully asynchronous coupling of finite state machines.

7.7 Formal Description

Let us now see if we can tidy up the informal definitions discussed so far. A communicating finite state machine can be defined as an abstract demon that accepts input symbols, generates output symbols, and changes its inner state in accordance with some predefined plan. For now, these symbols or “messages” are defined as abstract objects without contents. We will consider the extensions required to include value transfer. The finite state machine demons communicate via bounded FIFO queues that map the output of one machine upon the input of another. C_n^m Let us first formally define the concept of a queue.

A *message queue* is a triple (S, N, C) , where:

S is a finite set called the queue vocabulary.

N is an integer that defines the number of slots in the queue, and

C is the queue contents, an ordered set of elements from S .

The elements of S and C are called messages. They are uniquely named, but otherwise undefined abstract objects. If more than one queue is defined we require that the queue vocabularies be disjoint. Let M be the set of all messages queues, a superscript $1 \leq m \leq |M|$ is used to identify a single queue, and an index $1 \leq n \leq N$ is used to identify a slot within the queue. C_n^m then, is the n -th message in the m -th queue. A system vocabulary V can be defined as the conjunction of all queue vocabularies, plus a null element that we indicate with the symbol ε . Given the set of queues M , numbered from 1 to M , the system vocabulary V is defined as

$$V = \bigcup_{m=1}^{|M|} S^m \cup \varepsilon$$

Now, let us define a communicating finite state machine.

A *communicating finite state machine* is a tuple (Q, q_0, M, T) , where,

Q is a finite, non-empty set of states,

q_0 is an element of Q , the *initial state*,

M is a set of message queues, as defined above, and

T is a state transition relation.

Relation T takes two arguments, $T(q, a)$, where q is the current state and a is an action. So far, we allow just three types of actions: inputs, outputs, and a null action ε . The executability of the first two types of actions is conditional on the state of the message queues. If executed, they both change the state of precisely one message queue. Beyond this, it is immaterial, at least for our current purposes, what the precise definition of an input or an output action is.

The transition relation T defines a set of zero or more possible successor states in set Q for current state q . This set will contain precisely one state, unless non-determinism is modeled, as in Table 7.2. When $T(q, a)$ is not explicitly defined, we assume $T(q, a) = \emptyset$.

$T(q, \varepsilon)$ specifies spontaneous transitions. A sufficient condition for these transitions to be executable is that the machine be in state q .

7.8 Execution of Machines

Consider a system of P finite state machines, with overlapping sets of message queues. The union of the sets of all message queues is again called M . This system of communicating finite state machines is executed by applying the following rules, assuming asynchronous coupling only. The elements of finite state machine i are referred to with a superscript i .

FSM Execution Algorithm:

1. Set all machines in their initial state, and initialise all message queues to empty:

$$\forall (i), 1 \leq i \leq P \rightarrow q^i = q_0^i$$

$$\forall (i), 1 \leq i \leq |M| \rightarrow C^i = \emptyset$$

2. Select an arbitrary machine i and an arbitrary transition rule T^i with

$$T^i(q, a) \neq \emptyset \text{ and } a \text{ is executable}$$

and execute it.

3. If no executable transition rules remain, the algorithm terminates.

Action a can be an input, an output, or it can be the null action ϵ . Let $1 \leq d(a) \leq |M|$ be destination queue of an action a , and let $m(a)$ be the message that is sent or received, $m(a) \in S^{d(a)}$. Further, let N^i represent the number of slots in message queue i . In an asynchronous system, for instance, the following three rules can be used to determine if a is executable.

$$a = \epsilon \quad (1)$$

or

$$a \text{ is an input and } m(a) = C_1^{a(a)} \dots \dots \dots (2)$$

or

$$a \text{ is an output and } |C^{d(a)}| < IV \dots \dots \dots (3)$$

FSM Execution Algorithm does not necessarily terminate.

7.9 Minimisation of Machines

Consider the finite state machine shown in Table 7.8, with the corresponding state transition diagram in Fig. 7.3.

Condition		Effect	
Current State	In	Out	Next State
q0	Mesg1	-	q1
q0	Mesg0	-	q2
q1	-	ack1	q0
q2	-	ack0	q0

Table 7.8 Receiver-II

Though this machine has three states fewer than the machine from Table 7.5, it certainly looks like it behaves no differently. Two machines are said to be *equivalent* if they can generate the same sequence of output symbols when offered the same sequence of input symbols. The keyword here is *can*. The machines we study can make nondeterministic choices between transition rules if more than one is executable at the same time. This non determinism means that even two equal machines *can* behave differently when offered the same input symbols. The rule for equivalence is that the machines must have equivalent choices to be in equivalent states.

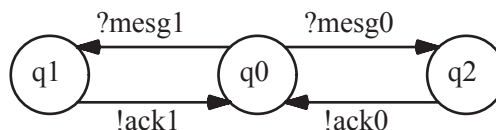


Fig. 7.3 State transition diagram for Table 7.8

States within a single machine are said to be equivalent if the machine can be started in any one of these states and generate the same set of possible sequences of outputs when offered any given test sequence of inputs. The definition of an appropriate equivalence relation for states, however, has to be chosen with some care. Consider the following PROMELA process.

```

proctype A ()
{
    if
    :: q?a -> q?b
  
```

```

    :: q?a -> q?c
  fi
}

```

Under the standard notion of language equivalence that is often defined for deterministic finite state machines, this would be equivalent to

```

proctype B()
{
    q?a;
    if
        :: q?b
        :: q?c
    fi
}

```

Since the set of all input sequences (the language) accepted by both machines is the same. It contains two sequences, of two messages each:

$$\{ q?a;q?b, q?a;q?c \}$$

The behavior of the two processes, however, is very different. The input sequence $q?a;q?b$, for instance, is always accepted by process B but may lead to an unspecified reception in process A. For nondeterministic communicating finite state machines, therefore processes A and B are not equivalent. The definitions given below will support that notion.

In the following discussion of equivalence, state minimisation, and machine composition, we will focus exclusively on the set of control states Q and the set of transitions T of the finite state machines. Specifically, the internal “state” of the message queues in set M is considered to be part of the environment of a machine and not contributing to the state of the machine itself. That this is a safe assumption needs some motivation. Consider, as an extreme case, a communicating finite state machine that accesses a private message queue to store internal state information. It can do so by appending messages with state information in the queue and by retrieving that information later. The message queue is internal and artificially increases the number of states of the machine.

When we consider the message queue to be part of the environment of a machine in the definitions that follow, we ignore the fact that the information that is retrieved from such a private queue is always fixed (i.e., it can only have been placed in the queue by the same machine in a previous state). If we say that two states of this machine are equivalent if they respond to the same input messages in the same way, we do in fact place a *stronger* requirement on the states than strictly necessary. We require, for instance, that the two states would respond similarly to messages that could never be in a private queue for the given state. To suppress state information that could be implicit in the messages queue contents therefore does not relax the equivalence requirements. As we will see, it does lead to simpler algorithms.

Using this approach, the set of control states of a communicating finite state machine can be minimised, without changing the external behavior of the machine, by replacing every set of equivalent states with a single state. More formally, we can say that this equivalence relation defines a partitioning of the states into a finite set of disjoint equivalence classes. The smallest machine equivalent to the given one will have as many states as the original machine has equivalence classes.

We can now define a procedure for the minimisation of an arbitrary finite state machine with $|Q|$ states.

FSM Minimisation Algorithm:

1. Define an array E of $|Q| \times |Q|$ Boolean values. Initially, every element $E[i, j]$ of the array is set to the truth value of the following condition, for all actions a :

$$T(i, a) \neq \emptyset \leftrightarrow T(j, a) \neq \emptyset$$

Two states are not equivalent unless the corresponding state transition relations are defined for the same actions.

2. If the machine considered contains only deterministic choices, T defines a unique successor state for all *true* entries of array E . Change the value of all those entries

$E[i, j]$ to the value of

$$A(a), E[T(i, a), T(j, a)]$$

It means that states are not equivalent unless their successors are also equivalent. When $T(i, a)$ and $T(j, a)$ can have more than one element, the relation is more complicated. The value of $E[i, j]$ is now set to *false* if either of the following two conditions is *false* for any action a .

$$\begin{aligned} \forall (p), p \in T(i, a) \rightarrow \exists (q), q \in T(j, a) \text{ and } E[p, q] \\ \forall (q), q \in T(j, a) \rightarrow \exists (p), p \in T(i, a) \text{ and } E[q, p] \end{aligned}$$

This means that states i and j are not equivalent unless for every possible successor state p of state i there is at least one equivalent successor state q of state j , and vice versa.

3. Repeat step 2 until the number of *false* entries in array E can no longer be increased.

The procedure always terminates since the number of entries of the array is finite and each entry can only be changed once, from *true* to *false*, in step 2. When the procedure terminates, the entries of the array define a partitioning of the $|Q|$ states into equivalence classes. State i , $1 \leq i \leq |Q|$, is equivalent with all states j , $1 \leq j \leq |Q|$, with $E[i, j] = \text{true}$.

	q0	q1	q2	q3	q4	q5
q0	1	0	0	1	0	0
q1	0	1	0	0	0	1
q2	0	0	1	0	1	0
q3	1	0	0	1	0	0
q4	0	0	1	0	1	0
q5	0	1	0	0	0	1

Table 7.9 Equivalence

If we apply this procedure to the finite state machine in Table 7.5, we obtain the stable array of values for E shown in Table 7.9 after a single application of the first two steps. A one in the table represents the Boolean value *true*. From the table we see that state pairs (q0, q3), (q1, q5), and (q2, q4) are equivalent. We can therefore reduce Table 7.5 to the three-state finite state machine that was shown in Table 7.8. It is necessarily the smallest machine that can realise the behavior of Table 7.5.

The procedure above can be optimised by noting, for instance, that array E is symmetric: for all values of i and j we must have $E(i, j) = E(j, i)$. Trivially, every state is equivalent with itself.

7.10 The Conformance Testing Problem

The procedure for testing equivalence of states can also be applied to determine the equivalence of two machines. The problem is then to determine that every state in one machine has an equivalent in the other machine, and vice versa. Of course, the machines need not be equal to be equivalent.

A variant of this problem is of great practical importance. Suppose we have a formal protocol specification, in finite state machine form, and an implementation of that specification. The two machines must be equivalent that is the implementation, seen as a black box, should respond to input signals exactly as the reference machine would. We cannot, however, know anything with certainty about the implementation's true internal structure.

We can try to establish equivalence by systematically probing the implementation with trial input sequences and by comparing the responses with those of the reference machine. The problem is now to find just the right set of test sequences to establish the equivalence or non-equivalence of the two machines. This problem is known in finite state machine theory as the *fault detection* or conformance testing problem. Carrying this one step further, we may also want to determine the internal structure of an unknown finite state machine, just by probing it with a known set of input signal and by observing its responses. This problem is known as the state verification problem. Without any further knowledge about the machine, that problem is unsolvable. Note, for instance, that in Fig. 7.1 state q_3 cannot be distinguished from state q_2 by any test sequence that starts with an input symbol one. Similarly, state q_1 cannot be distinguished from state q_3 by any sequence starting with a zero. Since every test sequence has to start with either a one or a zero there can be no single test sequence that can tell us reliably in which state this machine is.

7.11 Combining Machines

By collapsing two separate finite state machines into a single machine the complexity of formal validations based on finite state machine models may be reduced. The problem is to find a tuple (Q, q_0, M, T) for the combined machine, given two machines (Q^1, q_0^1, M^1, T^1) and (Q^2, q_0^2, M^2, T^2)

FSM Composition Algorithm:

1. Define the product set of the two sets of states of the two state machines. If the first machine has $|Q^1|$ states and the second machine has $|Q^2|$ states the product set contains $|Q^1| \times |Q^2|$ states. We initially name the states of the new machine by concatenating the state names of the original machines in a fixed order. This defines set Q of the combined machine. The initial state q of the new machine is the combination $q_0^1 q_0^2$ of the initial states of the two original machines.
2. The set of message queues M of the combined machine is the union of the sets of queues of the separate machines, $M^1 \cup M^2$. The two original sets need not be disjoint. The vocabulary V of the new machine is the combined vocabulary of M^1 and M^2 , and the set of actions a is the union of all actions that the individual machines can perform.
3. For each state $q^1 q^2$ in Q , define transition relation T for each action a as the nondeterministic choice of the corresponding relations of M^1 and M^2 separately, when placed in the individual states q^1 and q^2 . This can be written:

$$\forall (q^1 q^2) \forall (a), \rightarrow T(q^1 q^2, a) = T^1(q^1, a) \cup T^2(q^2, a)$$

The combined machine can now be minimised using FSM Minimisation Algorithm. and FSM Composition Algorithm can be readily adapted to combine more than two machines.

The greatest value of the composition technique from the last section is that it allows us to simplify complex behaviors. In protocol validations, for instance, we could certainly take advantage of a method that allows us to collapse two machines into one. One method would be to compose two machines using FSM Composition Algorithm remove all their internal interactions, i.e., the original interface between the two machines, and minimise the resulting machine.

There are two pieces missing from our finite state machine framework to allow us apply compositions and reductions in this way. First, the finite state machine model we have developed so far cannot easily represent PROMELA models. In the next section we show how the basic finite state machine model can be extended sufficiently to model PROMELA models elegantly. The second piece that is missing from our frame work is a method for removing internal actions from a machine without disturbing its external behavior.

7.12 Extended Finite State Machines

The finite state machine models we have considered so far still fall short in two important aspects: the ability to model the manipulation of variables conveniently and the ability to model the transfer of arbitrary values. These machines were defined to work with abstract objects that can be appended to and retrieved from queues and they are only synchronised on the access to these queues.

We make three changes to this basic finite state machine model. First, we introduce an extra primitive that is defined much like a queue: the variable. Variables have symbolic names and they hold abstract objects. The abstract objects, in this case, are integer values. The main difference from a real queue is that a variable can hold only one value at a time, selected from a finite range of possible values. Any number of values can be appended to a variable, but only the last value that was appended can be retrieved.

The second change is that we will now use the queues specifically to transfer integer values, rather than undefined abstract objects. Third, and last, we introduce a range of arithmetic and logical operators to manipulate the contents of variables.

Current State	In	Out	Next State
q0	s0	-	-
q0	s1	-	q1
q0	s2	-	q2
q0	rv	-	r0
r0	-	0	q0
q1	s0	-	q0
q1	s1	-	-
q1	s2	-	q2
q1	rv	-	r1
r1	-	1	q1
q2	s0	-	q0
q2	s1	-	q1
q2	s2	-	-
q2	rv	-	r2
r2	-	2	q2

Table 7.10 Finite state variable

The extension with variables, provided that they have a finite range of possible values, does not increase the computational power of finite state machines with bounded FIFO queues. A variable with a finite range can be simulated trivially by a finite state machine. Consider the six-state machine shown in Table 7.10 that models a variable with the range of values from zero to two. The machine accepts four different input messages. Three are used to set the pseudo variable to one of its three possible values. The fourth message, rv, is used to test the current value of the pseudo variable. The machine will respond to the message rv by returning one of the three possible values as an output message.

Thus, at the expense of a large number of states, we can model any finite variable without extending the basic model, as a special purpose finite state machine. The extension with explicit variables, therefore, is no more than a modeling convenience. Recall that the transition rules of a finite state machine have two parts: a condition and an effect. The conditions of the transition rules are now generalised to include boolean expressions on the value of variables, and the effects (i.e., the actions) are generalised to include assignment to variables.

An *extended finite state machine* can now be defined as a tuple (Q, q_0, M, A, T) , where A is the set of variable names. Q , q_0 , and M are as defined before. The state transition relation T is unchanged. We have simply defined two extra types of actions: Boolean conditions on and assignments to elements of set A . A single assignment can change the value of only one variable. Expressions are built from variables and constant values, with the usual arithmetic and relational operators.

In the spirit of the validation language PROMELA, we can define a condition to be executable only if it evaluates to *true*, and let an assignment always be executable. Note carefully that the extended model of communicating finite state machines is a *finite* state model, and almost all results that apply to finite state machines also apply to this model.

EXTENDED I/O

Input and output actions can now be generalised as well. We will define I/O actions as finite, ordered sets of values. The values can be expressions on variables from A , or simply constants. By definition the first value from such an ordered set defines the destination queue for the I/O, within the range $1 \dots |M|$. The remaining values define a data structure that is appended to, or retrieved from, the queue when the I/O action is performed. The semantics of executability can again be defined as in PROMELA.

EXAMPLE

Consider the following PROMELA fragment.

```
proctype Euclid
{
    pvar x, y;
    In?x,y;
    do
        :: (x > y) -> x = x - y
        :: (x < y) -> y = y - x
        :: (x == y) -> break
    od;
    Out!x
}
```

The process begins by receiving two values into variables x and y , and it completes by returning the greatest common divisor of these two values to its output queue. The matching extended finite state machine is shown in Table 7.11, where we combine all conditions, assignments and I/O operations in a single column.

Current State	Action	Next State
q0	In?x, y	q1
q1	$x > y$	q2
q1	$x < y$	q3
q1	$x = y$	q4

q2	$x = x - y$	q1
q3	$y = y - x$	q1
q4	Out!x	q5
q5	-	-

Table 7.11 Extended finite state machine

Set A has two elements, x and y .

We now have a simple mapping from PROMELA models to extended finite state machines. Algorithm 7.3, for instance, can now be used to express the combined behavior of two PROMELA processes by a single process. We noted before that this technique could be especially useful in combination with a *hiding* method that removes internal actions from a machine without disturbing its external behavior. We take a closer look at such methods in the next section.

7.13 Generalisation of Machines

Consider the following PROMELA model.

```

1. proctype generalize_me(chan ans; byte p)
2. {   chan internal[1] of { byte };
3.   int r, q;
4.
5.   internal!cookie;
6.   r = p/2;
7.   do
8.   :: (r <= 0) -> break
9.   :: (r > 0) ->
10.  q = (r*r + p)/(2*r);
11.  if
12.  :: (q != r) -> skip
13.  :: (q == r) -> break
14.  fi;
15.  r = q
16. od;
17. internal?cookie;
18. if
19. :: (q < p/3) -> ans!small
20. :: (q >= p/3) -> ans!great
21. fi
22. }
```

A process of this type will start by sending a message cookie to a local message channel. It will then perform some horrible computation, using only local variables, read back the message from the channel internal, and send one of two possible messages over an external message channel ans.

Now, for starters, nothing detectable will change in the external behavior of this process if we remove lines 2, 5 and 17. The message channel is strictly local, and there is no possible behavior for which any of the actions performed on the channel can be unexecutable. Lines 5 and 17 are therefore equivalent to skip operations and can be deleted from the model. Reductions, or *prunings*, of this type produce machines that have a equivalent external behavior to the non-reduced machines. This is not true for the next type of reduction we discuss: generalisation.

The horrible computation performed by the process, between lines 6 and 16, does not involve any global variables or message interactions. Once the initial value of variable `p` is chosen, the resulting message sent to channel `ans` is fixed. If we are interested in just the external behavior of processes of type `generalize_me`, independently of the precise value of `p`, the model could be rewritten as

```
proctype generalized(chan ans; byte p)
{
    if
    :: ans!small
    :: ans!great
    fi
}
```

This specification merely says that within a finite time after a process of this type is instantiated, it sends either a message of type `small` or a message of type `great` and terminates. To justify the reduction we must of course show that the loop in the original specification will always terminate. If this is not the case, or cannot be proven, the correct reduction would be

```
proctype generalized(chan ans; byte p)
{
    if
    :: (0)
    :: ans!small
    :: ans!great
    fi
}
```

where the possibility of blocking is preserved explicitly.

We call a reduction of this type, where uninteresting but strictly local, behavior is removed, a *generalisation*. A process of type `generalised` can do everything that a process of type `generalize_me` can do, but it can do more. The `generalised` process can, for instance, for any given parameter `p`, return either of the two messages, while the non-`generalised` processes will pick only one. The `generalised` processes are only more general in the way it can produce output, not in the way it can accept input, or in general in the way other processes can constrain its behavior via global objects.

The usefulness of generalisations in protocol validation can be explained as follows. Consider two protocol modules `A` and `B` whose combined behavior is too complex to be analyzed directly. We want to validate a correctness requirement for the processes in module `A`. We can do this by simplifying the behavior in module `B`, for instance by combining, pruning, generalising, and minimising machines. If the behavior in module `B` is `generalised` as discussed above, the new module `B` will still be capable of behaving precisely like the unmodified module `B`, but it can do more. If we can prove the observance of a correctness requirement for module `A` in the presence of the `generalised` module `B`, which may be easier, the result will necessarily also hold for the original, more complex, module `B`, because the original behavior is a subset of the new behavior.

Two things should be noted. First, if we are interested in proving a property of module `A` we simplify its environment, which in this case is module `B`. We do not change module `A` itself. Second, it is important that the modified behavior of module `B` does not, by virtue of the modifications, allow module `A` to pass its test. This is guaranteed by the fact that the `generalised` module `B` continues to adhere to all constraints that can be imposed by `A`, via global objects, such as message channels and variables. The validation, then, gives us the best of both worlds. It performs a stronger test, since it validates properties for more general conditions than defined in the original protocol, yet it is easier to perform, since a `generalised` process can be smaller than its original.

A general method for the reduction of an arbitrary PROMELA proctype definition can be described as follows. Identify selection and repetition structures in which all the guards are conditions on local variables only, and in which the union of all guards is true. Replace each of the guards identified in the first step with the PROMELA statement `skip`. Replace all assignments to local variables that are no longer part of any condition, with `skip`. Remove all redundant declarations and minimise or simplify the new proctype body, for instance, by combining equal clauses in selection and repetition structures, and by removing redundant `skip` statements.

If we apply this method to the process type `generalise_me`, after pruning away the interactions on channel `internal`, we can reduce it to

```
proctype generalized_2(chan ans; byte p)
{
    do
    :: break
    :: skip
    od;
    if
    :: ans!small
    :: ans!great
    fi
}
```

which is similar to and has the same external behavior as the (second) version we derived earlier with a little more hand waving. Note that the loop in the above version does not necessarily terminate.

7.14 Restricted Models

To conclude this chapter, we look at two other interesting variants of the basic finite state machine model that have been applied to the study of protocol problems. Many variations of the basic finite state machine model have been used for the analysis of protocol systems, both restrictions and extensions. The restricted versions have the advantage, at least in principle, of a gain in analytical power. The extended versions have the advantage of a gain in modeling power. The most popular variants of the finite state machine are formalised token nets, often derived from the Petri Net model. Below we briefly review the Petri Net model and discuss one of the variations, the FIFO Net.

7.14.1 Petri Net

A Petri Net is a collection of places, transitions, and directed edges. Every edge connects a place to a transition or vice versa. Places are graphically represented by circles, transitions by bars, and edges by directed arcs. Informally, a place corresponds to a condition and a transition corresponds to an event. The *input places* of transition T are the places that are directly connected to T by one or more edges. The input places correspond to conditions that must be fulfilled before the event corresponding to T can occur. The output places of a transition similarly correspond to the effect of the event on the conditions represented by the places.

Each place that corresponds to a fulfilled condition is marked with one or more *tokens* (sometimes called a *stone*). The occurrence of an event is represented in the Petri Net as the *firing* of a transition. A transition is enabled if there is at least one token in each of its input places. The effect of a firing is that one token is added to the markings of all output places of the firing transition, and one token is removed from the markings of all its input places.

Two transitions are said to conflict if they share at least one input place. If the shared place contains precisely one token, both transitions may be enabled to fire, but the firing of one transition disables the other. By definition the firing of any combination of two transitions is always mutually exclusive: only one transition can fire at a time. By assigning zero or more tokens to each place in the net we obtain an initial marking. Each firing creates a new marking. A series of firings is called an *execution sequence*. If for a given initial marking all possible execution

sequences can be made infinitely long, the initial marking, and trivially all subsequent markings, are said to be live. If in a certain marking no transition is enabled to fire, the net is said to hang. An initial making is said to be safe if no subsequent execution sequence can produce a marking where any place has more than one token.

A number of properties have been proven about Petri Nets, but mostly about still further simplified versions. Two examples of such variants are: Petri Nets in which precisely one edge is directed to and from each place. Such nets are called *marked graphs*. In a marked graph there can be no conflicting transitions. Petri Nets in which all transitions have at most one input place and one output place. These nets are called transition diagrams.

Fig. 7.4 gives an example of a Petri Net modeling a deadlock problem. Initially, the two top transitions $t1$ and $t2$ are enabled. After $t1$ fires, transition $t3$ becomes enabled. If it fires, all is well. If in this marking, however, transition $t2$ fires, the net will *hang*.

A token in a Petri Net symbolises more than the fulfillment of a *condition*, as described above. It also symbolises a *control flow point* in the program, and it symbolises a *privilege* to proceed beyond a certain point. A token models a shared resource that can be claimed by more than one transition. All these abstractions symbolise the enforcement of partial orderings on the set of possible execution sequences in the system modeled. Especially relevant to the protocol modeling problem is that mixing these abstractions can make it more difficult than necessary to distinguish computation from communication in a Petri Net model.

The complexity of a Petri Net representation rises rapidly with the size of the problem being modeled. It is virtually impossible to draw a clear net for protocol systems that include more than two or three processes. This makes the Petri Net models relatively weak in modeling power compared to communicating finite state machines, without offering an increase in analytical power. There are, for instance, no standard procedures, other than reachability analysis, to analyse a Petri Net for the presence of hang states. Neither are there standard procedures for simplifying a large Petri Net into one or more smaller, equivalent ones.

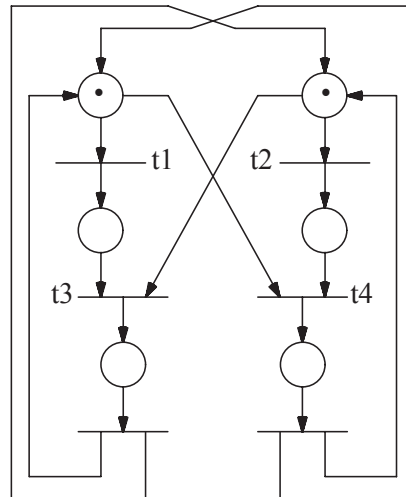


Fig. 7.4 Petri Net with hang state

One final note on the modeling power of basic Petri Nets. We observed above that the places in a Petri Net can be used to model conditions. It is fairly easy to model logical *and* and *or* tests on places using multiple edges, but there is no general way to model a logical *not*-operation (negation). With a logical *not*-operation it would be possible to define that a transition can fire if a place holds no tokens.

Of course, there are many good applications of Petri Net theory. They have been applied successfully to the study of a range of theoretical problems in parallel computation. For the above pragmatic reasons, however, we conclude that Petri Nets do not give us an advantage in the study of protocol design and validation problems.

7.14.2 FIFO Nets

FIFO Nets are an interesting generalisation of Petri Nets and a relatively recent addition to the range of tools proposed for studying distributed systems. A FIFO Net, like a Petri Net, has places, edges, and transitions, but the places contain symbols rather than tokens. The symbols are appended to and reclaimed from the places by transition firings. They are stored by the places in FIFO queues. Both incoming and outgoing edges of transitions are labeled with symbol names. A transition can only fire if the queue of each of its input places can deliver the symbol that corresponds to the edge connecting the transition to that place. Upon firing the labels on the outgoing edges specify which symbols are to be appended to the queues of the corresponding output places.

The generalisation of FIFO Nets is strong enough to make them equivalent in computational power to the finite state machines that we defined earlier. At last, there are no better procedures to analyse FIFO Nets for interesting protocol errors, such as deadlock. In some cases, procedures do exist for restricted versions of FIFO Nets, but again the restrictions generally reduce the modeling power too severely to make them of interest as a general tool for designing or analysing protocol systems.

Summary

- The formal model of a communicating finite state machine plays an important role in three different areas of protocol design: formal validation, protocol synthesis, and conformance testing.
- In the traditional finite state machine model, the environment of the machine consists of two finite and disjoint sets of signals: input signals and output signals.
- Turing machine is used extensively in theoretical computer science as the model of choice in, for instance, the study of computational complexity.
- In protocol design, finite state machines are most useful if they can directly model the phenomena in a distributed computer system.
- Synchronisation is achieved by defining both input and output signals to be conditional on the state of the message queues.
- The second method for coupling machines is based on a synchronous model of communication.
- A communicating finite state machine can be defined as an abstract demon that accepts input symbols, generates output symbols, and changes its inner state in accordance with some predefined plan.
- Two machines are said to be *equivalent* if they can generate the same sequence of output symbols when offered the same sequence of input symbols.
- Two states are not equivalent unless the corresponding state transition relations are defined for the same actions.
- The procedure for testing equivalence of states can also be applied to determine the equivalence of two machines.
- By collapsing two separate finite state machines into a single machine the complexity of formal validations based on finite state machine models may be reduced.
- The finite state machine models we have considered so far still fall short in two important aspects: the ability to model the manipulation of variables conveniently and the ability to model the transfer of arbitrary values.
- The most popular variants of the finite state machine are formalised token nets, often derived from the Petri Net model.
- A Petri Net is a collection of *places*, *transitions*, and directed *edges*.
- FIFO Nets are an interesting generalisation of Petri Nets and a relatively recent addition to the range of tools proposed for studying distributed systems.

References

- Kam, T., 1996. *Synthesis of Finite State Machines: Functional Optimization*, Springer.
- Diaz, M., 2010. *Petri Nets: Fundamental Models, Verification and Applications*, John Wiley & Sons.
- *Finite State Machines* [Pdf] Available at: <http://www.spinroot.com/spin/Doc/Book91_PDF/F8.pdf> [Assessed 30 May 2013].
- *Finite State Machines* [Pdf] Available at: <<http://www.cse.chalmers.se/~coquand/AUTOMATA/book.pdf>> [Assessed 30 May 2013].
- 2010. *Lecture 1-Finite State Machine* [Video online] Available at: <<https://www.youtube.com/watch?v=HyUK5RAJg1c>> [Accessed 4 June 2013].
- 2011. *Lec -26 Turing Machines* [Video online] Available at: <<https://www.youtube.com/watch?v=IhyEGNn-7Uo>> [Accessed 4 June 2013].

Recommended Reading

- Davis, S. J., Davis, J. and Reese, B. R., 2008. *Finite State Machine Datapath Design, Optimization, and Implementation*, Morgan & Claypool Publishers.
- Desel, J. and Esparza, J., 2005. *Free Choice Petri Nets*, Cambridge University Press.
- Dong, J., 2007. *Network Dictionary*, Javvin Technologies Inc.

Self Assessment

- The formal model of a communicating finite state machine plays an important role in _____ different areas of protocol design.
 - two
 - three
 - four
 - five
- Which of the following is a collection of places, transitions, and directed edges?
 - Petri Net
 - FIFO Net
 - PROMELA
 - Turing
- The procedure for testing equivalence of states can also be applied to determine the equivalence of _____ machines.
 - three
 - four
 - two
 - five

- Match the following

1. The second method for coupling machines	A. Is used extensively in theoretical computer science.
2. Turing machine	B. Is achieved by defining both input and output signals to be conditional.
3. I/O actions	C. Finite, ordered sets of values
4. Synchronisation	D. Based on a synchronous model of communication.

- 1-A, 2-C, 3-B, 4-A
 - 1-C, 2-D, 3-A, 4-D
 - 1-B, 2-A, 3-D, 4-C
 - 1-D, 2-A, 3-C, 4-B
- The _____ in the Turing machine model is a tape of infinite length.
 - environment
 - arbitrary square
 - transition rule
 - tape
 - The procedure for testing equivalence of states can also be applied to determine the _____.
 - variant
 - protocol specification
 - equivalence of two machines
 - input signals

7. How could the complexity of formal validations based on finite state machine models be reduced?
 - a. By setting the greatest value of the composition technique
 - b. By collapsing two separate finite state machines into a single machine
 - c. By modeling any finite variable without extending the basic model
 - d. By including Boolean expressions on the value of variables
8. A variable with a finite range can be simulated trivially by a _____.
 - a. finite state machine
 - b. turing machine
 - c. finite automata
 - d. tape
9. A _____ is a collection of places, transitions, and directed edges.
 - a. Transistor
 - b. FIFO nets
 - c. Petri Net
 - d. Tape
10. Which of the following statement is false?
 - a. Two transitions are said to conflict if they share at least one input place.
 - b. A series of firings is called an execution sequence.
 - c. The complexity of a Petri Net representation rises rapidly with the size of the problem being modeled.
 - d. Petri Nets are an interesting generalisation of Petri Nets and a relatively recent addition to the range of tools proposed for studying distributed systems.

Chapter VIII

Overview of Proposition and Predicate Logic

Aim

The aim of this chapter is to:

- explain the proposition logic
- elucidate the truth table for various equations
- explicate the predicate logic

Objectives

The objectives of this chapter are to:

- define proposition
- explain the valuation function
- explicate assignment function

Learning outcome

At the end of this chapter, you will be able to:

- analyse substitution
- identify natural deduction for predicate logic
- understand the meaning of semantic

8.1 Introduction

The subject of logic is to examine human reasoning and to formulate rules to ensure that such reasoning is correct. Modern logic does so in a formal mathematical way, hence names like “symbolic logic”, “formal logic”, “mathematical logic”. The logical approach includes the expression of human knowledge in a formal language (often called “knowledge representation language”), and second the application of reasoning rules to these formal representations. This formal character opens the possibility that the human reasoning process can be implemented on a computer. That does not necessarily mean that a logical language can model every aspect of human reasoning in a satisfactory way, but it does explain why formal logic is important for artificial intelligence, and for automated reasoning in general.

Below we describe two logical systems:

- In proposition logic we can express statements as a whole, and combinations of them. Intuitively, a statement is a sentence in which something is told about some reality, and which can be *true or false* about that reality. For example, if p is the statement “Albert is at home”, and q means “the door is locked”, then $q \rightarrow \neg p$ says: “if the door is locked, then Albert is not at home”.
- In first order predicate *logic* a statement has a specific inner structure, consisting of terms and predicates. Terms denote objects in some reality, and predicates express properties of, or relations between those objects. For example, the same example as before might be expressed as $Locked(d) \rightarrow \neg AtHome(a)$. Here, *Locked* and *AtHome* are predicates, and d and a are terms, all with obvious meanings.

One further possibility on predicate logic is that we can speak about some and all objects. For example, $\forall x. AtHome(x)$ means that every x is at home (leaving open for the moment whether x is a person or something else).

In the descriptions below we start with the formal language definition, both syntactically and semantically, and then give rules according to two approaches: natural deduction and semantic tableaux. The latter ones are treated at the end for proposition and predicate logic together. In addition there are other systems for reasoning rules like Hilbert style systems, sequent calculus, resolution, but we won’t discuss them here.

This text is very concise and only describes some basic definitions. For further questions about logical systems, like the strength or equivalence of several systems of rules, we refer the reader to the literature.

8.2 Proposition Logic

A proposition is a statement of language that formulates something about an external world. A proposition can either be true or false. Thus questions, commands, promises, etc., are not propositions.

8.2.1 Formal Language Definition

The formal language of proposition logic is defined recursively:

- there is an infinite list of elementary propositions $p, q, r, \dots$,
- if φ is a proposition, then $\neg \varphi$ is a proposition too,
- if φ, ψ are propositions, then $\varphi \vee \psi, \varphi \wedge \psi, \varphi \rightarrow \psi, \varphi \leftrightarrow \psi$ are propositions too,
- nothing else is a proposition.

The symbols $\neg, \vee, \wedge, \rightarrow, \leftrightarrow$ are (logical) *connectives*, called *negation* (for “not”), *disjunction* (“or”), *conjunction* (“and”), *implication* (“if then”), *equivalence* (“if and only if”), respectively. Some examples of propositions are

$$p \vee q, p \rightarrow (q \vee r), \quad (p \wedge q) \leftrightarrow (r \rightarrow ((\neg q) \vee p)).$$

In these examples *brackets* indicate in what order the proposition is constructed by the rules from the definition. In order to avoid too many brackets, connectives can be ordered according to their binding strength as follows (from strong to weak): $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$. Thus, the final example above might also have been written as:

$$p \wedge q \leftrightarrow r \rightarrow \neg q \vee p.$$

However, for reasons of clarity, the use of brackets may be helpful. Probably

$$(p \wedge q) \leftrightarrow (r \rightarrow \neg q \vee p)$$

expresses the essential structure of the same proposition more clearly.

8.2.2 Truth Tables

The question whether a proposition is true or false depends on the truth or falsity of the elementary propositions and the way in which they are combined by the connectives, as expressed in the following *truth tables* (0 stands for false, 1 for true):

p	$\neg p$
0	1
1	0

p	q	$p \vee q$	$p \wedge q$	$p \rightarrow q$	$p \leftrightarrow q$
0	0	0	0	1	1
0	1	1	0	1	0
1	0	1	0	0	0
1	1	1	1	1	1

Remark: The language of proposition logic is a *formal language*, i.e., the propositions form an inductively defined set of *syntactic* expressions, and the truth tables give the *semantics* of these expressions. For predicate logic we will define this more formally by means of a “semantic value function”.

8.2.3 Natural deduction for proposition logic

Below φ, ψ, χ are formulas in the language of proposition logic, i.e., they are constructed from elementary propositions $p, q, r, \dots$, and from the logical connectives $\neg, \wedge, \vee, \rightarrow$.

For every connective there are one or two introduction rules (on the left) and elimination rules (on the right). We will not present the rules for “ $\leftrightarrow$ ”, since $\varphi \leftrightarrow \psi$ can easily be considered as shorthand notation for $(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$.

The symbol $\perp$ is called *falsum*, and stands for a contradiction, i.e., a formula which is always false. For example, we may take $\perp$ as shorthand notation for any formula of the form $\varphi \wedge \neg \varphi$.

In the rules (VE), ($\rightarrow$ I), ($\neg$ I), ($\neg$ Ec) there are formulas between “[” and “]”. That means that these formulas are used as (temporary) *assumptions*, which are cancelled at the moment that such a rule is applied.

$\frac{\varphi \quad \psi}{\varphi \wedge \psi} (\wedge I)$ $\frac{\varphi}{\varphi \vee \psi} (\vee I_1) \quad \frac{\psi}{\varphi \vee \psi} (\vee I_2)$ $\frac{[\varphi] \quad \vdots \quad \psi}{\varphi \rightarrow \psi} (\rightarrow I)$ $\frac{[\varphi] \quad \vdots \quad \perp}{\neg \varphi} (\neg I)$		$\frac{\varphi \wedge \psi}{\varphi} (\wedge E_1) \quad \frac{\varphi \wedge \psi}{\psi} (\wedge E_2)$ $\frac{[\varphi] \quad \vdots \quad \chi \quad [\psi] \quad \vdots \quad \chi}{\chi} (\vee E)$ $\frac{\varphi \rightarrow \psi \quad \varphi}{\psi} (\rightarrow E)$ $\frac{[\neg \varphi] \quad \vdots \quad \perp}{\varphi} (\neg E_c)$
---	--	---

The rules for negation could also have been formulated without using $\perp$:

$\frac{[\varphi] \quad \vdots \quad \psi \quad [\varphi] \quad \vdots \quad \neg \psi}{\neg \varphi} (\neg I')$		$\frac{[\neg \varphi] \quad \vdots \quad \psi \quad [\neg \varphi] \quad \vdots \quad \neg \psi}{\varphi} (\neg E'_c)$
---	--	--

Thus, the rule $(\neg I)$ says that if some proposition ψ and its negation $\neg \psi$ can both be derived from the same assumption φ , this assumption leads to a contradiction. Hence, φ cannot be true, and we may conclude $\neg \varphi$. The rule $(\neg E'_c)$ can be explained in the same way.

However, note that first applying the $(\wedge I)$ -rule leads to $\psi \wedge \neg \psi$.

Since $\perp$ is considered as shorthand notation for a proposition of this form, we may also choose the simpler original formulation with $\perp$. For example, the rule $(\neg I)$ now directly says that if φ leads to a contradiction, we may conclude $\neg \varphi$.

The above set of rules characterises so-called classical logic (hence the index “c” in the $\neg E_c$ -rule). Based on a different notion of truth, intuitionistic logic replaces this rule by the following rule $\perp - (\neg E_i)$.

In classical logic formulas like $\neg\neg\varphi \rightarrow \varphi$ and $\varphi \vee \neg\varphi$ are provable, whereas in intuitionistic logic they are not. Intuitionistic logic and its background are outside the scope of this text.

Example: Proofs in natural deduction take the form of a *tree*. The tree below proves $\varphi \wedge \psi \rightarrow \chi$ from the assumption $\varphi \rightarrow \psi \rightarrow \chi$.

$$\begin{array}{c}
 \varphi \rightarrow \psi \rightarrow \chi \quad \frac{[\varphi \wedge \psi]}{\varphi} (\wedge E_1) \quad \frac{[\varphi \wedge \psi]}{\psi} (\wedge E_2) \\
 \hline
 \psi \rightarrow \chi \quad (\rightarrow E) \quad \psi \quad (\rightarrow E) \\
 \hline
 \chi \quad (\rightarrow I) \\
 \hline
 \varphi \wedge \psi \rightarrow \chi
 \end{array}$$

In this proof the formula $\varphi \wedge \psi$ is a (temporary) assumption that is used by the $\wedge$ -elimination rules (twice). This assumption is cancelled by the $\rightarrow$ I-rule. After this cancellation the only remaining assumption (axiom) in this proof is the formula $\varphi \rightarrow \psi \rightarrow \chi$. When there are several assumptions which are cancelled during a proof, it has to be indicated at which step in the proof these assumptions are cancelled.

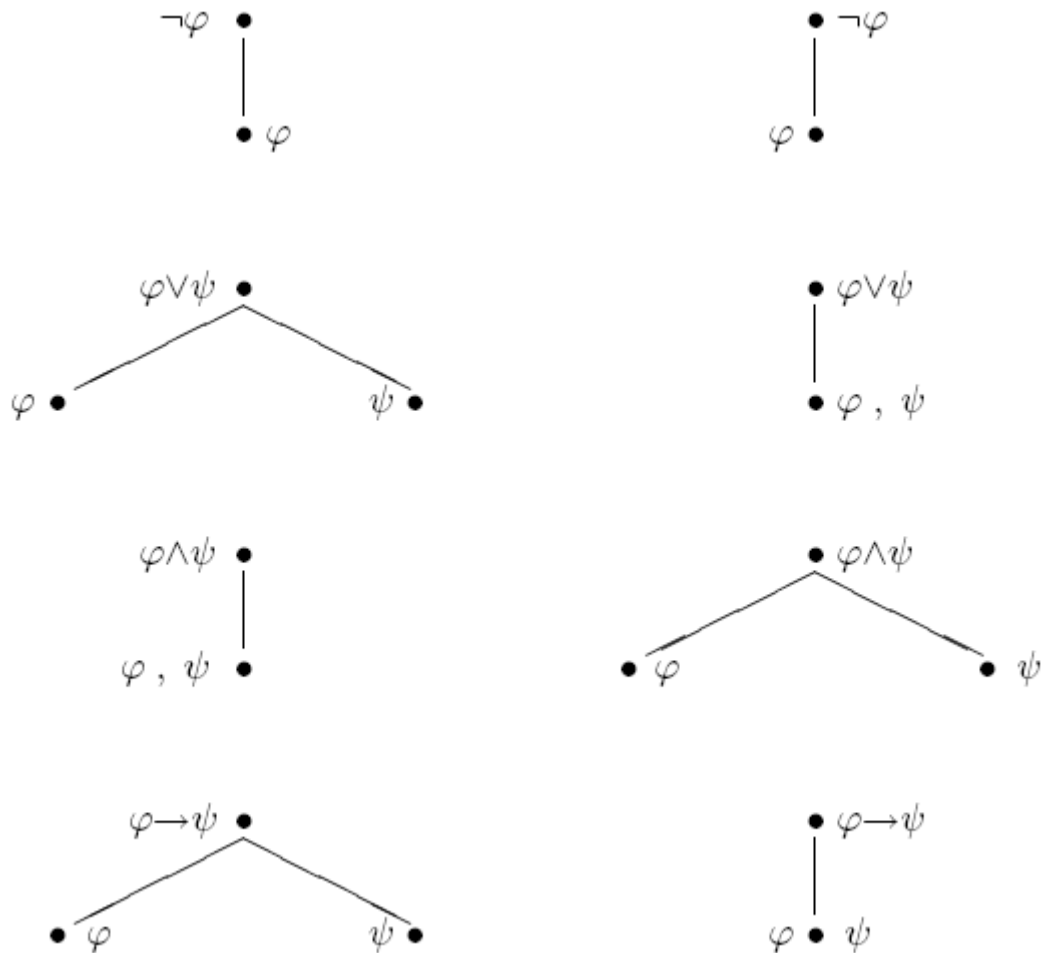
8.2.4 Semantic tableaux

In natural deduction a proof is “built up” from the premises towards the conclusion. With a semantic tableau we precede the other way, i.e., we “breakdown” the formulas. The method of semantic tableaux is mechanic in nature, whereas the method of natural deduction requires some creativity and understanding.

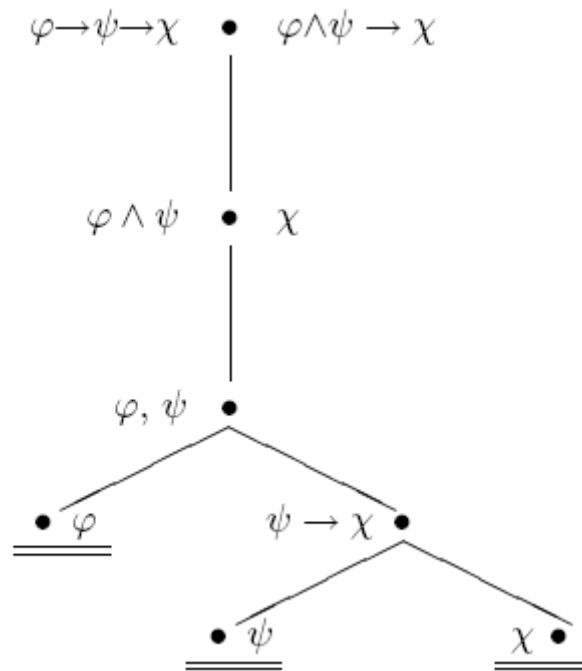
A semantic tableau is a tree in which every node consists of a bullet with some propositions on its left-hand and right-hand side. Formulas on the left-hand side of the bullet are supposed to be true, on the right-hand side they are supposed to be false.

A tree in the semantic tableau method starts by stating that all the premises are true, but the conclusion that has to be proved, is not true. The aim then is to find in a systematic way a truth value of the elementary propositions, which makes this starting point possible. If no such truth values can be found, the starting point cannot be the case, and thus the conclusion must be true whenever the premises are true.

For every logical connective there is a left rule and a right rule, saying that if a compound formula is true/false, then a certain truth value follows for the constituents of the compound formula. Thus, a rule has to be read from top to bottom. Sometimes there are a few possibilities, indicated by a “split” of the rule.



Example: We take the same example as before, i.e., the tree below proves $\varphi \wedge \psi \rightarrow \chi$ from the premise $\varphi \rightarrow \psi \rightarrow \chi$. However, the tree now starts with the assumption that the conclusion does not follow from the premise. Thus the tree starts with the premise on the left hand side (the true side), and the conclusion on the right hand side (the false side). The proof then proceeds by breaking the formulas into their constituent parts according to the rules above. A branch closes (indicated with two horizontal lines), when somewhere in that branch the same formula occurs at the left-hand side as well as at the right-hand side. That is, a branch closes when it represents an impossible situation. In the case of this example all branches close, i.e., the tree as a whole closes. Hence, the starting assumption that the premise is true and the conclusion is false is not possible. In other words, whenever the premise is true, the conclusion must also be true. Thus the premise implies the conclusion, which was to be proved.



8.3 Predicate Logic

Predicate logic assumes that the world consists of individual objects which may have certain properties and between which certain relations may hold (the general name for a property or a relation is *predicate*). Besides, there are operations which may be performed on these objects, the result of which is an object in the world again. For example, if the objects in the world are individual human beings, then there might be predicates like “male”, “female”, “older-than”, “married-to”. Note that the first two examples are about one person, the others about two. An example of an operation on human beings is “father-of”. Applying this operation to an individual delivers the father of that individual.

Distinguishing separate objects in the world gives the possibility to quantify over these objects, i.e., to make statements about all or some objects. In higher order predicate logic it is also possible to speak about all (or some) sets of objects, such as all families, some school classes. We restrict ourselves to first order logic in which one can only quantify over the individual objects themselves.

8.3.1 Formal Language Definition

People usually reason about specific topics, so in order to formulate sentences about such a topic in a formal language, one first has to fix the *alphabet* of the language:

- a *constant* (a, b, c,) is intended to denote a specific individual object
- a *function symbol* (f, g, h,) denotes an operation that may be performed on (sequences of) individual objects to yield another object
- a *predicate symbol* (P, Q, R,) denotes a property or relation that holds for (sequences of) individual objects.

Every function and predicate symbol has an *arity*, indicating how many arguments it requires.

The alphabet determines a language which is intended to express facts about a given world. In addition, every language of predicate logic contains *variables* which denote individual objects. The difference between a variable and a constant is that a constant denotes a fixed object, whereas the object denoted by a variable may vary (not within the same context or sentence, though).

The formal language of predicate logic consists of two parts: *terms* are intended to denote objects in the world; *propositions* are intended to express facts about these objects. Both terms and propositions are defined recursively:

Terms:

- Variables and constants are terms
- if f is an n -ary function symbol, and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term too
- nothing else is a term.

Propositions:

- if P is an n -ary predicate symbol, and $t_1, \dots, t_n$ are terms, then the expression $P(t_1, \dots, t_n)$ is an (atomic) proposition
- if φ is a proposition, then $\neg \varphi$ is a proposition too,
- if φ, ψ are propositions, then $\varphi \vee \psi, \varphi \wedge \psi, \varphi \rightarrow \psi, \varphi \leftrightarrow \psi$ are propositions too,
- if x is a variable, and φ is a proposition, then $\forall x \varphi$ and $\exists x \varphi$ are propositions too,
- nothing else is a proposition.

The symbols $\forall$ and $\exists$ are quantifiers, called the universal quantifier (for “for all”) and the existential quantifier (“for some”, i.e. “at least one”), respectively.

Some examples of propositions are:

$$(\forall x P(x)) \wedge (\exists y Q(y)), \quad \forall x P(x) \rightarrow \exists y Q(x, y, z), \quad \forall x P(x) \wedge \exists x Q(x)$$

When a variable x is inside the “scope” of a quantifier with the same variable x , then x is said to be *bound* by that quantifier. In the second example above x is bound by the quantifier $\forall x$, y is bound by $\exists y$, and z is not bound at all. The variable z is said to occur *free* in this proposition. Note that in the third example the x in $P(x)$ is bound by $\forall x$, whereas the x in $Q(x)$ is bound by $\exists x$. Outside the scope of a quantifier which binds a variable x , this variable x is not meaningful.

With respect to the use of brackets it is agreed that the scope of quantifiers extends to the right as far as possible.

As a more concrete example, take normal arithmetic with the following alphabet (we restrict ourselves to just a few of the standard symbols): 0, 1, 2, etc., are constants, + and * are two-place function symbols, > and = are two-place predicate symbols. Using infix notation, the following expressions are propositions according to the definitions above:

$$\begin{aligned} &\forall x (x + 1 > x), \forall x \forall y (x + y = y + x), \\ &\forall x \forall y \forall z (x * (y + z) > (x * y) + (x * z)), \\ &\forall x \forall y (x * y = 0 \rightarrow x = 0 \vee y = 0). \end{aligned}$$

These examples express true or false propositions about normal arithmetic. The following examples are expressions which are not correctly formulated according to the above syntactical rules:

$$\forall \exists y f(x(= gy), P(Q(a) \wedge R(b))).$$

8.4 Substitution

The proposition

$$\forall x \forall y (x + y = y + x)$$

is a true formula of arithmetic. One of the rules of natural deduction for predicate logic says that then

$$\forall y (2 + y = y + 2) \quad \text{and} \quad 2 + 3 = 3 + 2$$

also are true propositions of arithmetic. These formulas are obtained from the first one by removing the leading universal quantifier, and then by *substitution* of 2 for x and 3 for y . Substitution is a tricky operation, for example

$$\forall x \exists y (y > x)$$

is a true proposition, but substituting y for x in

$$\exists y (y > x)$$

yields

$$\exists y (y > y)$$

which is a false formula. The problem here is that the substituted y is bound by the existential quantifier, whereas before the substitution x was not bound by that quantifier.

Substituting a term t for a variable x in an expression e is written as $e[x:=t]$ and formally defined as follows (recursively, following the definition of terms and propositions):

- $a[x:=t] \equiv a$,
- $x[x:=t] \equiv t$,
- $y[x:=t] \equiv y$,
- $f(t_1, \dots, t_n)[x:=t] \equiv f(t_1[x:=t], \dots, t_n[x:=t])$,
- $P(t_1, \dots, t_n)[x:=t] \equiv P(t_1[x:=t], \dots, t_n[x:=t])$,
- $(\neg \varphi)[x:=t] \equiv \neg(\varphi[x:=t])$,

likewise for the other propositional connectives,

- $(\forall x \varphi)[x:=t] \equiv \forall x \varphi$,
- $(\forall y \varphi)[x:=t] \equiv \forall y' (\varphi' [x:=t])$, where $\varphi' \equiv \varphi [y:=y']$, y' not in t or φ ,
- *ibid* for the existential quantifier.

In this definition, $\equiv$ stands for syntactical equality. It is assumed that a is a constant, and x, y are (different) variables.

Remark: It is amazing how complex the definition of something so simple as substitution is. It is only given for completeness' sake – don't try to learn it by heart. It formulates what you would expect in practical situations anyway, but when a computer has to perform a substitution correctly, it is absolutely necessary to formulate the definition explicitly.

The only thing that should be carefully watched is whether the binding pattern of a proposition remains the same after some substitution.

8.5 Semantics

The syntax of a language is concerned with formulating expressions in the language correctly, semantics deals with the meaning of the expressions. Since the formal syntactical definition considers expression as abstract objects, which have no meaning by themselves, semantics can only be given to expressions by a function which maps syntactical objects to objects in some real world. A world consists of:

- a universe U of objects,
- a collection P of predicates on U , i.e., subsets of (ordered n -tuples of)

objects of U ,

- a collection F of functions on U , i.e., subsets of ordered $(n+1)$ -tuples of objects in U . Notice that the $(n+1)$ -st element of those tuples must be uniquely determined, given the first n elements of the tuple.

An assignment function α is a function from the language to the world, and maps every individual variable to an object in U .

An interpretation function I also is a function from the language to the world, and maps every individual constant to an object in U , every predicate symbol to a predicate in P , and every function symbol to a function in F .

A *valuation function* V is a function which assigns objects in U to terms, and which assigns truth values to propositions.

Let an assignment function α and an interpretation function I be given. Then the corresponding valuation function V is defined for terms as:

- $V(x) = \alpha(x)$ for every individual variable x ,
- $V(a) = I(a)$ for every individual constant a ,
- $V(f(t_1, \dots, t_n)) = d$ if $(V(t_1), \dots, V(t_n), d)$ is in $I(f)$.

For propositions the valuation function is defined as follows:

- $V(P(t_1, \dots, t_n)) = 1$ if $(V(t_1), \dots, V(t_n))$ is in $I(P)$,
- for combinations of propositions by means of the propositional connectives the valuation function is defined exactly as in proposition logic (by the truth tables),
- $V(\forall x. \varphi) = 1$ if for all d in U we have that $V(\varphi) = 1$, where α is replaced by $\alpha[x \mapsto d]$,
- $V(\exists x. \varphi) = 1$ if there is some d in U for which $V(\varphi) = 1$, where α is replaced by $\alpha[x \mapsto d]$,

The notation $\alpha[x \mapsto d]$ denotes a function which does the same as α , except for the argument x . The result for x is d .

If a formula φ is true in a world W for some given assignment function α and interpretation function I , we write $\langle W, I \rangle \models_{\alpha} \varphi$.

A formula φ “follows from” a theory Σ (i.e. a set of propositions) if for every W, I, α such that all formulas in Σ are true, we also have that φ is true. We write $\Sigma \models \varphi$.

Some technical names for this form of “follows from” are *entailment*, *semantic consequence*.

Formulated somewhat sloppy, $\Sigma \models \varphi$ says that in any world where the theory Σ holds, φ must also hold.

Remark: The same remark as with substitution applies for the definition of semantics: for humans the meaning of a predicate logic expression is more or less clear by itself (provided a person has a sufficient amount of experience). But in the context of a computer, an expression becomes meaningless, and meaning has to be given explicitly by the mathematical function V .

Natural deduction for predicate logic

For the propositional connectives in predicate logic, the rules are the same as before. In the quantifier rules we write $\varphi(x)$ to indicate that the variable x may occur free in the formula φ . Then $\varphi(a)$, $\varphi(t)$ are the results of substituting a , t (respectively) for x at all relevant positions in φ . Here we intend a to be a name, i.e., a variable or a constant, and t to be an arbitrary term.

For the rules with the quantifiers we need a little care. These extra precautions have to do with the following two points:

- Substitution: It may occur that after substitution some unwanted bindings of variables to quantifiers arise. Take, for example, the proposition $\varphi(x) \equiv \exists y. y > x$, and consider $\varphi(y)$.

- Arbitrariness: Names have to be arbitrary, i.e., nothing may be assumed about those names. Technically speaking, such names may not occur free in any of the assumptions.

Exercise: Formulate these restrictions more carefully and precisely.

$$\begin{array}{c|c}
 \frac{\varphi(t)}{\exists x. \varphi(x)} (\exists I) & \frac{\begin{array}{c} [\varphi(a)] \\ \vdots \\ \psi \end{array}}{\exists x. \varphi(x)} (\exists E) \\
 \frac{\varphi(a)}{\forall x. \varphi(x)} (\forall I) & \frac{\forall x. \varphi(x)}{\varphi(t)} (\forall E)
 \end{array}$$

If a formula φ can be proved from some theory Σ by rules of natural deduction, we write

$$\Sigma \vdash \varphi$$

There is the important completeness theorem:

$$\Sigma \models \varphi \Leftrightarrow \Sigma \vdash \varphi.$$

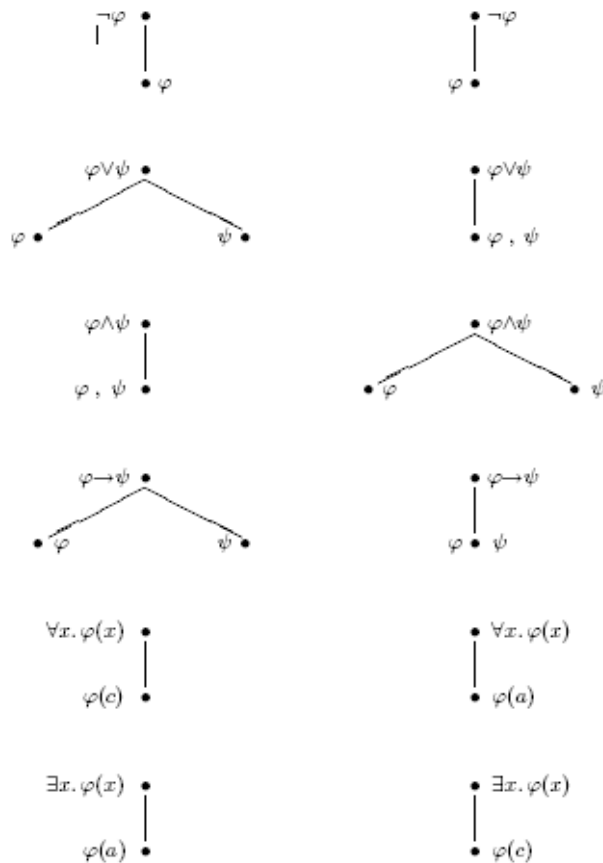
From left to right this is called “soundness”, expressing that with the proof rules no false conclusions from true theories can be drawn. From right to left is “completeness” proper: it says that all semantic consequences of a theory can be proved by the proof rules.

8.6 Semantic tableaux

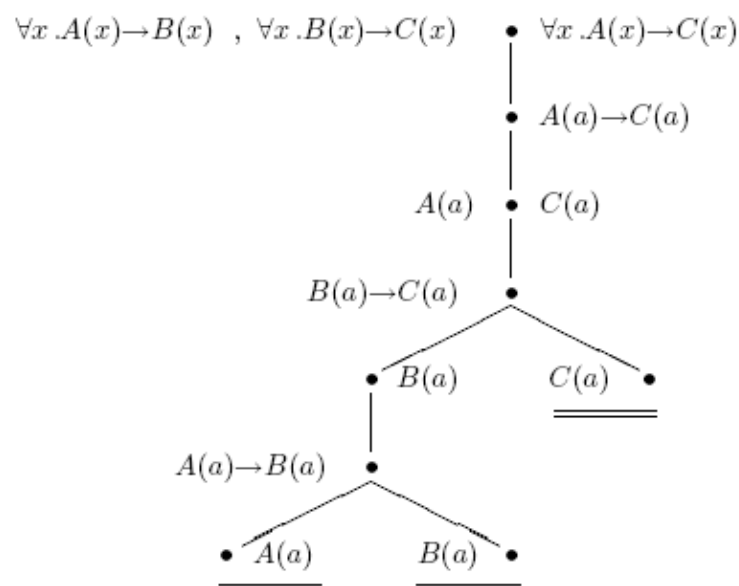
In natural deduction a proof is “built up” from the assumptions towards the conclusion. With a semantic tableau we proceed the other way, i.e., we “breakdown” the formulas. Hence, the method of semantic tableaux is mechanic in nature, whereas the method of natural deduction requires some creativity and understanding.

A semantic tableau is a tree in which every node consists of a bullet with some propositions on its left-hand and right-hand side. Formulas on the left-hand side of the bullet are supposed to be true, on the right-hand side they are supposed to be false. For every logical connective there is a left rule and a right rule, saying that if a compound formula is true/false, then a certain truth value follows for the constituents of the compound formula. Sometimes there are a few possibilities, indicated by a split of the rule.

With the quantifier rules some names are introduced: a name c may be any of the previously introduced names, name a is supposed to be new.



Example: The proof below assumes $\forall x. A(x) \rightarrow B(x)$ and $\forall x. B(x) \rightarrow C(x)$, and proves $\forall x. A(x) \rightarrow C(x)$. The technique of proving is the same as with proposition logic, i.e., let the assumptions be true, and suppose that the formula that has to be proved is false. Then all formulas are broken down according to the rules of semantic tableau, yielding various paths in a tree. If a path contains the same formula on the left (i.e., the true side) and on the right (the false side), then a contradiction arises and the path is closed (indicated by a double horizontal line). If all paths are closed, the proof is complete. If there remains an open path (i.e., a path which is not closed), there is no proof, and that open path provides a counter-example to the proposition to be “proven”. In the example below all paths are closed.



Summary

- Modern logic does so in a formal mathematical way, hence names like “symbolic logic”, “formal logic”, “mathematical logic”.
- In *proposition logic* we can express statements as a whole, and combinations of them.
- In *first order predicate logic* a statement has a specific inner structure, consisting of terms and predicates.
- A *proposition* is a statement of language that formulates something about an external world.
- The question whether a proposition is true or false depends on the truth or falsity of the elementary propositions and the way in which they are combined by the connectives.
- A semantic tableau is a tree in which every node consists of a bullet with some propositions on its left-hand and right-hand side.
- Predicate logic assumes that the world consists of individual objects which may have certain properties and between which certain relations may hold (the general name for a property or a relation is *predicate*).
- The formal language of predicate logic consists of two parts: *terms* are intended to denote objects in the world; *propositions* are intended to express facts about these objects.
- An *assignment function* α is a function from the language to the world, and maps every individual variable to an object in U .
- An *interpretation function* I also is a function from the language to the world, and maps every individual constant to an object in U , every predicate symbol to a predicate in P , and every function symbol to a function in F .
- A *valuation function* V is a function which assigns objects in U to terms, and which assigns truth values to propositions.
- A proposition can either be true or false.
- A semantic tableau is a tree in which every node consists of a bullet with some propositions on its left-hand and right-hand side.

References

- Epstein, L. R., 2011. *Classical Mathematical Logic: The Semantic Foundations of Logic*, Princeton University Press.
- Keith, D., *The Essence of Expert Systems*, Pearson Education India.
- Kuper, J., *Overview of proposition and predicate logic* [Pdf] Available at: <<http://wwwhome.ewi.utwente.nl/~infrieks/MHMI/2005.jk.pdf>> [Accessed 4 June 2013].
- 10. *Predicate Logic* [Pdf] Available at: <<http://www.cs.hmc.edu/~keller/cs60book/10%20Predicate%20Logic.pdf>> [Accessed 4 June 2013].
- *Propositional Logic I: Propositions, Assertions, Truth Values* [Video online] Available at: <<http://www.youtube.com/watch?v=re8WW5IFmsg>> [Accessed 4 June 2013].
- 8.1 *Predicate Logic: Symbols & Translation* [Video online] Available at: <<http://www.youtube.com/watch?v=GgHWvJsR3XE>> [Accessed 4 June 2013].

Recommended Reading

- Lee, H. K., 2005. *First Course on Fuzzy Theory and Applications*, Springer.
- Konar, A., *Artificial Intelligence and Soft Computing: Behavioral and Cognitive Modeling of the Human Brain, Volume 1*, CRC Press.
- Jamshidi, M., Vadiiee, N. and Ross, T., 1993. *Fuzzy Logic and Control: Software and Hardware Applications, Volume 2*, Pearson Education.

Self Assessment

1. In first order predicate logic a statement has a specific inner structure, consisting of terms and _____.
 - a. predicates
 - b. propositions
 - c. systems
 - d. logics
2. A _____ is a statement of language that formulates something about an external world.
 - a. predicates
 - b. proposition
 - c. system
 - d. logic

3. Match the following

1. Disjunction	A. $\neg$
2. Implication	B. $\vee$
3. Negation	C. $\wedge$
4. Conjunction	D. $\rightarrow$

- a. 1-C, 2-D, 3-A, 4-B
 - b. 1-D, 2-A, 3-B, 4-C
 - c. 1-A, 2-B, 3-C, 4-D
 - d. 1-B, 2-D, 3-A, 4-C
4. What does the symbol ' $\perp$ ' is called?
 - a. Between
 - b. Intuitionistic
 - c. Falsum
 - d. Axiom
5. A _____ is a tree in which every node consists of a bullet with some propositions on its left-hand and right-hand side.
 - a. breakdown
 - b. semantic tableau
 - c. natural deduction
 - d. logical connective
6. The _____ determines a language which is intended to express facts about a given world.
 - a. alphabet
 - b. symbol
 - c. number
 - d. proposition

7. If P is an n -ary predicate symbol, and $t_1, \dots, t_n$ are terms, then the expression $P(t_1, \dots, t_n)$ is an _____.
a. existential quantifier
b. universal quantifier
c. atomic proposition
d. proposition
8. The logical approach includes the expression of _____ in a formal language, and second the application of reasoning rules to these formal representations.
a. human knowledge
b. formal character
c. computer
d. human reasoning
9. A _____ in the semantic tableau method starts by stating that all the premises are true, but the conclusion that has to be proved, is not true.
a. predicate
b. tree
c. logic
d. symbol
10. The formal language of predicate logic contains the terms which are _____.
a. leading universal quantifier
b. two-place predicate symbols
c. variables and constants
d. intended to denote objects in the world

Application I

Recognising Behavioural Patterns at Runtime using Finite Automata

Abstract

During reverse engineering, developers are often required to understand the undocumented design of software. In particular, recognising design patterns in the software can provide reverse engineers with considerable insight on the software structure and its internal characteristics. Researchers have therefore proposed techniques based on static analysis to automatically recover design patterns in a program. Unfortunately, most design patterns comprise not only structural, but also significant behavioral aspects. Although static analysis is well suited for the recognition of structural aspects, it is typically limited and imprecise in analysing behavior. To address this limitation, a new technique is presented that complements our existing static analysis with a dynamic analysis, so as to perform more accurate design pattern recognition. The dynamic analysis is based on:

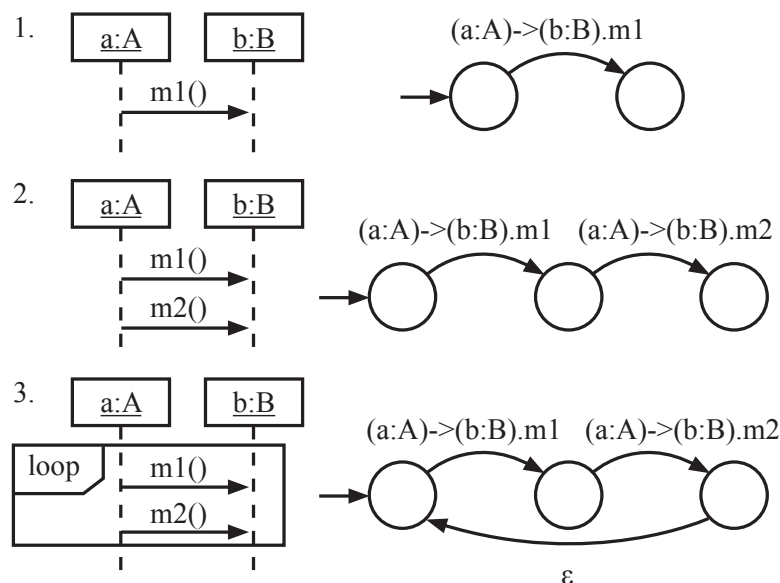
- Transforming behavioral aspects of design patterns into finite automata,
- Identifying and instrumenting relevant method calls
- Monitoring relevant calls at runtime and matching them against the automata.

The results of the dynamic analysis are then used to compute the likelihood of a pattern to be in the code. This case study describes our technique and presents a preliminary empirical study performed to assess the technique.

From behavioral patterns to finite automata

Behavioral patterns can be interpreted as regular expressions. The symbols used in these regular expressions consist of a combination of the method call, its caller, and callee. We use finite automata to match the regular expressions. In this section, we describe how behavioral patterns are transformed into Deterministic Finite Automata (DFA) that get method call events as input and check method traces for their conformance to the behavioral patterns.

To generate DFAs, we define, for each syntactic element of a behavioral pattern (e.g., method call, optional, and alternative fragment), a transformation into parts of a Non-Deterministic Finite Automaton (NFA). The complete NFA for a behavioral pattern is constructed by combining the NFA parts for each element of the pattern. The NFA is then transformed into a DFA. For the sake of space, we do not present the complete set of transformations for all syntactical elements. Instead, we illustrate a few of these transformations to give the reader the intuition of how our transformation works as shown in below figure. Specifically, we show how we (1) transform a method call that occurs in a behavioral pattern into its corresponding part of an NFA, (2) combine two partial NFAs that correspond to two consecutive calls and (3) transform a loop into its corresponding NFA fragment.



Method calls are the simplest syntactic element in a behavioral pattern. We transform a method call into two states and a transition between them, as shown in Figure 5(1). The symbol accepted by the transition consists of the concatenation of variables derived from the behavioral pattern: the caller's object, (a: A), a call symbol, “->”, the callee's object, (b: B), and the called method, m1 (the latter two separated by a dot). The variables will be bound to concrete values incrementally during the dynamic analysis.

(Source: Wendehals, L., 2006. *Recognizing Behavioral Patterns at Runtime using Finite Automata* [Pdf] Available at: <<http://www.cc.gatech.edu/~orso/papers/wendehals.orso.WODA06.pdf>> [Accessed 7 June 2013]).

Questions

1. What are the limitations of static analysis and what technique is presented to overcome this limitation?

Answer

Static analysis is well suited for the recognition of structural aspects; it is typically limited and imprecise in analysing behavior. To address this limitation, a new technique is presented that complements the existing static analysis with a dynamic analysis, so as to perform more accurate design pattern recognition.

2. Dynamic analysis is based on which points?

Answer

The dynamic analysis is based on:

- Transforming behavioral aspects of design patterns into finite automata
- Identifying and instrumenting relevant method calls
- Monitoring relevant calls at runtime and matching them against the automata.

3. What is Deterministic Finite Automata (DFA)?

Answers

A deterministic finite automaton M is a quintuple (S, A, δ, s_1, F)

Where:

S = finite set of states,

A = alphabet,

$s_1 \in S$ is the initial state,

$F \subseteq S$ is the set of final states, and

The transition function $\delta: S \times A \rightarrow S$, is defined such that $\forall s \in S, \forall c \in A, \delta(s, c) \in S$ is uniquely determined.

Application II

Automata Theory in Biology

Automata theory is the basis for the theory of formal languages. A proper treatment of formal language theory begins with some basic definitions:

- A symbol is simply a character, an abstraction that is meaningless by itself.
- An alphabet is a finite set of symbols.
- A word is a finite string of symbols from a given alphabet.
- Finally, a language is a set of words formed from a given alphabet.

To the casual observer, biology is an impossibly complex science. Traditionally, the intricacy and variation found in life science has been attributed to the notion of natural selection. Species become “intentionally” complex because it increases their chance for survival. For example, a camouflage-patterned toad will have a far lower risk of being eaten by a python than a frog colored entirely in orange. This idea makes sense, but automata theory offers a simpler and more logical explanation, one that relies not on random, optimising mutations but on a simple set of rules.

Basic automata theory shows that simplicity can naturally generate complexity. Apparent randomness in a system results only from inherent complexities in the behavior of automata, and seemingly endless variations in outcome are only the products of different initial states. A simple mathematical example of this notion is found in irrational numbers. The square root of nine is just 3, but the square root of ten has no definable characteristics. One could compute the decimal digits for the lifetime of the universe and never find any kind of recurring pattern or orderly progression; instead, the sequence of numbers seems utterly random. Similar results are found in simple two-dimensional cellular automaton. These structures form gaskets and fractals that sometimes appear orderly and geometric, but can resemble random noise without adding any states or instructions to the set of production rules.

The most classic merging of automata theory and biology is John Conway’s Game of Life. “Life” is probably the most frequently written program in elementary computer science. The basic structure of Life is a two-dimensional cellular automaton that is given a start state of any number of filled cells. Each time step, or generation, switches cells on or off depending on the state of the cells that surround it. The rules are defined as follows:

- All eight of the cells surrounding the current one are checked to see if they are on or not.
- Any cells that are on are counted, and this count is then used to determine what will happen to the current cell:
 - Death: if the count is less than 2 or greater than 3, the current cell is switched off.
 - Survival: if (a) the count is exactly 2, or (b) the count is exactly 3 and the current cell is on, the current cell is left unchanged.
 - Birth: if the current cell is off and the count is exactly 3, the current cell is switched on.

Like any manifestation of automata theory, the Game of Life can be defined using extremely simple and concise rules, but can produce incredibly complex and intricate patterns.

In addition to the species-level complexity illustrated by the Game of Life, complexity within an individual organism can also be explained using automata theory. An organism might be complex in its full form, but examining constituent parts reveals consistency, symmetry, and patterns. Simple organisms, like maple leaves and star fish, even suggest mathematical structure in their full form. Using ideas of automata theory as a basis for generating the wide variety of life forms we see today, it becomes easier to think that sets of mathematical rules might be responsible for the complexity we notice every day.

Inter-species observations also support the notion of automata theory instead of the specific and random optimisation in natural selection. For example, there are striking similarities in patterns between very different organisms:

- Mollusks and pine cones grow by the Fibonacci sequence, reproducible by math.
- Leopards and snakes can have nearly identical pigmentation patterns, reproducible by two-dimensional automata.

With these ideas in mind, it is difficult not to imagine that any biological attribute can be simulated with abstract machines and reduced to a more manageable level of simplicity.

(Source: <http://www-cs-faculty.stanford.edu/~eroberts/courses/soco/projects/2004-05/automata-theory/apps.html#biology>).

Questions

1. Define automata.
2. Enlist the applications of automata theory.
3. How is automata theory applicable in biology?

Application III

Regular expressions in Unix

In the UNIX operating system, various commands use an extended regular expressions language that provides shorthand for many common expressions. In this, we can write character classes (A character class is a pattern that defines a set of characters and matches exactly one character from that set.) to represent large set of characters. There are some rules for forming this character classes:

The dot symbol (.) is to represent 'any character'

The regular expression $a+b+c+\dots+z$ is represented by $[abc\dots z]$.

Within a character class representation, - can be used to define a set of characters in terms of a range. For example, $a-z$ defines the set of lower-case letters and $A-Z$ defines the set of upper-case letters. The endpoints of a range may be specified in either order (i.e. both $0-9$ and $9-0$ define the set of digits).

If our expression involves operators such as minus then we can place it first or last to avoid confusion with the range specifier, i.e., $[-0-9]$. The special characters in UNIX regular language can be represented as characters using \ symbol, i.e., \ provides the usual escapes within character class brackets. Thus $[[\]]$ matches either [or], because \ causes the first] in the character class representation to be taken as a normal character rather than the closing bracket of the representation.

Special notations

$[:digit:]$	-	Same as $[0-9]$
$[:alpha:]$	-	same as $[A-Za-z]$
$[:alnum:]$	-	Same as $[A-Za-z0-9]$

Operators

	-	Used in place of +
?	-	0 or 1 of
		$R?$ Means 0 or 1 occurrence of R
+	-	1 or more of
		$R+$ means 1 or more occurrence of R
{n}	-	n copies of
		$R\{3\}$ means 1 or more occurrence of R
^	-	Compliment of

If the first character after the opening bracket of a character class is ^, the set defined by the remainder of the class is complemented with respect to the computer's character set. Using this notation, the character class represented by '.' can be described as $[\^n]$. If ^ appears as any character of a class except the first, it is not considered to be an operator. Thus, $[\^abc]$ matches any character except a, b, or c but $[a^\wedge bc]$ or $[abc^\wedge]$ matches a, b, c or ^.

When more than one expression can match the current character sequence, a choice is made as follows:

1. The longest match is preferred.
2. Among the rules which match the same number of characters, the rule given first is preferred.

(Source: <http://www.mec.ac.in/resources/notes/notes/automata/Re%20application.htm>).

Questions

1. Define Regular expressions.
2. Explain how Regular expressions can be used in UNIX.
3. What are the rules for forming the character classes?

Bibliography

References

- 10. *Predicate Logic* [Pdf] Available at: <<http://www.cs.hmc.edu/~keller/cs60book/10%20Predicate%20Logic.pdf>> [Accessed 4 June 2013].
- 2010. *Lecture 1-Finite State Machine* [Video online] Available at: <<https://www.youtube.com/watch?v=HyUK5RAJglc>> [Accessed 4 June 2013].
- 2010. *Context-Free Grammars* [Video online] Available at: <<https://www.youtube.com/watch?v=s0jD8bfvHLg>> [Accessed 7 June 2013].
- 2011. *Lec -26 Turing Machines* [Video online] Available at: <<https://www.youtube.com/watch?v=IhyEGNn-7Uo>> [Accessed 4 June 2013].
- 2011. *Lec-20 Pushdown Automata* [Video online] Available at: <<https://www.youtube.com/watch?v=8O6FUFwWt5A>> [Accessed 7 June 2013].
- 8.1 *Predicate Logic: Symbols & Translation* [Video online] Available at: <<http://www.youtube.com/watch?v=GgHWvJsR3XE>> [Accessed 4 June 2013].
- *Automata Theory : Context Free Grammar (CFG) to Chomsky Normal Form (CNF) part 1* [Video online] Available at: <<https://www.youtube.com/watch?v=zCmGPUZM-RE>> [Accessed 7 June 2013].
- *Context Free Grammars* [Pdf] Available at: <<http://people.cs.umass.edu/~mccallum/courses/inlp2007/lect5-cfg.pdf>> [Accessed 7 June 2013].
- *Context-free grammars and languages* [Pdf] Available at: <<http://www.cs.bath.ac.uk/~cf/cm10020/lecture07.pdf>> [Accessed 7 June 2013].
- *Deterministic Finite Automata* [Video online] Available at: <https://www.youtube.com/watch?v=3GxTDsQFY_w> [Accessed 7 June 2013].
- *DFA Deterministic finite state automaton* [Video online] Available at: <<https://www.youtube.com/watch?v=FNyRkNzOFXs>> [Accessed 7 June 2013].
- Diaz, M., 2010. *Petri Nets: Fundamental Models, Verification and Applications*, John Wiley & Sons.
- Epstein, L. R., 2011. *Classical Mathematical Logic: The Semantic Foundations of Logic*, Princeton University Press.
- *Finite Automata* [Pdf] Available at: <<http://www.eecs.wsu.edu/~ananth/CptS317/Lectures/FiniteAutomata.pdf>> [Accessed 7 June 2013].
- *Finite Automata* [Video online] Available at: <<https://www.youtube.com/watch?v=-xkcs-t99Do>> [Accessed 7 June 2013].
- *Finite State Machines* [Pdf] Available at: <<http://www.cse.chalmers.se/~coquand/AUTOMATA/book.pdf>> [Assessed 30 May 2013].
- *Finite State Machines* [Pdf] Available at: <http://www.spinroot.com/spin/Doc/Book91_PDF/F8.pdf> [Assessed 30 May 2013].
- *Finite-State Machines* [Pdf] Available at: <<http://www.cse.chalmers.se/~coquand/AUTOMATA/book.pdf>> [Accessed 7 June 2013].
- *Finite-State Machines and Pushdown Automata* [Pdf] Available at: <http://cs.brown.edu/~jes/book/pdfs/ModelsOfComputation_Chapter4.pdf> [Accessed 7 June 2013].
- Hopcroft, J. E., 2008. *Introduction to Automated Theory, Languages and Computation*, Pearson Education India.
- *Introduction-Finite Automata* [Pdf] Available at: <<http://users.ece.utexas.edu/~adnan/verif-04/fsa.pdf>> [Accessed 7 June 2013].
- Kam, T., 1996. *Synthesis of Finite State Machines: Functional Optimization*, Springer.
- Keith, D., *The Essence of Expert Systems*, Pearson Education India.

- Krithivasan, K. 2009. *Introduction to Formal Languages, Automata Theory and Computation*, Dorling Kindersley (India) Pvt.Ltd.
- Kumar, R. 2011. *Lecture on Push Down Automata* [Video online] Available at: <<https://www.youtube.com/watch?v=7GZi7tk-mvA>> [Accessed 7 June 2013].
- Kumar, R. 2010. *Theory of Automata, Languages and Computation*, Tata Mc Graw Hill Publication Private Limited.
- Kuper, J., *Overview of proposition and predicate logic* [Pdf] Available at: <<http://wwwhome.ewi.utwente.nl/~infrieks/MHMI/2005.jk.pdf>> [Accessed 4 June 2013].
- Lawson, M. V., *Finite automata* [Pdf] Available at: <<http://www.ma.hw.ac.uk/~markl/preprints/Lawson.pdf>> [Accessed 7 June 2013].
- *Lec-26 Turing Machine* [Video online] Available at: <<https://www.youtube.com/watch?v=IhyEGNn-7Uo>> [Accessed 7 June 2013].
- *Lecture 12-Turing Machines* [Video online] Available at: <<https://www.youtube.com/watch?v=mPec64RUCsk>> [Accessed 7 June 2013].
- *Lecture 38-Finite State Automaton* [Video online] Available at: <<https://www.youtube.com/watch?v=-K0cRsZSnLc>> [Accessed 7 June 2013].
- *Lecture 5-Context Free Languages* [Video online] Available at: <<https://www.youtube.com/watch?v=7axUgepqCFc>> [Accessed 7 June 2013].
- Meduna, A., 1999. *Automata and Languages: Theory and Applications*, Springer.
- Mukherjee, D., 2013. *Automata: Introduction to Grammar* [Video online] Available at: <<https://www.youtube.com/watch?v=sBmg9Nf06tg>> [Accessed 7 June 2013].
- Natarajan, A. M., 2005. *Theory of Automata and Formal Languages*, New Age International.
- Padma Reddy, A.M., 2011. *Finite Automata and Formal Languages: A Simple Approach*, Sanguine Technical Publishers.
- *Propositional Logic 1: Propositions, Assertions, Truth Values* [Video online] Available at: <<http://www.youtube.com/watch?v=re8WW5IFmsg>> [Accessed 4 June 2013].
- Puntambekar, A.A., 2007. *Theory of Automata and Formal Languages*, Technical Publications.
- *Pushdown Automata* [Pdf] Available at: <<http://www.eecs.wsu.edu/~ananth/CptS317/Lectures/PDA.pdf>> [Accessed 7 June 2013].
- *Pushdown Automata* [Pdf] Available at: <<http://www.math.uaa.alaska.edu/~afkjm/cs351/handouts/pda.pdf>> [Accessed 7 June 2013].
- *Regular Languages* [Pdf] Available at: <<http://www.cs.uky.edu/~lewis/texts/theory/languages/reg-lang.pdf>> [Accessed 7 June 2013].
- *Regular Languages and Finite Automata* [Pdf] Available at: <<http://www.cl.cam.ac.uk/teaching/0910/RLFA/reglfa.pdf>> [Accessed 7 June 2013].
- Sharma, A., 2006. *Theory of Automata and Formal Languages*. Firewall Media.
- Simon, M., 1999. *Automata Theory*, World Scientific Publishing Co.Pte, Ltd.
- Sunitha, K. V. N. and Kalyani, N., 2010. *Formal Languages and Automated Theory*, Tata McGraw-Hill Education.
- *Turing Machines* [Pdf] Available at: <<http://www.cs.uky.edu/~lewis/texts/theory/computability/tur-mach.pdf>> [Accessed 7 June 2013].

Recommended Reading

- Agar, J., 1997. *Turing and the Universal Machine: The Making of the Modern Computer* (Revolutions of Science). Totem Books.
- Carlisle, M. C., 2009. *An Introduction to Languages and Machines*, CreateSpace.
- Dexter C. K., 1997. *Automata and Computability*. Springer

- Desel, J. and Esparza, J., 2005. *Free Choice Petri Nets*, Cambridge University Press.
- Dong, J., 2007. *Network Dictionary*, Javvin Technologies Inc.
- Davis, S. J., Davis, J. and Reese, B. R., 2008. *Finite State Machine Datapath Design, Optimization, and Implementation*, Morgan & Claypool Publishers.
- Herken, R., 1995. *The Universal Turing Machine*, 2nd ed., Springer.
- Higuera, C., 2010. *Grammatical Inference: Learning Automata and Grammars*, Cambridge University Press.
- Hopcroft, J. E., Motwani, R. and Jeffrey D. U., 2006. *Introduction to Automata Theory, Languages and Computation*. 3rd ed., Addison Wesley.
- Jamshidi, M., Vadiee, N. and Ross, T., 1993. *Fuzzy Logic and Control: Software and Hardware Applications, Volume 2*, Pearson Education.
- Kallmeyer, L., 2010. *Parsing Beyond Context-Free Grammars*, 1st ed., Springer.
- Konar, A., *Artificial Intelligence and Soft Computing: Behavioral and Cognitive Modeling of the Human Brain, Volume 1*, CRC Press.
- Lee, H. K., 2005. *First Course on Fuzzy Theory and Applications*, Springer.
- Linz, P., 2006. *An Introduction to Formal Language and Automata*, 4th ed., Jones & Bartlett Pub.
- Mark, V. L., 2003. *Finite Automata*, 1st ed., Chapman and Hall/CRC.
- Martin, J., 2002. *Introduction to Languages and the Theory of Computation*. McGraw-Hill Science/Engineering/Math, 3rd ed.
- Meduna, A., 2000. *Automata and Languages: Theory and Applications*, Springer.
- Revesz, G. E., 1991. *Introduction to Formal Languages*, Dover Publications.
- Rich, E. A., 2007. *Automata, Computability and Complexity: Theory and Applications, 1st ed.*, Prentice Hall.
- Shallit, J., 2008. *A Second Course in Formal Languages and Automata Theory*, 1st ed., Cambridge University Press.
- Simon, M., 1999. *Automata Theory*, 1st ed., World Scientific Publishing Company.
- Singh, A., 2009. *Elements of Computation Theory*, 1st ed., Springer.
- Webber, A., 2008. *Formal Language: A Practical Introduction*. Franklin, Beedle & Associates, Inc.
- Zvi, K., Jha, N. K., 2009. *Switching and Finite Automata Theory*, 3rd ed., Cambridge University Press.

Self Assessment Answers

Chapter I

1. a
2. c
3. a
4. d
5. b
6. b
7. c
8. c
9. b
10. a

Chapter II

1. a
2. b
3. c
4. a
5. b
6. b
7. c
8. d
9. d
10. a

Chapter III

1. c
2. a
3. d
4. b
5. c
6. a
7. d
8. a
9. b
10. a

Chapter IV

1. a
2. b
3. a
4. a
5. b
6. c
7. a
8. a
9. c
10. a

Chapter V

1. c
2. d
3. a
4. a
5. a
6. b
7. c
8. b
9. a
10. d

Chapter VI

1. a
2. b
3. c
4. d
5. a
6. b
7. c
8. d
9. a
10. b

Chapter VII

1. b
2. a
3. c
4. d
5. a
6. c
7. b
8. a
9. c
10. d

Chapter VIII

1. a
2. b
3. d
4. c
5. b
6. a
7. c
8. a
9. b
10. d